UNREAL ANIMATION • 游戏客户端秋招版

UE 动画系统：从骨骼到动画蓝图

从一根骨骼的 FTransform 讲起：参考姿势、局部空间与组件空间、蒙皮与压缩；再到四元数混合、混合空间、同步组、分层骨骼混合；然后拆开动画蓝图的线程模型、状态机、惯性化、蒙太奇与 IK；最后是 URO、LOD、并行求值这些线上真正要面对的性能问题。

前置白话篇 5 章 + 正文 10 章 + 附录

8 个交互实验（可拖动 / 可调参）

MSVC /O2 实测 12 组实验

40 道面试追问 + 速记卡

CHAPTER 00

先背这个：30 秒面试版答案

面试官说“讲讲 UE 的动画系统”，先用一分钟把全貌讲清楚，再等他挑一个点往下问。

问：讲讲 UE 的动画系统。

数据层：`USkeleton` 定义骨骼层级（按拓扑序存放：父骨骼下标总比子骨骼小）和插槽、曲线、同步标记这些“公共语言”；`USkeletalMesh` 是蒙皮网格，每个顶点绑定最多若干根骨骼和权重；`UAnimSequence` 存每根骨骼随时间变化的局部变换，经过压缩（UE 5.3 起默认 ACL）。混合空间、蒙太奇、Aim Offset 都是在这些序列之上再组织一层。

逻辑层：动画蓝图（`UAnimInstance`）分两张图。**EventGraph** 在游戏线程读角色状态、写变量；**AnimGraph** 是一棵节点树（序列播放器、混合空间、状态机、分层骨骼混合、IK……），每帧先 **Update**（推进时间、算权重），再 **Evaluate**（采样、混合，得到局部空间姿势，必要时转到组件空间做 IK）。这部分可以放到工作线程，跑在 `FAnimInstanceProxy` 上。

混合：位移线性插值，旋转用四元数加权求和后归一化（带最短路径符号修正），所以多路混合与顺序无关。混合空间用三角剖分 + 重心坐标算各采样权重；同步组 / 同步标记让不同长度的走跑循环对齐脚步；分层骨骼混合按骨骼分支设置权重，上半身做动作、下半身继续跑，旋转可以选在组件空间混合，防止上半身被骨盆带偏。

输出：组件空间姿势乘以参考姿势的逆矩阵得到蒙皮矩阵，交给渲染线程，在 GPU 上做线性混合蒙皮。**性能**主要靠：多线程更新与求值、Fast Path / 线程安全函数、URO 降低远处角色的更新频率、LOD 减骨、不可见时不刷新骨骼、主从姿势（Leader Pose）共享骨骼。

基础不牢？先读前置篇

如果“四元数、局部空间、蒙皮矩阵、重心坐标”这些词还说不清，先读紧接着的**前置篇 B1~B5**：B1 骨骼动画怎么让模型动起来，B2 变换与坐标空间（含 FK 手算），**B3 什么是四元数**（含两个交互实验），B4 插值、权重与混合，B5 名词白话词典。**附录 A** 对照 Notion 原笔记逐条给出需要修正的说法。

一帧里，动画系统在做什么

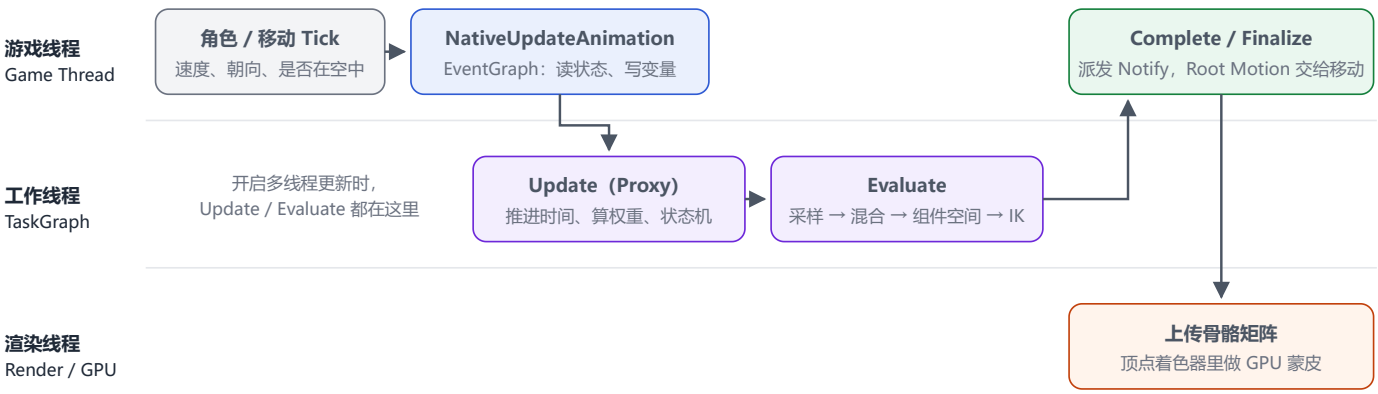


图 0-1　一帧中动画数据的流向：游戏线程准备变量 → 工作线程算出姿势 → 回到游戏线程收尾 → 渲染线程做蒙皮。第 6 章会把每一步对应到具体的函数

概念速查表

概念	是什么	一句话要点
Skeleton（骨架）	骨骼层级 + 插槽 + 曲线 / 同步标记名	动画资产绑定的是 Skeleton 而不是 Mesh；多个 Mesh 可共享一个 Skeleton
Reference Pose	骨骼的绑定姿势	蒙皮矩阵 = 当前组件空间变换 × 参考姿势组件空间变换的逆
局部 / 组件空间	相对父骨骼 / 相对 SkeletalMeshComponent	混合大多在局部空间做；IK、Mesh Space 混合在组件空间做
AnimSequence	每骨骼随时间的变换轨道	压缩：去常量轨、量化、关键帧精简；UE 5.3 起默认 ACL
Blend Space	按 1~2 个参数在多个采样之间混合	三角剖分 + 重心坐标；输入平滑与采样权重速度是两回事
Sync Group / Marker	让多条循环动画对齐	归一化时间同步 vs 同步标记（按左右脚落地对齐）
Layered Blend per Bone	按骨骼分支混合	Blend Depth 控制权重渐变；Mesh Space Rotation Blend 防止上半身被骨盆带偏
Additive / Aim Offset	“差值”动画	Additive = 动画 − 基准姿势；Aim Offset 是网格空间加性混合空间
State Machine	状态 + 过渡规则	交叉淡化要同时求值两边；惯性化只求值目标
Montage / Slot	代码触发的一次性动画	按 Slot 插入 AnimGraph；分 Section、带 Notify、可网络同步
AnimInstance / Proxy	动画蓝图的运行实例	EventGraph 在游戏线程；AnimGraph 可以在工作线程，靠 Proxy 隔离数据
URO / LOD / Budget	降频、减骨、按预算调度	远处角色隔几帧更新一次；LOD 只算主干骨骼

面试注意：最容易丢分的五句话

- “骨骼旋转混合直接对欧拉角插值”——**错。**Yaw 170° 和 −170° 各占一半，欧拉插值得到 0°（转了一整圈的反方向），正确是 180°（E2）。要用四元数，并且 dot < 0 时取反走最短路径。
- “分层混合上半身就用 Local Space”——不一定。下半身骨盆扭了 40° 时，Local Space 混合会让瞄准方向也偏 40°；勾选 **Mesh Space Rotation Blend** 才能保持上半身在组件空间的朝向（E4），代价是这一步慢一个数量级。
- “走和跑放进同一个混合空间就自然了”——不同步时两条动画有一半时间支撑脚相反（E6：50.4%），必须用同步组；脚步时机不一样时还要同步标记。
- “动画蓝图的 EventGraph 也在工作线程跑”——**错。**EventGraph（BlueprintUpdateAnimation）在游戏线程；多线程只覆盖 AnimGraph 的更新与求值，以及标记为线程安全的函数（BlueprintThreadSafeUpdateAnimation）。
- “惯性化就是更平滑的淡入淡出”——它的关键是**只求值目标姿势**，把切换瞬间的差值按五次多项式衰减掉（E7：求值次数减半，速度突变也更小）；源姿势在过渡开始后就不再计算，它的 Notify 也就不会再触发。

本书怎么读

前置篇 · 白话基础

骨骼动画原理、坐标空间与变换、四元数、插值与重心坐标、名词词典；零基础从这里开始。

第一部分 · 骨骼与动画数据

Skeleton / Mesh / Sequence 的关系；空间与 FK；蒙皮；动画压缩与误差累积。

第二部分 · 混合

四元数混合与加性动画；混合空间与同步组；分层骨骼混合与 Mesh Space。

第三部分 · 动画蓝图

AnimInstance、Proxy 与线程模型；状态机、惯性化、蒙太奇、Root Motion；IK 与 UE5 的新动画工具。

第四部分 · 性能与实战

URO、LOD、预算分配器、Leader Pose；网络同步、排错清单；实测总表与 40 道面试题。

附录 A · 对照原笔记

Notion 原笔记中 20 处需要修正或补充的说法，附依据和更稳妥的面试表述。

关于实测数据与版本

书中的“实测”全部来自 `demos/anim_demo.cpp`：不依赖 UE，用一套最小数学库复现动画管线里的关键算法（约定与 UE 一致：`A*B` 先 A 后 B，组件空间 = 局部 × 父组件空间，骨骼按拓扑序存放），在 Windows 11 + MSVC 14.44（`cl /std:c++latest /EHsc /O2`，x64，float）下运行，输出见 `demos/run_output.txt`。骨架仿 UE4 Mannequin 的 68 根骨骼。E11、E12 是前置篇的四元数与欧拉角实验。绝对耗时与机器有关，**请看相对关系。**

引擎细节以 UE 5.x 为准，涉及版本变化的地方会单独标出（如 5.1 起 Master Pose 改名为 Leader Pose，5.3 起 ACL 成为默认压缩编解码器、新增实验性的 Dead Blending 节点）。书中的混合空间、同步、分层混合、过渡曲线插图由构建脚本用和 demo 相同的算法实时计算生成。

PART ZERO

前置篇　用大白话把基础讲清楚

正文默认你知道“骨骼、矩阵、四元数、插值”是什么。这一篇不假设任何前置知识：先建立直觉，再给出每个概念最少需要的数学，配上手算例子和实测数字。读完再看第 1 章起的正文会轻松很多。

BASICS · B1

骨骼动画到底是怎么让模型动起来的

一句话：骨头动，皮跟着动。动画数据只记录“骨头怎么摆”，皮怎么变形交给显卡算。

本章目标

- 用自己的话说清：网格、骨骼、蒙皮权重、动画四样东西各管什么
- 知道为什么动画只存骨头、不存顶点
- 理解“姿势（Pose）”“关键帧”“插值”这几个最常出现的词
- 能按顺序说出一帧里动画系统做的七件事

B1.1 骨头动，皮跟着动

人的皮肉自己不会动，是骨头带着它们动。游戏角色也是这样做的：

1. 美术先做一层**“皮”**：由成千上万个三角形组成的模型，三角形的顶点就叫**顶点**。这层皮叫**网格（Mesh）**。
2. 在皮里面放一套**“骨头”**：几十到上百根骨骼，按身体结构连成一棵树（骨盆连脊柱，脊柱连肩膀，肩膀连上臂……）。这叫**骨骼 / 骨架（Skeleton）**。
3. 告诉引擎**每个顶点“粘”在哪几根骨头上、各粘多牢**。比如肘部的一个顶点：一半跟上臂走，一半跟前臂走。这组比例叫**蒙皮权重（Skin Weights）**，绑定这个过程叫**蒙皮（Skinning）**。
4. 做动画时，只记录**每根骨头在每个时刻怎么摆**。这就是**动画序列（Animation Sequence）**。

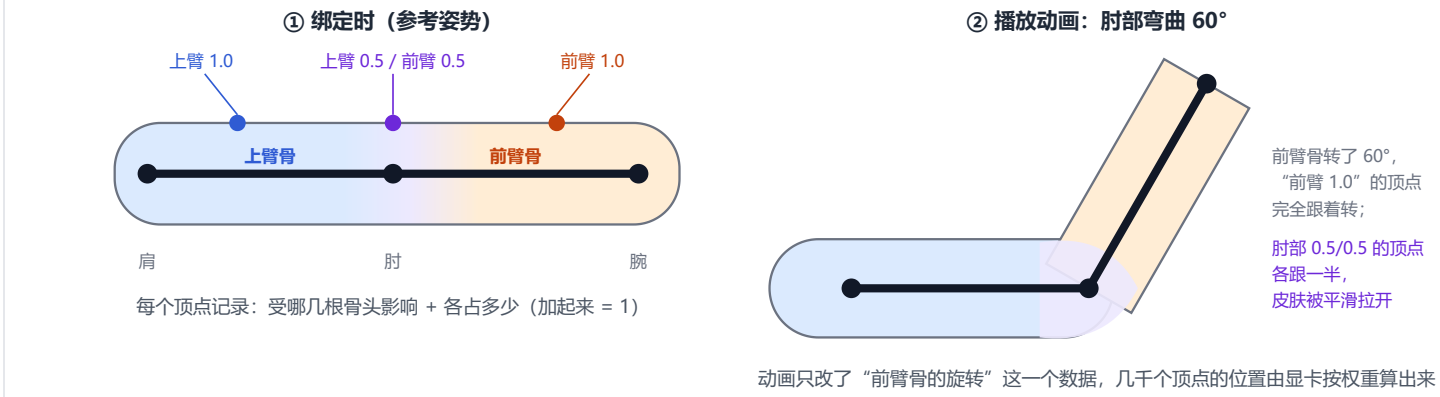


图 B1-1 骨骼、网格与蒙皮权重的关系。颜色表示顶点主要跟随哪根骨头：蓝 = 上臂，橙 = 前臂，紫 = 两者各一半

东西	白话	UE 里对应的资产 / 类	里面有什么
网格	皮	`USkeletalMesh`（骨骼网格体）	顶点、三角形、UV、材质、**蒙皮权重**、LOD
骨骼	骨头 + 它们的连接关系	`USkeleton`（骨架）	骨骼名、谁是谁的父亲、绑定时的初始摆法、Socket、曲线名
蒙皮权重	胶水：皮粘在哪根骨头上	存在 SkeletalMesh 里	每个顶点：若干个（骨骼编号，权重）
动画	骨头随时间的摆法	`UAnimSequence`（动画序列）	每根骨头每个时刻的位置 / 旋转 / 缩放，以及通知、曲线
运行实例	场景里“一个活的角色”	`USkeletalMeshComponent` + 动画蓝图	把网格和动画组合起来，每帧算出姿势并交给渲染

B1.2 为什么动画只记录骨头，不记录顶点

换个方式想：如果直接记录“每一帧每个顶点在哪”，数据量有多大?

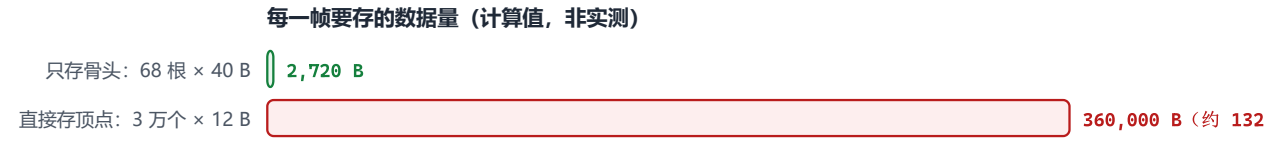


图 B1-2 一根骨头的摆法 = 旋转 16 B + 位置 12 B + 缩放 12 B；一个顶点的位置 = 12 B。只存骨头，数据小两个数量级

除了数据小，只存骨头还有三个好处：

- **可复用**：同一套骨头的所有模型（不同衣服、不同体型）都能播放同一条动画。顶点动画只属于某一个模型。
- **好混合**：走和跑各取一半，只需要混合 68 根骨头的摆法；混合 3 万个顶点则贵得多，而且没有“关节”的概念，混合结果容易变形。
- **好修改**：程序可以在运行时改某根骨头（让头看向敌人、让脚贴住台阶），顶点自动跟上。

例外：什么时候真的存顶点

面部表情常用 **Morph Target（变形目标）**：存“笑的时候每个顶点偏移多少”，用一个 0~1 的权重控制强度。远处成百上千的群众可以用 **顶点动画贴图（VAT）**：把顶点偏移烘焙进贴图，完全不跑骨骼动画。它们都是骨骼动画的补充，而不是替代。

B1.3 姿势、关键帧与插值

- **姿势（Pose）**：某一瞬间所有骨头的摆法，在代码里就是一个数组，每根骨头一个“变换”（位置 + 旋转 + 缩放，见 B2）。动画系统里的“混合”“修正”，都是在姿势这个数组上做运算。
- **关键帧（Key）**：动画数据不是连续的，而是按固定频率（常见 30 次/秒）存的一组姿势。
- **插值（Interpolation）**：游戏可能跑 120 帧/秒，比数据频率高。两个关键帧之间的姿势按时间比例“补”出来。例如肘部在第 10 帧弯 40°、第 11 帧弯 46°，当前时间落在两帧之间 1/4 处，就取 40 + (46 − 40) × 1/4 = 41.5°。所以动画数据是 30 帧，画面照样流畅。
- **骨骼是一棵树**：每根骨头只有一个父亲（根骨骼没有）。父亲动了，孩子全部跟着动：肩膀一转，肘、手、手指都跟着走。所以计算时必须**先算父亲、再算孩子**（B2 会手算一遍）。

B1.4 一帧里发生了什么（白话版）

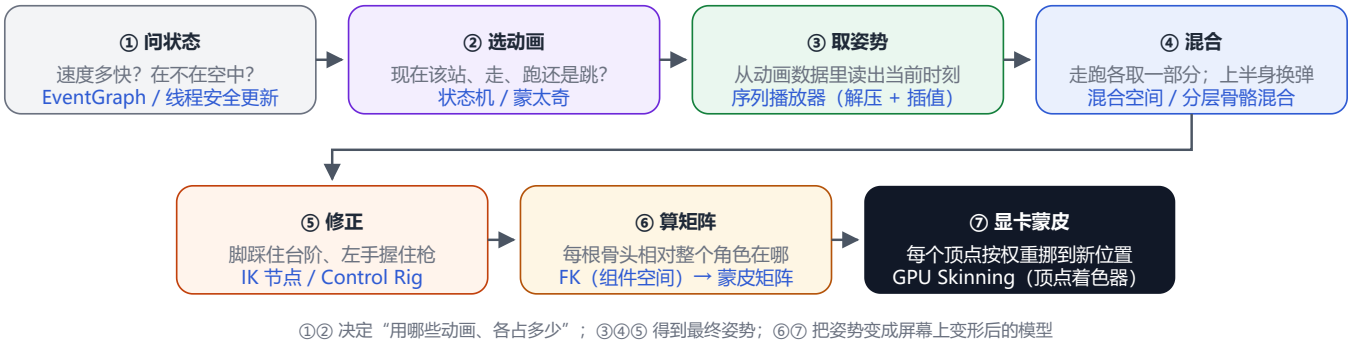


图 B1-3 一帧里动画系统做的七件事。蓝字是正文里对应的 UE 术语；第 6 章会把它们对应到具体函数和线程

面试注意：三句话区分三个资产

- **Skeleton 里没有顶点**：它只是骨头的名字、父子关系和初始摆法，是动画和网格之间的“共同语言”。
- **SkeletalMesh 里有蒙皮权重**：这是它和普通静态模型（StaticMesh）最大的区别。
- **AnimSequence 里没有顶点**：只有骨头随时间的变换，所以能被同骨架的所有模型复用。

本章速记

- 网格 = 皮，骨骼 = 骨头树，蒙皮权重 = 皮粘在哪根骨头上，动画 = 骨头随时间的摆法。
- 只存骨头：数据小约 132 倍（68 骨 vs 3 万顶点），可复用、好混合、好在运行时修改。
- 姿势 = 一个时刻所有骨头的变换数组；关键帧之间按时间比例插值，所以 30 帧数据也能流畅播放。
- 一帧七步：问状态 → 选动画 → 取姿势 → 混合 → 修正 → 算矩阵 → 显卡蒙皮。

▶ **自测：同一个角色换了一件更宽松的外套（新的 SkeletalMesh），需要重新做动画吗？**

▶ **自测：为什么手指骨骼在远处可以不算？**

BASICS · B2

位置、旋转、缩放与坐标空间

“这根骨头在哪”这个问题，答案取决于“相对谁”。弄清变换和空间，FK、蒙皮矩阵、IK 都只是同一件事的不同用法。

本章目标

- 知道一个变换（Transform）由哪三部分组成，作用到点上的先后顺序
- 分清局部空间、组件空间、世界空间，会用“手在哪”的例子解释
- 手算一遍两节手臂的 FK，理解“父子级联”
- 知道矩阵是什么、乘法顺序为什么有两种约定，UE 里 FTransform 和 FQuat 的顺序为什么相反
- 用一个数值例子讲清“参考姿势的逆”在蒙皮里做了什么

B2.1 一个变换 = 缩放 + 旋转 + 平移

描述一根骨头“怎么摆”，需要回答三个问题：**多大**（缩放 Scale）、**朝哪**（旋转 Rotation）、**在哪**（平移 Translation）。合起来常简称 **TRS**。UE 的 `FTransform` 就是这三样：`FVector Scale3D`、`FQuat Rotation`、`FVector Translation`。

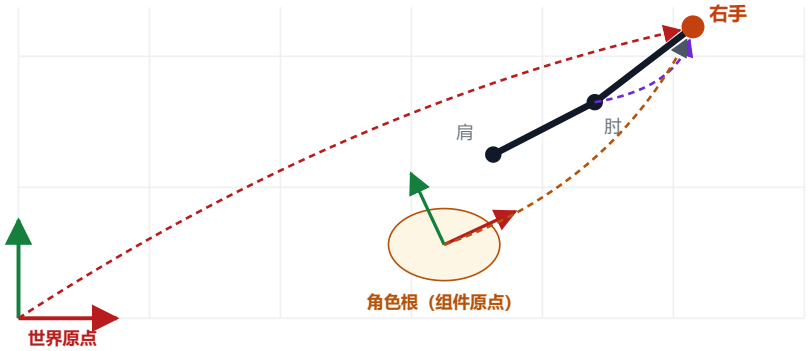
把变换作用到一个点上，顺序是**先缩放、再旋转、最后平移**（`FTransform::TransformPosition`）。顺序不能随便换：

点 (1, 0, 0)	第一步	第二步	结果
先绕 Z 转 90°，再平移 (10, 0, 0)	转完：(0, 1, 0)	平移：(10, 1, 0)	**(10, 1, 0)**
先平移 (10, 0, 0)，再绕 Z 转 90°	平移完：(11, 0, 0)	转完：(0, 11, 0)	**(0, 11, 0)**

直观理解：先平移再旋转，旋转是绕**原点**转的，点已经被推远了，于是转出一个大圈。骨骼的变换总是“先在自己的位置上转好，再放到该放的地方”，所以是先旋转后平移。

B2.2 坐标空间：同一只手，三种说法

问“你的右手在哪？”，可以有三种回答，都对，只是参照物不同：



三种说法，都在描述同一只手

局部空间（相对父骨骼）
“在前臂末端，沿前臂方向 28 cm”

组件空间（相对角色根）
“在角色前方 40、右侧 25、离地 120 cm”

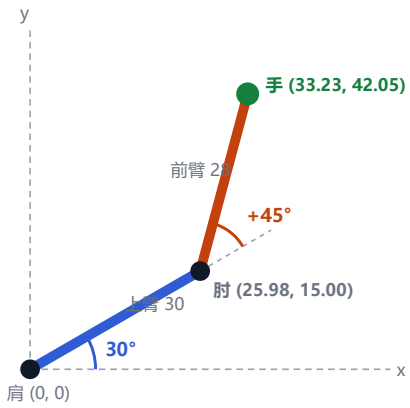
世界空间（相对地图原点）
“在地图坐标 (12040, −335, 220)”

图 B2-1 三种坐标空间（俯视示意，数字为举例）。动画数据存的是局部空间；角色整体移动或转身时，局部空间和组件空间的数都不变，只有世界空间的数在变

空间	参照物	为什么需要它
局部空间（Local / Bone Space）	父骨骼	动画数据存这个：同一条“挥手”动画，角色站哪都能用；混合大多在这里做
组件空间（Component / Mesh Space）	角色自己的原点（SkeletalMeshComponent）	“手离地多高”“头朝哪”这类问题要在这里回答：IK、瞄准、蒙皮矩阵
世界空间（World Space）	整个关卡的原点	和别的物体打交道：武器挂点、射线检测、脚下的地面高度

B2.3 父子级联：FK 手算一遍

两节手臂：肩膀转 30°，上臂长 30；肘部在此基础上再转 45°，前臂长 28。手在哪？（二维、单位 cm，数字来自实测 E11(g)）



一层一层往下算（先父后子）

① 肩：位置 (0, 0)，朝向 30°

② 肘 = 肩的位置 + 沿“肩的朝向”走 30

= (30·cos30°, 30·sin30°) = (25.98, 15.00)

肘的朝向 = 肩的朝向 + 自己的 45° = 75°

③ 手 = 肘的位置 + 沿“肘的朝向”走 28

= (25.98 + 28·cos75°, 15 + 28·sin75°)

= (33.23, 42.05)

角度一路累加、位置一路累加 —— 这就是 comp[i] = local[i] × comp[parent]
只改肩膀的 30°，肘和手会自动跟着动，但它们自己的局部数据不变

图 B2-2 FK（正向运动学）的手算：每根骨头的局部数据只说“相对父亲转了多少、离父亲多远”，组件空间的结果要从根往下一层层累积出来

三维里道理一样，只是“角度相加”变成了“旋转相乘”（三维旋转不能简单地把角度加起来，B3 会讲为什么），“沿朝向走 L”变成了“用父亲的旋转转一下自己的偏移，再加上父亲的位置”。这正是第 1 章那一行代码：

FK 的一行核心（UE 的 FTransform 乘法：A * B = 先 A 后 B）

```
for (int32 i = 1; i < NumBones; ++i)
    ComponentSpace[i] = LocalSpace[i] * ComponentSpace[Parent[i]];
// 展开看：位置 = 父旋转(父缩放 × 自己的局部位置) + 父位置；旋转 = 父旋转 · 自己的局部旋转
```

B2.4 矩阵是什么，乘法顺序为什么有两种写法

矩阵就是一张数字表。一个 4×4 矩阵能把“缩放 + 旋转 + 平移”打包在一起，拿一个点去乘它，三件事一次做完；两个矩阵相乘，就是把两次变换**合成一次**。GPU 喜欢矩阵，因为“乘一个矩阵”是它最擅长的运算。

麻烦在于**写法约定**：同样是“先做 A 再做 B”，

约定	“先 A 后 B”写作	谁在用
行向量（点写在左边：v · M）	`A * B`	UE 的 `FMatrix`、`FTransform`，DirectX 传统写法
列向量（点写在右边：M · v）	`B · A`	大多数数学课本、OpenGL / GLM、本书 demo 里的 3×3 矩阵
UE 的 `FQuat`	`B * A`	四元数按数学惯例，**和 FTransform 正好相反**（B3.7）

面试注意：手写变换公式前，先说约定

“蒙皮矩阵是 InvBind × Current 还是 Current × InvBind？”两种写法在各自的约定下都对。面试时先说一句“按 UE 的 FTransform 约定，A*B 表示先 A 后 B”，再写公式，就不会被追问绕晕。最常见的翻车是在 UE 里混用 `FTransform` 和 `FQuat` 的乘法：前者先左后右，后者先右后左。

B2.5 逆变换，以及“参考姿势的逆”在做什么

逆变换就是“撤销”：一个变换把点从 A 处搬到 B 处，它的逆就把点从 B 搬回 A。最常见的两个用法：

- **组件空间 → 局部空间**：IK 改完了手在组件空间的位置，要转回“相对前臂”的局部数据，就乘一次父骨骼组件变换的逆。
- **蒙皮里的“参考姿势的逆”**（也叫逆绑定矩阵）：见下图。



图 B2-3 蒙皮的两步（示意，偏移量画得比坐标轴比例大）：先用“参考姿势的逆”算出顶点相对骨头在哪（②），再按骨头当前的摆法放回去（③）。骨头不动时两步互相抵消，模型保持绑定时的样子

把②③合成一个矩阵，就是第 1 章的**蒙皮矩阵**：`SkinMatrix = InvRefPose × CurrentComponent`（UE 约定，先逆绑定再当前）。CPU 每帧为每根骨头算一个，上传给显卡；显卡对每个顶点按权重把几根骨头的结果加起来（B1-1 里肘部 0.5/0.5 的顶点就是两个结果各取一半）。

本章速记

- 变换 = 缩放 + 旋转 + 平移；作用到点上：**先缩放、再旋转、后平移**，顺序换了结果就变。
- 局部 = 相对父骨骼（动画存这个）；组件 = 相对角色根（IK、蒙皮）；世界 = 相对地图（挂件、射线）。
- FK：从根往下逐层累积；手算例子：肩 30°、肘 +45° → 手在 (33.23, 42.05)。
- 矩阵 = 打包的变换，相乘 = 合成；**UE：FTransform 的 A*B 先 A 后 B，FQuat 的 A*B 先 B 后 A。**
- 逆 = 撤销；参考姿势的逆把顶点变成“相对骨头的位置”，再乘当前变换放回去 = 蒙皮矩阵。

▶ **自测：角色整体向前跑了 10 米，局部空间、组件空间、世界空间里手的坐标分别变不变？**

▶ **自测：为什么不直接用顶点的绝对坐标乘以骨头的当前变换，还要先乘参考姿势的逆？**

BASICS · B3

什么是四元数：从“转多少、绕哪根轴”讲起

四元数就是把“绕哪根轴转、转多少度”编码成 4 个数的方法。它难懂的地方只有三处：为什么存半角、为什么 q 和 −q 是一回事、为什么乘法顺序不能换。本章把这三处讲透。

本章目标

- 知道表示三维旋转的四种方式，以及欧拉角具体出了什么问题（万向节锁、多解、跳变）
- 从二维复数旋转出发，理解四元数是它在三维的推广
- 会用 $q = (\cos \theta/2, n \cdot \sin \theta/2)$ 手写常见旋转的四元数，并能反推轴和角
- 理解 $q \cdot v \cdot q^{-1}$、半角、q 与 −q、乘法不交换、共轭即逆、归一化
- 知道什么时候用四元数、什么时候用矩阵，以及 UE 里 FQuat / FRotator 的常用接口和坑

30 秒版：什么是四元数?

四元数是 4 个数 (w, x, y, z)，单位长度的四元数表示一个三维旋转：**绕单位轴 n 转 θ，就是 $q = (\cos \theta/2, n \cdot \sin \theta/2)$**。w 管“转多少”，(x, y, z) 管“绕哪根轴”。旋转向量用 $q \cdot v \cdot q^{-1}$，两个旋转叠加就是两个四元数相乘（顺序不能换），逆就是共轭。

引擎用它而不用欧拉角，是因为它**没有万向节锁、插值平滑（slerp / nlerp）、组合旋转便宜、只占 4 个 float、漂移了归一化一下就好**。代价是不直观，所以编辑器面板上仍显示欧拉角（UE 的 FRotator），底层存储和计算用四元数（FQuat）。注意 q 和 −q 是同一个旋转，插值前要检查点积符号。

B3.1 为什么旋转这么麻烦

位置很简单：三个数 (x, y, z)，两个位置相加、取平均，逐个分量算就行。旋转也只有 3 个自由度，但它的三个“分量”会**互相影响**：先抬头再转身，和先转身再抬头，结果不一样。所以旋转没法像位置那样逐分量加减，于是出现了好几种表示方法：

表示	用几个数	直观吗	主要问题	UE 里谁在用
欧拉角（yaw / pitch / roll）	3	**最直观**	万向节锁；一个朝向有多种写法；插值会绕远、会跳变	`FRotator`：编辑器面板、玩法逻辑（“转身 90°”）
旋转矩阵	9（3×3）	**不直观**	占内存；累乘会“走样”变形；不好插值	`FMatrix`：渲染、GPU 蒙皮
轴角（轴 n + 角度 θ）	4	**比较直观**	两个轴角不好直接组合、插值	构造四元数时常用（`FQuat(Axis, Angle)`）
四元数	4	**不直观**	需要理解 q 与 −q、乘法顺序	`FQuat`：动画数据、姿势混合、FTransform 的旋转

B3.2 欧拉角的三个问题，用数字看清

欧拉角把一个旋转拆成三次“绕坐标轴转”：本书 demo 的约定是先绕 X 转 roll（翻滚），再绕 Y 转 pitch（俯仰），最后绕 Z 转 yaw（偏航）。UE 的 FRotator 也是 Pitch / Yaw / Roll 三个角，只是轴的正方向约定略有不同，下面的现象完全一样。

问题一：万向节锁——pitch 到 90° 时少了一个方向

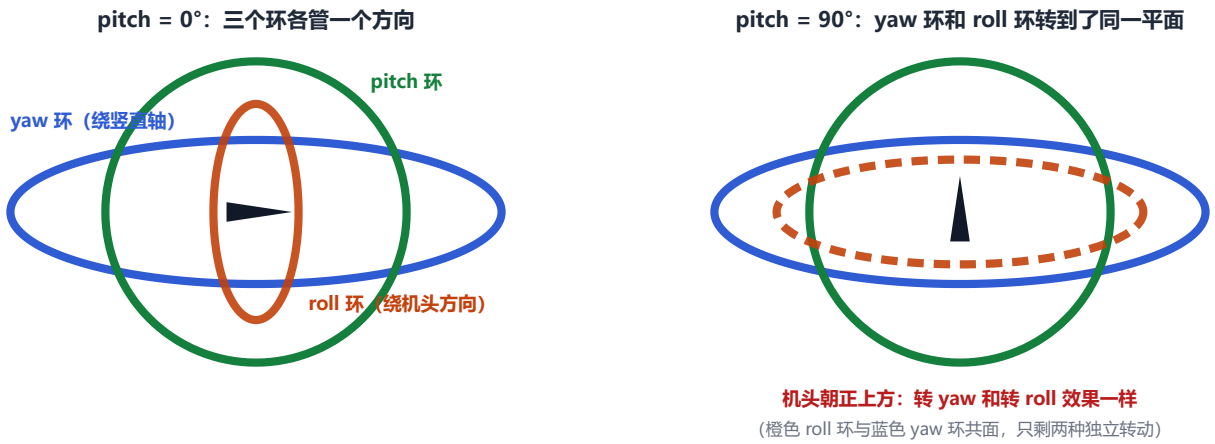


图 B3-1　万向节锁示意。三个套在一起的环依次决定 yaw、pitch、roll；中间的 pitch 环转到 90° 后，最外的 yaw 环和最里的 roll 环共面，绕的是同一根轴

实测 E12(a)：比较几组欧拉角对应的朝向差多少

pitch	(30, p, 0) vs (0, p, 30)	(30, p, 0) vs (0, p, −30)	(30, p, 0) vs (60, p, 30)
0°	42.18°	42.18°	42.18°
45°	55.34°	22.74°	22.74°
90°	60.00°	**0.00°**	**0.00°**

pitch = 90° 时，(30, 90, 0)、(0, 90, −30)、(60, 90, 30) 是**同一个朝向**：只有 yaw − roll 起作用，yaw 和 roll 各自的值失去了意义。demo 还算出了原因：此时 roll 绕的局部 X 轴在世界里指向 (0, 0, −1)，和 yaw 绕的世界 Z 轴重合。

白话：物体本身仍然可以朝任何方向转，坏掉的是**“用三个角来描述它”的这个方法**——在那个位置附近，有一个方向的转动没法用“改一点点角度”来表达，只能让 yaw 和 roll 同时大幅变化。

问题二：同一个朝向，很多种写法

实测 E12(b)：(0, 0, 0) 和 (180, 180, 180) 是同一个朝向（相差 0.0000°）；(90, 0, 0)、(450, 0, 0)、(−270, 0, 0) 也都一样。所以“两个 FRotator 的数字不同”不代表“朝向不同”，比较朝向要转成四元数或看夹角。

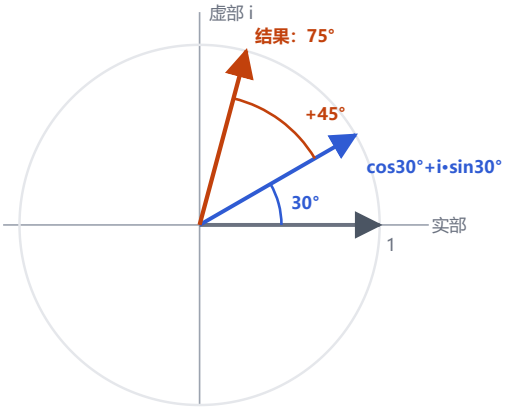
问题三：翻过头顶时数值跳变

实测 E12(c)：朝向绕 pitch 轴从 80° 匀速转到 100°，每步 0.02°，每步都换算成欧拉角

越过 90° 的那一步，**朝向只变了 0.020°，欧拉角却从 (yaw 5°, roll 0°) 跳到 (yaw −175°, roll 180°)。**如果在这两帧之间对欧拉角做插值（比如网络同步时插值 FRotator），中点的朝向离两端都有 **180°**——角色会在一帧里转一整个半圈。

再加上第 3 章的例子（Yaw 170° 和 −170° 各占一半，欧拉插值得到 0°），这就是动画系统**存储和混合一律用四元数**的原因。

B3.3 先看二维：复数相乘就是旋转



二维旋转 = 乘一个长度为 1 的复数

平面上的点 (x, y) 写成复数 x + y·i（$i^2 = -1$）。

乘以 (cos θ + i·sin θ)，等于把它绕原点转 θ：

```
(cos θ + i sin θ)(x + y i)
= (x cos θ − y sin θ) + (x sin θ + y cos θ) i
```

右边正好是二维旋转公式。两个旋转连着做 = 两个复数相乘：

```
(cos30°+i sin30°)(cos45°+i sin45°) = 0.2588 + 0.9659 i
= cos75° + i·sin75°，长度 1.0000（实测 E11(a)）
```

二维只需要“转多少”一个数；三维还要说“绕哪根轴”，
于是需要三个虚数单位 i、j、k —— 这就是四元数。

图 B3-2 单位复数表示二维旋转，相乘就是角度相加。四元数是同一个思路在三维的推广：单位四元数表示三维旋转，相乘就是旋转叠加

B3.4 四元数长什么样

四元数写成 `q = w + x·i + y·j + z·k`，其中 i、j、k 是三个“虚数单位”，满足 `i² = j² = k² = ijk = −1`（哈密顿 1843 年想出这条规则，当场刻在了都柏林的一座桥上）。平时直接写成 4 个数 **(w, x, y, z)**，或者 **(w, v)**，v = (x, y, z) 叫“虚部 / 向量部分”。

做游戏只需要记住一条：**绕单位轴 n 转 θ 角的旋转，对应的四元数是**

轴角 → 四元数（必背）

```
q = ( cos(θ/2),  n · sin(θ/2) )
  = ( w = cos(θ/2),  x = nx·sin(θ/2),  y = ny·sin(θ/2),  z = nz·sin(θ/2) )

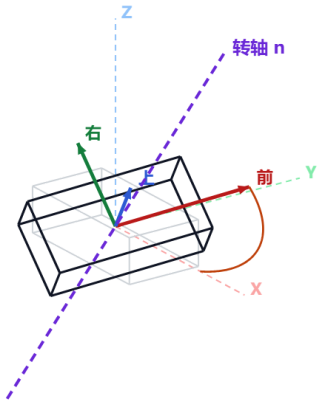
反推：θ = 2 · acos(w)          n = (x, y, z) / sin(θ/2)
条件：w² + x² + y² + z² = 1     （单位四元数才表示纯旋转）
```

白话：**w 管“转了多少”**（w = 1 是没转，w = 0 是转了 180°），**(x, y, z) 管“绕哪根轴”**（方向就是转轴，长度是 sin(θ/2)）。

实测 E11(b)：常见旋转的四元数（demo 打印）

旋转	w	x	y	z	怎么看出来
不转	1	0	0	0	单位四元数，FQuat::Identity
绕 Z 轴 90°	0.7071	0	0	0.7071	cos45° 和 sin45°；只有 z 非零 → 轴是 Z
绕 Z 轴 180°	0	0	0	1	w = 0 → 转了 180°
绕 X 轴 90°	0.7071	0.7071	0	0	只有 x 非零 → 轴是 X
绕 (1,1,1) 轴 120°	0.5	0.5	0.5	0.5	反推：θ = 2·acos(0.5) = 120°，n = (0.5774, 0.5774, 0.5774)
绕 Z 轴 360°	−1	0	0	0	转一整圈，朝向和不转一样，但四元数是 −1（见 B3.6）

交互实验 • 轴角 → 四元数




```
轴 n = (0.433, 0.250, 0.866)
θ = 90°   θ/2 = 45.0°

q = (w, x, y, z)
w = cos(θ/2) =  0.7071
x = nx·sin   =  0.3062
y = ny·sin   =  0.1768
z = nz·sin   =  0.6124
|q| = 1.0000

-q = (-0.707, -0.306, -0.177, -0.612)
   表示同一个旋转
反推 2·acos(w) = 90.0°
```

调轴的方向和转角：紫色虚线是转轴，灰框是没转之前的样子，橙色细线是“前”方向的尖端在转动过程中走过的圆弧。把角度拉到 360° 以上，看四元数怎么变号。

B3.5 用四元数转一个向量：$q \cdot v \cdot q^{-1}$

1. 把要转的向量 v 写成 w = 0 的四元数：(0, v)。
2. 左边乘 q，右边乘 q 的逆 q^{-1}（单位四元数的逆就是共轭，见 B3.7）。
3. 结果的 w 仍是 0，(x, y, z) 就是转好的向量。

为什么四元数里存的是半角 θ/2？ 因为 q 在左右各乘了一次，每一次“贡献”半个角度，合起来正好转 θ。如果存的是整角，转出来就是 2θ。

实际代码不会真的做两次四元数乘法，而是展开成几次叉积。UE 的 `FQuat::RotateVector` 就是这样写的：

FQuat::RotateVector 的思路（u = (X, Y, Z)，W 为实部）

```
// v' = q·v·q⁻¹ 展开后：v' = v + 2w(u × v) + 2u × (u × v)
// 整理成更省的形式：
const FVector T      = 2.f * FVector::CrossProduct(u, V);
const FVector Result = V + (W * T) + FVector::CrossProduct(u, T);
```

实测 E11(c)：三种算法对得上

10 万个随机向量（长度 1~100）、随机轴和角度：$q \cdot v \cdot q^{-1}$ 与罗德里格斯旋转公式最大相差 6.80e-05，展开公式与罗德里格斯最大相差 7.38e-05，都是 float 舍入误差（相对误差约 1e-6）。例：绕 Z 90° 的 q 作用于 (1, 0, 0) 得到 (0, 1, 0)——X 轴被转到了 Y 轴。

B3.6 q 和 −q：同一个旋转的两个名字

把 (w, x, y, z) 四个数全部取反，夹心公式 $q \cdot v \cdot q^{-1}$ 里两个负号抵消，转出来的向量完全一样。所以**每个旋转都对应两个四元数**：q 和 −q（数学上叫“双重覆盖”）。从公式看也很自然：转 θ 和转 θ + 360°（多转一圈）朝向相同，但 cos(θ/2) 和 sin(θ/2) 都差了一个负号。

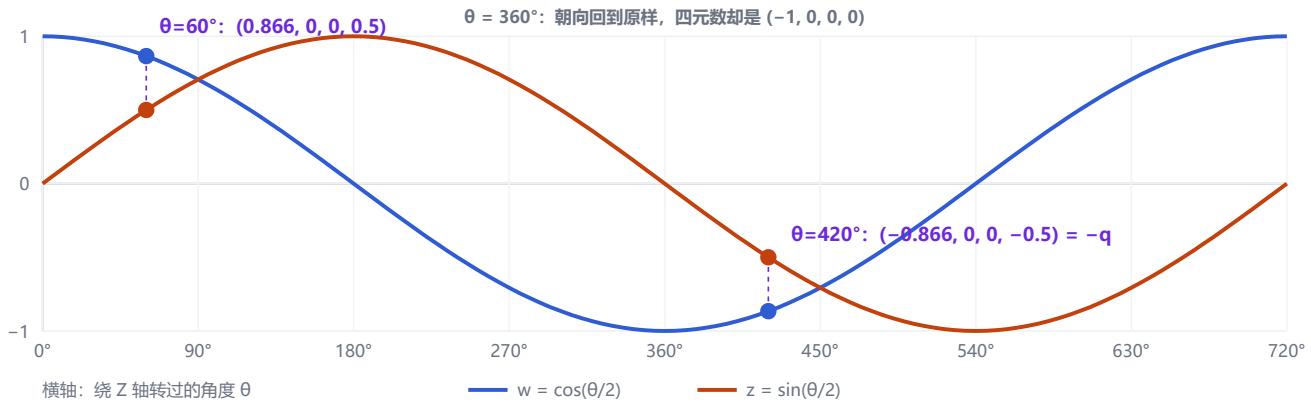


图 B3-3　绕 Z 轴转 θ 时四元数的 w = cos(θ/2) 和 z = sin(θ/2)。θ = 60° 与 θ = 420° 是同一个朝向，四元数却互为相反数；θ 要转满 720° 四元数才回到原值

实测 E11(d)

绕 Z 60°：q = (0.8660, 0, 0, 0.5000)；绕 Z 420°：q = (−0.8660, 0, 0, −0.5000) = −q。两者以及 −q 作用于 (3, 4, 5) 都得到 (−1.9641, 4.5981, 5.0000)。

面试注意：q 与 −q 的三个后果

- **插值前要检查点积**：dot(q_1, q_2) < 0 时，先把其中一个取反再插值，否则会走“远的那条路”（第 3 章 E2：170° 与 −170° 混合，不检查得到 0°，检查后得到 180°）。UE 的 `FQuat::Slerp`、`FQuat::FastLerp`、姿势混合用的 `AccumulateWithShortestRotation` 内部都做了这件事；自己手写混合时最容易漏。
- **判断“两个朝向是否相同”看 |dot| 是否接近 1**，或者直接算夹角（`AngularDistance`），不要逐分量比较。
- **压缩时可以丢掉 w**：先把 w < 0 的四元数整体取反，保证 w ≥ 0，就能只存 x、y、z，解压时 $w = \sqrt{1 - x^2 - y^2 - z^2}$（第 2 章）。

B3.7 乘法 = 旋转叠加；顺序不能换；逆 = 共轭

两个四元数相乘，得到“先做一个旋转、再做另一个旋转”的合成旋转。但**旋转本身就不满足交换律**，拿手机试试：先向前翻 90° 再向左转 90°，和先左转再前翻，手机最后的朝向不一样。

实测 E11(e)：两个 90° 旋转，换个顺序

顺序	前方 (1,0,0) 转到	上方 (0,0,1) 转到
先绕 X 90°，再绕 Y 90°	(0, 0, −1)	(0, −1, 0)
先绕 Y 90°，再绕 X 90°	(0, 1, 0)	(1, 0, 0)

两种顺序得到的朝向相差 **120.0°**。

UE 类型	`C = A * B` 的含义	记法
`FQuat`	**先 B 后 A**（右边先作用）	和数学书一样：$q \cdot v \cdot q^{-1}$ 里 q 在左边
`FTransform`	**先 A 后 B**（左边先作用）	FK：`Local * ParentComponent`

- **逆 = 共轭**：单位四元数的逆就是把虚部取反 (w, −x, −y, −z)，表示“反着转回去”。实测 E11(f)：$q \cdot q^*$ 与单位四元数最大相差 3.58e-07。
- **两个旋转的“差”** = $B \cdot A^{-1}$（UE：`B * A.Inverse()`），表示“从 A 再转多少能到 B”。实测：Yaw 75° · (Yaw 30°)$^{-1}$ = 绕 Z 转 45.0°。加性动画的旋转部分就是这么算的（第 3 章）。

B3.8 插值：从 A 转到 B 的中间

把所有单位四元数想象成一个球面（实际是四维空间里的球面），球面上每个点是一个朝向。从 A 转到 B：

- **lerp**：直接按比例混合 4 个分量，相当于在球**内部**走直线，中间点长度小于 1，不再是合法旋转。
- **nlerp**：lerp 之后再归一化，把点“推回”球面。路径和 slerp 一样，但速度不均匀（中间快、两头慢）。
- **slerp**：沿球面的大圆弧匀速走，角度随 t 严格线性变化，但要算 acos 和 sin，更贵。

第 3 章的实测：两姿势夹角 60° 时 nlerp 与 slerp 最大只差 0.27°，而 slerp 慢 4.86 倍，所以姿势混合用 nlerp。

B3.9 归一化与“漂移”：四元数也会走样，只是好修

实测 E12(d)：每次绕同一轴转 0.37°，连乘 100 万次（float，中途不修正）

表示	误差	单位向量变换后的长度
3×3 旋转矩阵	$\lvert M^{T}M - I\rvert$ 最大 2.34e-02（不再正交）	0.988216（压扁 1.18%）
四元数（不归一化）	$\lvert q\rvert = 1.025983$	1.052641（放大 5.3%，因为非单位四元数会把向量缩放 $\lvert q\rvert^2$ 倍）
四元数（每步归一化）	$\lvert q\rvert = 1$	**1.000000**

结论要说准确：**两者都会因为浮点误差慢慢走样。**区别在修复成本：四元数除以自己的长度就重新变成合法旋转（4 个数一次开方）；矩阵要重新正交化（例如 Gram-Schmidt），更贵，还得决定“以哪根轴为准”。所以常听到的“四元数不会形变”是不严谨的，正确说法是“四元数归一化便宜，所以容易保持正确”。

B3.10 什么时候用四元数，什么时候用矩阵

实测 E11(h)：各 1000 万次，ns/次（本机 float 标量实现，未手工 SIMD）

操作	四元数	3×3 矩阵	谁快
组合两个旋转	**0.78**	7.64	四元数（16 次乘法 vs 27 次）
旋转一个向量	1.64	**0.50**	矩阵（9 次乘法，最适合批量）
四元数 → 矩阵	6.97（每根骨头每帧只需一次）		—
存储	16 B	36 B	四元数

这正是 UE 的分工：**存储、混合、FK 组合**旋转时用四元数（动画数据、FTransform）；最后要把成千上万个顶点转一下时，先把每根骨头转成**矩阵**，交给 GPU 蒙皮。

B3.11 在 UE 里怎么用

FRotator / FQuat 常用写法（UE5，FVector 为双精度）

```
// 1) 编辑器和玩法逻辑里用 FRotator：单位是“度”，构造参数顺序是 (Pitch, Yaw, Roll)
FRotator Rot(0.f, 90.f, 0.f);                          // 不是 (Roll, Pitch, Yaw)！
FQuat    Q    = Rot.Quaternion();                       // 转成四元数
FRotator Back = Q.Rotator();                           // 转回来：Pitch 过 ±90° 时可能得到另一组等

// 2) 轴角构造：角度用“弧度”
FQuat Q2(FVector::UpVector, FMath::DegreesToRadians(45.f));
FVector Axis; FVector::FReal Angle;
Q2.ToAxisAndAngle(Axis, Angle);                        // 反推轴和角

// 3) 旋转 / 反向旋转向量
FVector Fwd   = Q.RotateVector(FVector::ForwardVector);
FVector Local = Q.UnrotateVector(WorldDir);            // 等价于用 Q.Inverse() 去转

// 4) 组合：FQuat 的 A * B 表示“先 B 后 A”
FQuat Combined = Q2 * Q;                               // 先 Q，再 Q2

// 5) 差值：从 Current 转到 Target 还需要的旋转
FQuat Delta = Target * Current.Inverse();

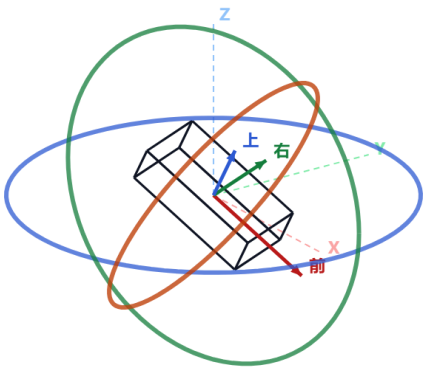
// 6) 插值：都已经处理了最短路径
FQuat S = FQuat::Slerp(A, B, Alpha);                   // 匀角速度，结果已归一化
FQuat L = FQuat::FastLerp(A, B, Alpha).GetNormalized();   // nlerp：FastLerp 本身不归一化

// 7) 比较朝向：看夹角，不要逐分量比较（q 与 -q 是同一朝向）
const double Deg = FMath::RadiansToDegrees(A.AngularDistance(B));
```

面试注意：UE 里四元数相关的坑

- **存储顺序**：`FQuat` 的成员和构造函数顺序是 `(X, Y, Z, W)`，而数学书和本书常写 (w, x, y, z)。手填数字时最容易填错。
- **FRotator 的参数顺序**是 (Pitch, Yaw, Roll)，单位是度；`FQuat(Axis, Angle)` 的角度是弧度。
- **不要对 FRotator 直接线性插值**：会出现 B3.2 的绕远和跳变。用 `FMath::RInterpTo`（内部走最短角度差）或转成 FQuat 再 Slerp。
- **累乘之后要 Normalize**：每帧 `Q = Delta * Q` 累积旋转（比如摄像机），隔一段时间要归一化一次（E12(d)）。
- **FQuat 与 FTransform 的乘法顺序相反**（B3.7），在同一段代码里混用时最容易写反。

交互实验 • 欧拉角与万向节锁




```
(yaw, pitch, roll) = (20, 30, 10)

四元数 q = qz·qy·qx
w 0.9515   x 0.0381
y 0.2685   z 0.1449

roll 轴在世界中的方向:
(0.81, 0.30, -0.50)
yaw 轴: (0, 0, 1)

yaw − roll = 10°
yaw + roll = 30°
pitch≠±90°: 三个角各自独立
```

约定与 demo 相同：先 roll（绕 X）、再 pitch（绕 Y）、最后 yaw（绕 Z）。蓝 / 绿 / 橙三个环分别是 yaw / pitch / roll 的转动平面。把 pitch 拉到 90°，再分别拖 yaw 和 roll：盒子转的是同一个方向。

本章速记

- 欧拉角直观但有三个问题：**万向节锁**（pitch 90° 时只有 yaw − roll 起作用，E12）、**多解**、**跳变**（越过头顶时 0.02° 的变化让 yaw 跳 180°）。
- 单位复数 = 二维旋转；单位四元数 = 三维旋转：**q = (cos θ/2, n•sin θ/2)**，w 管多少、xyz 管轴。

- 转向量：**$q \cdot v \cdot q^{-1}$**（所以存半角）；**q 与 −q 同一旋转**，插值前检查点积；**乘法不交换**（两种顺序差 120°）；**逆 = 共轭**；差值 = $B \cdot A^{-1}$。
- 四元数也会漂移，但归一化便宜；组合旋转四元数快（0.78 vs 7.64 ns），转大量向量矩阵快（0.50 vs 1.64 ns）。
- UE：FRotator（度，Pitch/Yaw/Roll）给人看，FQuat（X,Y,Z,W）给机器算；FQuat 的 A*B 先 B 后 A，FTransform 先 A 后 B。

▶ **自测：一个四元数 w = 0，说明了什么?**

▶ **自测：为什么插值前要检查两个四元数的点积，而混合位置时不用?**

▶ **自测：既然四元数这么好，为什么 GPU 蒙皮还用矩阵?**

BASICS · B4

插值、权重与混合：动画系统最常做的运算

混合空间、状态机过渡、分层混合、加性叠加，说到底都是“按比例把几个姿势合成一个”。把 lerp、权重、重心坐标这几个词弄懂，第 3~5 章就是在讲它们的不同用法。

本章目标

- 会算 lerp，知道“权重”和“归一化”是什么意思
- 知道混合两个姿势时，每根骨头的位置、旋转、缩放各怎么混
- 用“调整图层”的类比理解加性动画
- 用“小三角形面积”的直观方法理解混合空间的重心坐标和三角剖分

B4.1 lerp：从 A 走到 B 的百分之几

lerp（线性插值）：从 A 出发，朝 B 走全程的 α（0~1）那么远。

lerp

```
lerp(A, B, α) = A + (B − A)·α  =  (1 − α)·A + α·B
例：A = 10，B = 50，α = 0.25  →  10 + 40 × 0.25 = 20
```

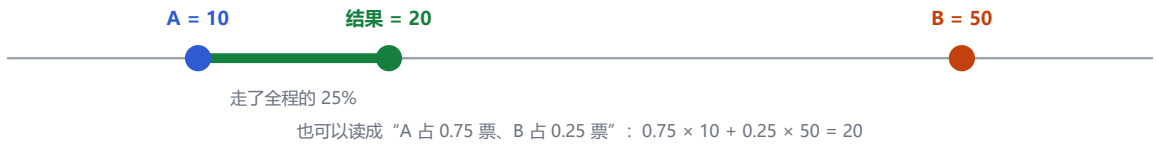


图 B4-1 lerp 的两种读法：“从 A 往 B 走 α”，或者“A 和 B 按权重 (1−α, α) 投票”。后一种读法推广到三个、四个参与者，就是多路混合

B4.2 权重与归一化：大家投票，总票数是 1

动画混合里的**权重**就是每个参与者的“票数”，规定加起来等于 1。如果某一步算出来不是 1（比如 Walk 0.6、Run 0.6，合计 1.2），就各自除以总和，变成 0.5、0.5，这叫**归一化**。

为什么一定要等于 1？举个反例：两条动画里同一根骨头的位置都是 (0, 0, 10)（就是骨长），按 0.6 + 0.6 加起来是 (0, 0, 12)，骨头凭空长了 20%。权重和为 1，才能保证“大家都一样时，结果也一样”。

B4.3 混合一个姿势 = 每根骨头分别插值

一个姿势是一组骨头的变换（B2）。混合两个姿势，就是对**每一根骨头**分别做：位置 lerp、缩放 lerp、旋转用四元数 nlerp（B3.8），旋转插值前检查点积符号。

某根骨头	位置	旋转（绕竖直轴）
Walk 姿势	(0, 0, 10)	20°
Run 姿势	(0, 0, 14)	60°
α = 0.25（Walk 0.75 / Run 0.25）	(0, 0, 11)	nlerp 约 29.9°（slerp 正好 30°，两者只差 0.08°）

交叉淡化（状态机过渡、Blend 节点）只是让 α 随时间从 0 走到 1；**混合空间**是根据速度、方向算出几个 α；**分层混合**是让不同骨头用不同的 α（上半身 1、下半身 0）。运算本身都一样。

B4.4 加性：只保存“差多少”

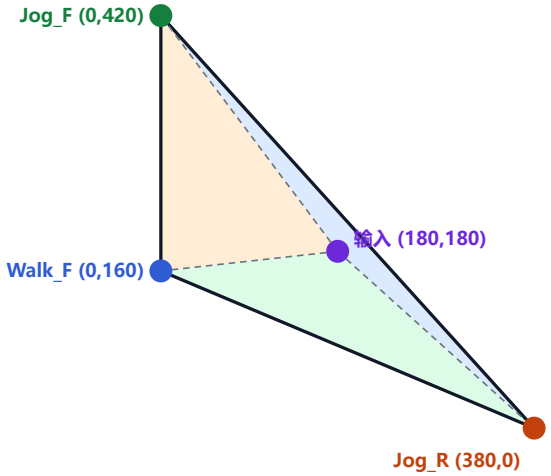
类比修图软件的**调整图层**：它不存一整张图，只存“亮度 +20”，叠到哪张图上都能用。加性动画也是这样：只存“相对某个基准姿势多弯了几度、多移了几厘米”，可以叠加到走、跑、蹲任何姿势上。

步骤	例子：呼吸动画叠到跑步上（胸椎前倾角）
制作时：加性 = 动画 − 基准	基准姿势胸椎 0°，呼吸最高点 4° → 差值 Δ = +4°
运行时：结果 = 目标 + α·Δ	跑步时胸椎前倾 15° → 叠上去 19°；α = 0.5 时 17°

旋转的“差”不能直接相减，要用四元数乘逆：$\Delta = R_{动画} \cdot R_{基准}^{-1}$，应用时 $R = \Delta^{\alpha} \cdot R_{目标}$（B3.7、第 3 章）。上表只是绕同一根轴的特例，所以看起来像加减法。

B4.5 重心坐标：三角形里的点怎么分票

二维混合空间（第 4 章）里，输入点落在某个三角形内，三个顶点各对应一条动画，要决定三条动画各占多少权重。答案是**重心坐标**，有一个很直观的算法：



规则：顶点的权重 = 它“对面”那块小三角形的面积占比

输入点把大三角形切成三块，每块正对一个顶点：

- 蓝块（对着 Walk_F）占 15.8% → Walk_F 权重 0.158
- 绿块（对着 Jog_F）占 36.8% → Jog_F 权重 0.368
- 橙块（对着 Jog_R）占 47.4% → Jog_R 权重 0.474

点越靠近某个顶点，它对面那块就越大，那条动画的票就越多；点正好在顶点上，那条动画的权重就是 1。

两个保证：

① 三个权重加起来正好是 1；

② 按权重平均三个顶点的坐标，正好回到输入点：

$$0.158\cdot(0,160) + 0.368\cdot(0,420) + 0.474\cdot(380,0) = (180, 180)$$

所以混合出的“速度”就是想要的速度——这是不滑步的前提（数字与实测 E5 一致）

图 B4-2　重心坐标的面积解释。三个颜色块分别正对三个顶点，面积占比就是该顶点（该动画）的权重

输入点落在**哪个**三角形里，取决于采样点事先被连成了什么样的三角网——这一步叫**三角剖分**。UE 用 **Delaunay 三角剖分**：尽量连成“胖”的三角形，规则是每个三角形的外接圆里不能有别的采样点。为什么要胖？细长的三角形里，输入只要移动一点点，权重就会剧烈变化，动画看起来会抖。

B4.6 同步：两个人一起走路，要踩同一个点

混合两条循环动画（走和跑）时，如果它们各播各的，一条在迈左脚，另一条在迈右脚，混出来就是两只脚都悬空。**同步**就是让它们按同样的节奏走：要么按“走到一个周期的百分之几”对齐（归一化时间），要么在动画里标出“左脚落地”“右脚落地”，按这些标记对齐（同步标记）。第 4 章的实测：不同步时 50.4% 的时间支撑脚不一致，按标记同步后为 0%。

本章速记

- lerp(A, B, α) = (1−α)A + αB；权重 = 票数，**总和必须为 1**，否则姿势会被放大缩小。
- 混合姿势 = 每根骨头：位置 / 缩放 lerp，旋转 nlerp（检查符号）。过渡、混合空间、分层混合只是 α 的来源不同。

- 加性 = 调整图层：只存“差多少”，可叠到任何姿势；旋转的差用乘逆。
- 重心坐标：**顶点权重 = 对面小三角形的面积占比**，和为 1，加权坐标 = 输入；Delaunay 让三角形尽量胖，权重变化更平稳。

▶ **自测：交叉淡化 0.2 秒，过渡的第 0.05 秒，源动画和目标动画各占多少?**

▶ **自测：混合空间的输入点落在三角网外面会怎样?**

BASICS · B5

名词白话词典：一句话 + 一个类比

正文里出现的术语，在这里各用一句大白话解释。面试时一个词卡住，整段回答就断了；先保证每个词都能用自己的话说出来。

B5.1 资产与数据

名词	一句话白话	类比 / 例子	详见
Skeleton 骨架	骨头的名字、父子关系和初始摆法	一份“人体骨骼名册”，所有衣服和动画都按它对号入座	第 1 章
SkeletalMesh 骨骼网格体	带蒙皮权重的模型	皮 + 皮粘在哪根骨头上的说明书	B1、第 1 章
蒙皮 / 蒙皮权重	每个顶点跟随哪几根骨头、各跟多少	肘部的皮一半跟上臂、一半跟前臂	B1、第 2 章
参考姿势 Reference Pose	美术绑定时骨头的摆法（T-Pose / A-Pose）	量体裁衣时站的那个姿势	B2、第 1 章
Socket 插槽	挂在某根骨头上的一个“挂钩点”	手上的挂钩，用来挂武器、特效	第 1 章
Virtual Bone 虚拟骨骼	在引擎里额外加的、不参与蒙皮的骨头：挂在源骨骼下，位置跟着目标骨骼	“左手相对右手在哪”的一根参考线，做 IK 目标	第 1 章、附录 A
重定向 Retargeting	把动画用到体型或骨架不同的角色上	同一套舞蹈，高个子和矮个子跳出来步幅不同	第 1 章
AnimSequence 动画序列	一段动画：每根骨头随时间的变换	一卷胶片，每一格记录骨头怎么摆	第 2 章
关键帧 / 采样 / 插值	按固定频率存的姿势；播放时在两帧之间按比例补出中间值	30 帧的数据，在 120 帧的游戏里也流畅	B1、B4
动画压缩	去掉不动的轨道、降低精度、删掉能插值出来的帧	把视频压成 mp4	第 2 章
曲线 Curve	动画里随时间变化的一个数字	“脸红程度”“脚离地多高”，用来驱动材质、表情、逻辑	第 2 章
Notify / Notify State	动画时间轴上的事件 / 一段时间区间	“第 0.3 秒播脚步声” / “0.2~0.5 秒开启伤害判定”	第 2、7 章

B5.2 数学与空间

名词	一句话白话	类比 / 例子	详见
变换 Transform（TRS）	多大、朝哪、在哪	FTransform = 缩放 + 四元数旋转 + 平移	B2
局部 / 组件 / 世界空间	相对父骨骼 / 相对角色根 / 相对地图原点	“前臂末端 28 cm” / “角色前方 40 cm” / “地图坐标”	B2
FK 正向运动学	给每节的角度，算末端在哪	肩转 30°、肘再转 45° → 手在 (33.23, 42.05)	B2、第 1 章
IK 逆向运动学	给末端目标，反算每节怎么转	“手要碰到门把手，肩和肘该转多少”	第 8 章
矩阵	把变换打包成一张表，乘一次就完成变换	GPU 蒙皮用 3×4 矩阵	B2
欧拉角 / FRotator	用三个角度描述朝向	Pitch 抬头低头、Yaw 左右转、Roll 侧翻	B3
万向节锁	pitch 到 ±90° 时，yaw 和 roll 变成绕同一根轴	三层陀螺仪的两个环转到了同一平面	B3
四元数 / FQuat	把“绕哪根轴、转多少”编码成 4 个数	q = (cos θ/2, n·sin θ/2)	B3
lerp / nlerp / slerp	直线插值 / 直线插值后拉回球面 / 沿球面匀速插值	nlerp 快，slerp 匀速	B3、第 3 章
归一化	把长度调回 1，或把权重调成和为 1	四元数每隔一段时间要归一化一次	B3、B4
蒙皮矩阵 / 逆绑定矩阵	“先变成相对骨头的位置，再按骨头当前摆法放回去”	顶点 55 → 相对手骨 +5 → 手骨移到 70、转 90° → (70, 25)	B2

B5.3 混合

名词	一句话白话	类比 / 例子	详见
Pose 姿势	某一时刻所有骨头的变换数组	一张定格照片	B1
混合 Blend / 权重	几个姿势按票数合成一个，票数和为 1	走 0.7 + 跑 0.3	B4、第 3 章
交叉淡化 Cross Fade	过渡期间权重从 0 慢慢变到 1	视频转场的淡入淡出	第 7 章
加性动画 Additive	只存“比基准姿势多了多少”，可以叠在任何姿势上	修图的调整图层	B4、第 3 章
Aim Offset 瞄准偏移	按瞄准角度挑选并叠加加性姿势	边跑边把枪口抬高	第 3 章
Blend Space 混合空间	按 1~2 个参数自动给几条动画分配权重	速度 290 → 走 0.5 + 慢跑 0.5	第 4 章
重心坐标 / 三角剖分	三角形里的点按“对面小三角形面积”给三个顶点分票 / 把采样点连成三角形	图 B4-2	B4、第 4 章
Sync Group 同步组 / Leader / Follower	让几条循环动画按同一个节奏走；权重最高的定节奏，其余跟着	齐步走，排头兵喊口令	第 4 章
Sync Marker 同步标记	在动画里标出左右脚落地，按标记对齐	“都在左脚落地那一刻对齐”	第 4 章
Layered Blend per Bone 分层骨骼混合	不同骨头用不同的混合权重	上半身换弹，下半身照跑	第 5 章
Blend Depth 混合深度	权重从过滤骨骼开始，花几层骨头从小渐变到 1；负数表示排除这个分支	腰部逐节过渡，不会“折断”	第 5 章
Mesh Space Rotation Blend	旋转按“相对角色”的朝向混合，而不是“相对父骨骼”	不管腿怎么扭，枪口一直对准前方	第 5 章

B5.4 动画蓝图与运行时

名词	一句话白话	类比 / 例子	详见
Animation Blueprint / AnimInstance	决定“这一帧放什么动画、怎么混”的程序	动画导演	第 6 章
EventGraph	收集角色状态、算出变量（游戏线程）	导演助理去问演员现在跑多快	第 6 章
AnimGraph	用变量驱动节点，算出最终姿势	导演按剧本把镜头剪出来	第 6 章
Update / Evaluate	先推进时间、算权重；再真正采样、混合出姿势	先排好“谁占多少戏份”，再拍	第 6 章
Proxy 代理	动画计算用的一份数据拷贝，让它能在别的线程算	把资料复印一份带走，不占用原件	第 6 章
Fast Path 快速路径	节点引脚直接读变量，不走蓝图虚拟机	直接抄答案，不现场演算	第 6 章、附录 A
Property Access / 线程安全函数	在工作线程里安全读取角色数据 / 可在工作线程运行的函数	UE5 推荐替代 EventGraph 的写法	第 6 章
Cached Pose 缓存姿势	把一段子图的结果存起来，多处使用只算一次	复印件	第 5 章
Compact Pose / RequiredBones	本帧真正需要计算的那些骨头	远处角色不算手指	第 1、9 章
State Machine 状态机	“站、走、跑、跳”这些状态，以及什么条件下切换	交通信号灯的切换规则	第 7 章
Conduit / State Alias	状态机里的“路口”/“一组状态的别名”	减少连线	第 7 章
Inertialization 惯性化	切换时只算新动画，把新旧之间的差距平滑衰减掉	急转弯时身体惯性地“甩”过去	第 7 章
Montage 蒙太奇 / Section / Slot	代码触发的一次性动画 / 它的分段 / 它在 AnimGraph 里插入的位置	连招：Combo1 → Combo2 → Combo3	第 7 章
Branching Point	蒙太奇里精确到帧、同步触发的事件	连招输入窗口	第 7 章

Root Motion 根运动	由动画里根骨骼的位移来移动角色	翻滚多远由动画说了算	第 7 章
In-Place 原地动画	动画原地踏步，移动由代码决定	跑步机上跑步	第 7 章
Pole / Joint Target	两骨骼 IK 里决定肘、膝朝哪边弯的参考点	“膝盖要朝前弯”	第 8 章
Control Rig / IK Rig	UE5 的节点化绑定与 IK 工具 / IK 目标与重定向设置	在引擎里做“绑定”	第 8 章
距离匹配 / 步幅调整 / 朝向调整	按剩余距离选播放位置 / 拉伸步幅贴合速度 / 扭转下半身贴合方向	减少滑步、减少动画数量	第 8 章
Motion Matching	每帧在动画库里搜“最像现在、最符合接下来要走的路”的那一帧	从海量动作捕捉里自动挑镜头	第 8 章
URO / Budget Allocator	远处角色降低更新频率 / 按总预算分配更新频率	远处的人少看几眼	第 9 章
LOD	远处用更简单的模型、更少的骨头	远景画得粗一点	第 9 章
Leader Pose（旧称 Master Pose）	部件直接复用主模型的骨骼结果，自己不算	衣服、头发跟着身体走	第 9 章

PART ONE

第一部分　骨骼与动画数据

动画系统的输入是什么：骨骼怎么存、姿势怎么表示、空间怎么换算，网格怎么跟着骨骼动，动画数据怎么压到几十 KB。

CHAPTER 01

骨架、网格与姿势：动画系统的数据模型

先把 Skeleton、SkeletalMesh、AnimSequence 的关系理清，再看一根骨骼的变换怎么从“相对父骨骼”变成“相对整个角色”。

本章目标

- 说清 UE 动画相关资产的引用关系，以及为什么动画绑定的是 Skeleton 而不是 Mesh
- 理解骨骼按拓扑序存放的意义，以及 Mesh / Skeleton / Compact Pose 三套骨骼下标
- 分清局部空间、组件空间、世界空间，能写出 FK 的一行核心代码
- 知道参考姿势、插槽、虚拟骨骼、重定向分别解决什么问题

1.1 资产关系：一切围绕 Skeleton

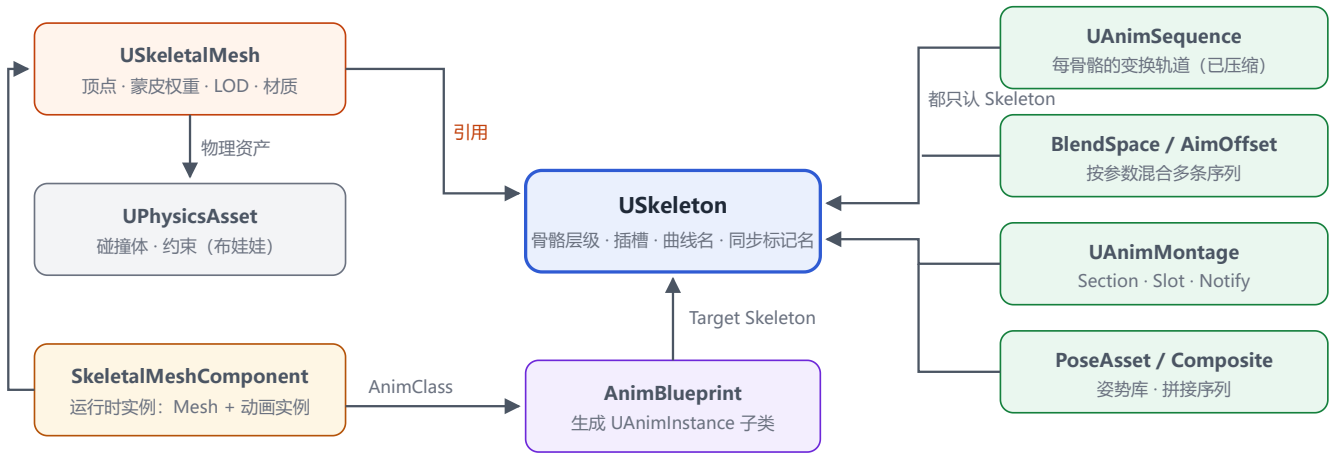


图 1-1 动画相关资产的引用关系。动画资产和动画蓝图都绑定在 Skeleton 上，所以“同一个 Skeleton 的不同 Mesh”可以共享全部动画

- **USkeleton**：骨骼名和父子关系（`FReferenceSkeleton`）、插槽（Socket）、虚拟骨骼、动画曲线名、Slot 组、同步标记名、重定向设置。它是一套“命名约定”，动画数据按骨骼名对上号。
- **USkeletalMesh**：可见的网格。每个顶点有若干骨骼影响和权重；自己也存一份参考骨骼（可以比 Skeleton 少骨骼，但必须是它的子集，层级一致）。
- **UAnimSequence**：一条动画。每根有动画的骨骼一条轨道（旋转 / 平移 / 缩放），另外还有曲线、Notify、同步标记、Root Motion 设置、加性设置、压缩设置。
- **SkeletalMeshComponent**：运行时把 Mesh 和动画实例（`AnimClass` 指定的动画蓝图）组合起来，每帧拿到姿势、算蒙皮矩阵、交给渲染。

> **游戏客户端实战：换装为什么能共享动画**
>
> 角色换装的常见做法是：身体、头、衣服是同一 Skeleton 的多个 SkeletalMesh。主 Mesh 跑动画蓝图，其余部件用 **Leader Pose Component**（UE 5.1 之前叫 Master Pose）直接复用主 Mesh 的骨骼变换，不再各自求值（第 9 章）。部件骨骼必须是主骨架的子集且名字一致，否则对不上。

1.2 骨骼怎么存：数组 + 父下标，按拓扑序

UE 不用指针树存骨骼，而是用一个数组：每根骨骼存名字和父骨骼下标（`FMeshBoneInfo{Name, ParentIndex}`），并且**保证父骨骼的下标一定小于子骨骼**（拓扑序）。参考姿势 `RefBonePose` 是同样长度的局部变换数组。



图 1-2　骨骼层级在 UE 里就是一个按拓扑序排好的数组。FK、蒙皮矩阵、LOD 减骨都依赖这个性质

拓扑序带来三个好处：

1. **FK 只需一个 for 循环**，不需要递归，也不需要访问标记；数据是连续数组，缓存友好。
2. **任意“前缀”或“保持顺序的子集”仍是合法骨架。**LOD 减骨、只算必需骨骼（Required Bones）时，把下标数组过滤一遍就行。
3. **父子关系判断很便宜**：沿 ParentIndex 往上走，下标单调递减，一定会终止。

三套骨骼下标：面试时很容易被问晕

下标	相对谁	用在哪
Skeleton Bone Index	USkeleton 的骨骼数组	动画序列的轨道按它（经映射表）对应骨骼
Mesh Bone Index	USkeletalMesh 自己的参考骨骼	蒙皮、Socket、物理资产；Mesh 可能比 Skeleton 少骨骼
Compact Pose Bone Index	本帧**实际需要计算**的骨骼（`FBoneContainer` 的 RequiredBones）	AnimGraph 求值时的 `FCompactPose`；随 LOD 变化

面试注意：写自定义动画节点时的经典 bug

在 `Evaluate_AnyThread` 里拿到的是 `FCompactPose`，下标是 `FCompactPoseBoneIndex`。如果直接用 Skeleton 或 Mesh 的骨骼下标去取，切 LOD 后骨骼变少，就会取错骨骼甚至越界。正确做法是在 `CacheBones_AnyThread` 里用 `FBoneReference::Initialize(RequiredBones)` 预先解析，求值时调用 `GetCompactPoseIndex`，并检查是否有效（这根骨骼在当前 LOD 可能被剔掉了）。

1.3 三种空间与 FK

空间	相对于	谁在用
局部空间（Local / Bone / Parent Space）	父骨骼	动画序列存的就是它；大部分混合在这里做
组件空间（Component / Mesh Space）	SkeletalMeshComponent 的原点	IK、Look At、Mesh Space 混合、蒙皮矩阵
世界空间（World Space）	世界原点	Socket 挂武器、射线检测、脚 IK 的地面高度

UE 的 `FTransform` 是“旋转（四元数）+ 平移 + 三维缩放”，乘法 `A * B` 的含义是**先应用 A，再应用 B**。于是组件空间变换就是：

局部空间 → 组件空间（FK 的全部核心）

```
// 与 FCSPose::CalculateComponentSpaceTransform 的语义一致
// 骨骼按拓扑序存放：处理 i 时，Parent[i] 一定已经算完
ComponentSpace[0] = LocalSpace[0];
for (int32 i = 1; i < NumBones; ++i)
{
    ComponentSpace[i] = LocalSpace[i] * ComponentSpace[Parent[i]];   // 先局部，再父骨骼
}
// 世界空间再乘一次组件到世界的变换
WorldSpace[i] = ComponentSpace[i] * ComponentToWorld;
```

反过来，组件空间转回局部空间：`Local[i] = Component[i] * Component[Parent].Inverse()`（也就是 `GetRelativeTransform`）。**IK 节点改的是组件空间，改完要转回局部空间**，这就是 AnimGraph 里“Local To Component / Component To Local”转换节点存在的原因。

实测 E1：68 根骨骼（仿 Mannequin，平均深度 6.8、最大 11），2000 个角色

写法	每角色耗时	相对	说明
A 线性数组：`comp[i]=local[i]*comp[par[i]]`	**1.12 us**	1.00x	67 次复合
B 指针树递归 DFS（节点散落在堆上）	3.95 us	3.54x	同样 67 次复合，多了指针追逐和递归
C 每根骨骼单独回溯到根	6.31 us	5.65x	459 次复合（Σ深度），结果一样但浪费

三种写法结果一致（最大位置差 4.2e-5，float 舍入）。指针树的计算量和线性数组完全一样，慢在**内存访问**：节点散落在堆上，每访问一个子节点都可能是一次缓存未命中（这个倍数对内存布局很敏感：本书两次完整运行分别测得 1.28x 和 3.54x，表中是最终版本的数字）。这是每个角色每帧都要做的事：拓扑序数组让它成为一个最简单的线性循环，也方便 SIMD 和多线程。

1.4 参考姿势与蒙皮矩阵

美术在 DCC 里把网格“绑”到骨骼上时，骨骼处在某个姿势（通常是 T-Pose 或 A-Pose），这就是**参考姿势 / 绑定姿势**。顶点坐标是在这个姿势下给出的。运行时骨骼动了，顶点要跟着动：

蒙皮矩阵

```
SkinMatrix[b]  = InvRefPoseComponent[b] * CurrentComponent[b]      // UE 的 A*B：先 A 后 B
顶点新位置     = Σ_k  weight_k * ( SkinMatrix[bone_k] 作用于 绑定位置 )
```

直观理解：先用 `InvRefPose` 把顶点从“绑定时的组件空间”搬到“这根骨骼的局部空间”，再用当前姿势搬回组件空间。`InvRefPose` 在加载时就算好缓存起来。**骨骼一动不动时，SkinMatrix 是单位矩阵**，网格保持绑定时的样子。

1.5 插槽、虚拟骨骼与重定向

Socket（插槽）

Virtual Bone（虚拟骨骼）

挂在某根骨骼上的一个相对变换，用来挂武器、特效、摄像机。可以加在 Skeleton 上（所有 Mesh 共享）或某个 Mesh 上（只对它生效）。运行时 `GetSocketTransform` = 相对变换 × 骨骼的组件 / 世界变换。

在 Skeleton 里加的、DCC 里不存在的骨骼（名字带 `VB` 前缀）：它是**源骨骼（父）的子骨骼**，但变换**跟随目标骨骼**，即“目标骨骼的位置，用源骨骼的空间来表达”。不参与蒙皮，常用作 IK 目标、瞄准参考点，不用回 DCC 改骨架、重导所有动画。

重定向（Retargeting）

同一 Skeleton、不同体型：设置每根骨骼的平移重定向方式（Animation / Skeleton / AnimationScaled 等），让平移按骨骼比例缩放。不同 Skeleton：UE5 用 **IK Rig + IK Retargeter** 做链到链的映射，可离线导出，也可运行时重定向。

深入：为什么 Translation Retargeting 通常只让 root / pelvis 用 Animation

动画里大部分骨骼的“平移”其实就是骨长。高个子角色播放矮个子的动画时，若所有骨骼都用动画里的平移（Animation 模式），骨长被改成矮个子的，网格就会被拉扯变形。所以常见设置是：**其余骨骼用 Skeleton 模式（保留自己的骨长，只用动画的旋转）**；pelvis 用 AnimationScaled（按身高比例缩放上下起伏）；root 用 Animation（保留位移）。

1.6 FTransform 的坑：非均匀缩放

FTransform 用“四元数 + 平移 + 缩放”而不是 4×4 矩阵，好处是插值简单、内存小（UE5 双精度版本更大，动画内部大多用 float 版本的数据）。代价是：**父骨骼有非均匀缩放、子骨骼有旋转时，真实结果包含剪切（shear），而 FTransform 表达不了**，复合后会有误差。所以绑定时尽量避免非均匀缩放的骨骼带有旋转的子骨骼；需要“压扁 / 拉伸”效果时，把缩放放在叶子骨骼或单独的辅助骨骼上。

本章速记

- 动画资产和动画蓝图都绑定 **Skeleton**；同 Skeleton 的多个 Mesh 共享动画，换装部件可用 Leader Pose 复用骨骼。

- 骨骼 = **数组 + 父下标，拓扑序**；FK 是一个循环：`comp[i] = local[i] * comp[parent]`（E1：比指针树快 3.54x，比逐根回溯快 5.6x）。
- 三套下标：Skeleton / Mesh / **Compact Pose**；自定义节点要在 CacheBones 里用 FBoneReference 解析。
- 蒙皮矩阵 = 参考姿势逆 × 当前组件空间变换；IK 在组件空间做，完成后转回局部空间。
- Socket 挂东西，Virtual Bone 做 IK 目标，Retarget 处理体型差异；FTransform 不支持剪切。

▶ **自测：一个武器 Mesh 想挂在右手上，并且随动画抖动，应该挂在骨骼上还是 Socket 上？两者的区别？**

▶ **自测：为什么 LOD 减骨不需要重新构建骨骼层级？**

CHAPTER 02

蒙皮、动画序列与压缩

骨骼动了，网格怎么跟着变形；一条动画在硬盘和内存里长什么样；为什么压缩误差要在组件空间衡量。

本章目标

- 写出线性混合蒙皮（LBS）的公式，知道 UE 在 GPU 上做蒙皮
- 理解 LBS 的“糖纸扭曲”和“关节塌陷”，以及 UE 用扭转骨和修正形变来解决
- 说清 AnimSequence 里有什么：轨道、曲线、Notify、同步标记、Root Motion
- 掌握动画压缩的三板斧，以及误差为什么会沿骨骼层级累积

2.1 线性混合蒙皮（LBS）

每个顶点记录最多 N 根影响骨骼及权重（权重和为 1）。变形后的位置是各骨骼蒙皮矩阵作用后的**加权平均**：

Linear Blend Skinning

```
v' = Σ_k  w_k · (SkinMatrix[b_k] · v)        // = (Σ_k w_k · SkinMatrix[b_k]) · v
     先把几个 3×4 矩阵按权重加起来，再乘一次顶点 —— GPU 上就是这么做的
```

- **在哪算**：UE 默认 GPU 蒙皮。CPU 每帧只上传骨骼矩阵（每根 3×4），顶点着色器里按顶点的骨骼下标和权重取矩阵、加权、变换。需要在 CPU 拿顶点位置（比如精确命中检测）时才用 CPU 蒙皮，开销大得多。
- **影响骨骼数**：每个顶点的最大影响骨骼数可以按 LOD 设置（常用 4，可以更多），越多越准也越贵。
- **Section 骨骼数**：一个渲染 Section 能用的骨骼矩阵数受常量缓冲大小限制（移动端的经典上限是 75），超出时导入会把 Section 拆开，多一次 Draw Call。

2.2 LBS 的两个经典问题

矩阵线性平均的结果不再是刚体变换。两根骨骼夹角越大，平均出来的矩阵“缩”得越厉害：

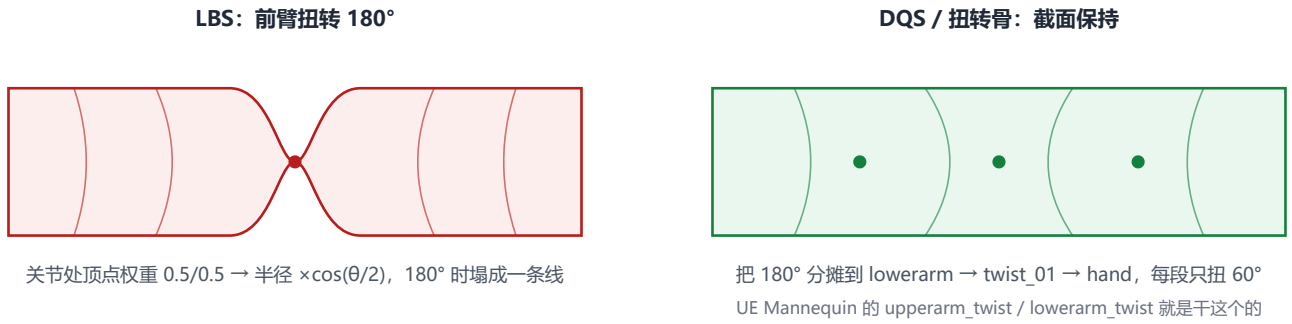


图 2-1 “糖纸扭曲”（candy-wrapper）：LBS 在扭转时体积丢失。UE 的标准做法是加扭转骨把大角度拆小，再配合修正形变

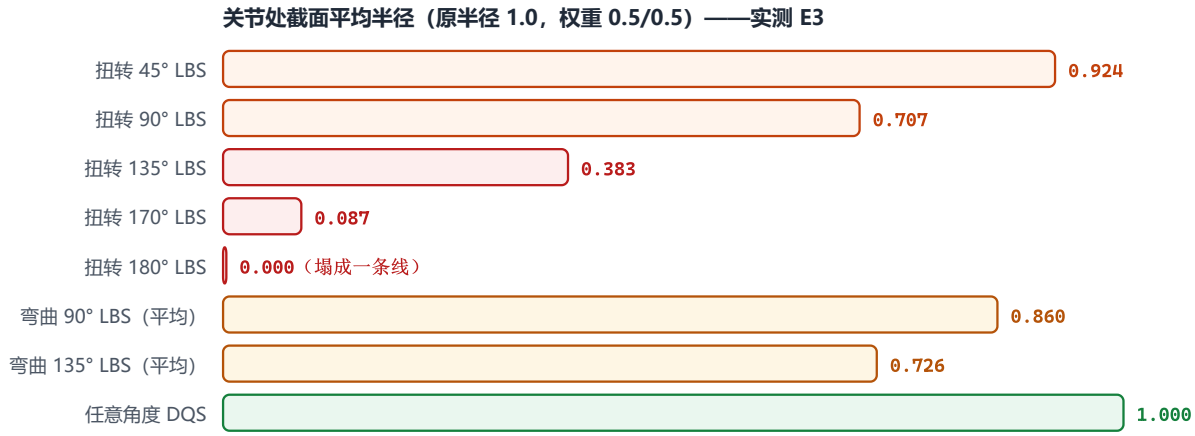


图 2-2 实测 E3：LBS 扭转时半径严格等于 cos(θ/2)；弯曲时关节内侧被“压扁”（弯曲 90° 最小半径 0.707）；双四元数蒙皮（DQS）始终保持 1.0

方案	原理	代价（E3：3 万顶点 × 4 影响，CPU）	UE 里的地位
LBS	矩阵加权平均	0.196 ms	默认，GPU 蒙皮
DQS	对偶四元数加权平均再归一化，结果仍是刚体变换	0.848 ms（4.33x）	引擎没有内置；有“鼓包”问题，不支持缩放
扭转骨	大扭转分摊到多根骨骼	多几根骨骼	Mannequin 标配，由动画或 Twist Corrective 节点驱动
修正形变	按关节角度驱动 Morph Target 或辅助骨骼	Morph 的 GPU 开销	Pose Driver（RBF）/ 曲线驱动 Morph

面试注意：别只说“DQS 解决了糖纸问题”

DQS 确实保体积，但它在弯曲时会让关节外侧**鼓起来**，也处理不了缩放，而且比 LBS 贵（E3：4.33x）。工业界主流仍是 LBS + 扭转骨 + 修正形变（Morph Target / 辅助骨骼）。能说出“为什么 Mannequin 有 twist 骨”，比背 DQS 公式更像做过项目。

2.3 动画序列里有什么

内容	说明
骨骼轨道	每根有动画的骨骼一条：旋转、平移、缩放各一组 key。没动的轨道在压缩时只留一个值或直接省略（与参考姿势相同）
曲线（Curves）	随时间变化的浮点值：驱动 Morph Target、材质参数，或给逻辑用（例如“脚离地距离”供距离匹配）。曲线名登记在 Skeleton 上
Notify / Notify State	时间轴上的事件：脚步声、特效、开关武器碰撞。State 有 Begin / Tick / End
同步标记（Sync Markers）	例如 L / R 表示左右脚落地，供同步组对齐（第 4 章）
Root Motion	是否把 root 骨骼的位移提取出来交给角色移动（第 7 章）
加性设置	Additive 类型（局部 / 网格空间）和基准姿势（第 3 章）
压缩设置	骨骼压缩编解码器和曲线压缩设置

运行时“采样一条序列”就是：根据当前时间找到前后两个 key，旋转 nlerp、平移 / 缩放 lerp；没有轨道的骨骼直接用参考姿势。`UAnimSequence::GetAnimationPose` 最终调用解压代码完成这件事。

> **Notify 是怎么触发的**
>
> 普通 Notify 是**排队触发**的：求值时只是把本帧扫过的 Notify 收集起来，等回到游戏线程（第 6 章的 Complete 阶段）再统一派发，所以可能比动画画面晚一帧左右。混合中的多条动画都会产生 Notify，可以设 **Trigger Weight Threshold** 过滤掉权重很低的，或在混合空间里设 Notify Trigger Mode 为“只触发最高权重的动画”。蒙太奇里的 Notify 可以设为 **Branching Point**，在推进时间时同步、精确地触发，适合连招衔接等对时机敏感的逻辑，但更贵。

2.4 动画压缩：为什么要压、怎么压

不压缩的话，一条 4 秒、30 fps、68 根骨骼的动画，每帧每骨存 16 B 旋转 + 12 B 平移 + 12 B 缩放，就是 329,120 B。一个角色几百条动画，几十个角色，内存很快就爆了。压缩的三板斧：

1. **去掉常量轨道**：大量骨骼的平移就是骨长，缩放全是 1，手指在跑步动画里也不动，这些只存一个值。

2. **量化**：单位四元数满足 $x^2+y^2+z^2+w^2=1$，保证 $w \geq 0$ 后可以只存 xyz，每个分量 16 位甚至更少；平移按每条轨道的取值范围归一化后量化。
3. **关键帧精简**：能被前后两个 key 插值出来、误差在阈值内的中间帧删掉，代价是要存“这个 key 是哪一帧”。ACL 的做法不同：保持均匀采样，按误差给每条轨道选择不同的位宽（可变位率）。



图 2-3 E9：68 骨骼 × 121 帧，48 根骨骼有旋转动画。阈值太小时几乎删不掉帧，多存的“帧号”反而让体积变大

误差会沿骨骼层级累积

压缩器如果只在**局部空间**控制误差（“每根骨骼旋转误差 ≤ 0.2°”），到了组件空间误差会叠加：手指的组件空间朝向 = 骨盆、三节脊柱、锁骨、上臂、前臂、手、三节手指的局部旋转一路乘下来，每一节的误差都会传下去。

精简阈值（局部）	局部旋转最大误差	组件空间旋转最大误差	组件空间位置最大误差
0.05°	0.079°	0.148°	0.032 cm
0.20°	0.202°	**0.444°（2.2 倍）**	0.264 cm
0.50°	0.499°	**1.638°（3.3 倍）**	0.699 cm

（0.05° 阈值时局部误差 0.079° 超过阈值，是因为精简在量化之后进行，两步的误差叠加了。）

深入：ACL 怎么衡量误差——虚拟顶点

ACL（Animation Compression Library，UE 5.3 起是默认的骨骼压缩编解码器）不看局部角度，而是在每根骨骼周围放一圈**虚拟顶点**（距离骨骼若干厘米的点，模拟蒙皮网格），把它们按完整层级变换到**组件空间**，比较压缩前后的位移。这样误差直接对应“屏幕上网格会偏多少厘米”，并且自动考虑了层级累积：越靠近根的骨骼影响的子孙越多，就会被分配更高的精度。挂长武器的骨骼可以用更大的虚拟顶点距离，防止枪口抖动。

面试注意：压缩相关的高频追问

- **“为什么手指 / 武器在压缩后抖？”** 误差在层级末端被放大（手指在 11 层深处），或者长武器把小角度误差放大成大位移。对策：调整这些骨骼的误差设置 / 虚拟顶点距离，或单独给这类动画更严格的压缩设置。
- **“压缩的代价只在内存吗？”** 还有**解压 CPU**：每个角色每帧每条参与混合的动画都要解压一次。混合空间里 3 个采样、再叠一个状态机过渡，就是 4~6 次解压。这也是惯性化能省开销的原因（第 7 章）。
- **“Root 骨骼的误差为什么特别要紧？”** 它影响所有骨骼，而且 Root Motion 的位移直接驱动角色移动，误差会累积到位置上。

本章速记

- LBS：骨骼矩阵按权重相加再乘顶点，**GPU 蒙皮**；每顶点影响数、每 Section 骨骼数都有上限。
- LBS 扭转时半径 = **cos(θ/2)**（E3：180° 塌成 0）；UE 用**扭转骨 + 修正形变**解决，DQS 引擎没内置。
- AnimSequence = 骨骼轨道 + 曲线 + Notify + 同步标记 + Root Motion + 加性 / 压缩设置；普通 Notify 排队到游戏线程派发。
- 压缩三板斧：去常量轨（3.37x）→ 量化（8.60x）→ 关键帧精简 / 可变位率；**误差沿层级累积**（局部 0.5° → 组件 1.64°），ACL 用组件空间的虚拟顶点衡量。

▶ **自测：为什么量化四元数时可以丢掉 w？丢掉之后符号怎么办？**

▶ **自测：角色在 30 fps 的动画数据下，游戏跑 120 fps，画面会“一卡一卡”吗?**

PART TWO

第二部分　混合：动画系统的核心运算

混合空间、状态机过渡、分层混合、加性叠加，底下都是同一件事：把几个姿势按权重合成一个。先把数学弄对，再看 UE 把它包装成了哪些工具。

CHAPTER 03

混合的数学：四元数、多路混合与加性动画

一个姿势就是一组骨骼变换。混合两个姿势，就是对每根骨骼的平移、旋转、缩放分别插值——难点全在旋转。

本章目标

- 说清为什么旋转混合必须用四元数，并且要做“最短路径”符号修正
- 比较 nlerp 和 slerp 的误差与速度，理解 UE 为什么用“加权求和再归一化”
- 理解加性动画的定义、基准姿势、局部空间加性与网格空间加性的区别
- 知道 Aim Offset 本质是什么

先修：B3 什么是四元数

本章默认你知道四元数 q = (cos θ/2, n·sin θ/2)、q 与 −q 是同一旋转、乘法不交换。如果这些还陌生，先读前置篇 B3（有轴角 → 四元数、万向节锁两个交互实验），再回来看会顺很多。

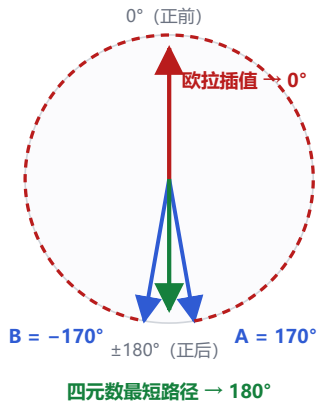
3.1 混合一个姿势：逐骨骼、逐分量

两姿势混合（与 FAnimationRuntime::BlendTwoPosesTogether 的思路一致）

```
// Alpha = 0 → 全是 A；Alpha = 1 → 全是 B
for (int32 i = 0; i < NumBones; ++i)
{
    Out[i].Translation = FMath::Lerp(A[i].Translation, B[i].Translation, Alpha);
    Out[i].Scale3D     = FMath::Lerp(A[i].Scale3D,     B[i].Scale3D,     Alpha);
    FQuat QB = B[i].Rotation;
    if ((A[i].Rotation | QB) < 0.f) QB = -QB;                  // 最短路径：dot < 0 就取反
    Out[i].Rotation = (A[i].Rotation * (1.f - Alpha) + QB * Alpha).GetNormalized();   // nlerp
}
```

多路混合（混合空间的 3 个采样、Blend Multi）同理：所有姿势按权重**累加**，旋转累加前先和第一个参与者比较符号，最后统一归一化。UE 里对应 `FTransform::AccumulateWithShortestRotation` + `NormalizeRotation`，权重在进入混合前会被归一化到和为 1。

3.2 为什么不能对欧拉角插值



同一个旋转有两个四元数：q 和 −q

Yaw 170° → q = (cos 85°, 0, 0, sin 85°)

Yaw −170° → q = (cos 85°, 0, 0, −sin 85°)

dot = cos²85° − sin²85° = cos 170° ≈ −0.985 < 0

不取反直接 lerp：走 340° 的远路 → 结果 0°

dot < 0 时把 B 取反再 lerp：走 20° 的近路 → 180°

欧拉角还有万向节锁、三个轴的插值顺序不唯一等问题，
所以 UE 的姿势数据和混合一律使用四元数（FRotator 只用于编辑和逻辑）

图 3-1 实测 E2：Yaw 170° 与 −170° 各占 50%，欧拉插值和“不检查符号的四元数 lerp”都得到 0°，nlerp（带符号修正）和 slerp 得到 180°

3.3 nlerp 还是 slerp

slerp 沿球面大圆匀速插值，角度随 t 线性变化；nlerp 是直接线性插值四个分量再归一化，路径相同但**角速度不均匀**（中间快、两头慢）。两者的偏差随夹角增大：

两姿势夹角	**30°**	**60°**	**90°**	**120°**	**150°**	**179°**
nlerp 相对 slerp 的最大偏差（E2）	0.07°	0.27°	0.92°	2.23°	4.51°	8.00°

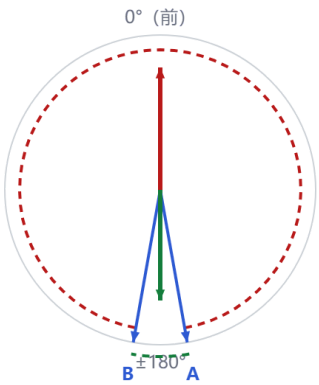
速度上（E2，2000 万次）：nlerp 2.73 ns/次，slerp 13.27 ns/次，**slerp 慢 4.86 倍**（多了 acos 和两次 sin）。

UE 姿势混合用的是 nlerp（加权求和 + 归一化），原因有三：

1. **快**：每个角色每帧每根骨骼要混合好几次，这是动画求值里调用最频繁的操作之一。

2. **误差小**：参与混合的通常是相近的姿势（同一套运动的不同方向、同一时刻的两条动画），同一根骨骼的夹角很少超过 30°~60°，偏差不到 0.3°。
3. **多路混合与顺序无关**：见下一节。slerp 只能两两混合。

交互实验 • 两个 Yaw 角的混合：欧拉插值 vs 四元数





欧拉插值	0.0°
四元数 nlerp	-180.0°
slerp	-180.0°
两者夹角	20°
nlerp 最大偏差	0.01°
欧拉 vs 正确	180.0°

拖动滑块：把 A、B 放在 ±180° 两侧，看欧拉插值怎么“绕远路”；右图是 nlerp 与 slerp 的角度随 t 的变化（夹角越大，nlerp 越偏离直线）。

3.4 多路混合：为什么“加权求和”比“链式 slerp”好

三个姿势各占 1/3。用 slerp 只能这样做：`slerp(slerp(A, B, 1/2), C, 1/3)`。换一个顺序，结果就变了：

实测 E2：Yaw0 / Yaw80 / Pitch80 三路等权混合

- 加权求和 + 归一化：三种顺序的结果两两最大夹角 **0.0000°**（加法满足交换律）。
- 链式 slerp：三种顺序的结果两两最大夹角 **1.7150°**，与加权求和最多差 1.412°。

混合空间里三个采样权重每帧都在变，参与者的顺序也可能变，**结果不应该依赖遍历顺序。**这也是 UE 选择“累加再归一化”的根本原因。

面试注意：符号修正要以谁为参考

多路累加时，每个四元数都要和**同一个参考**（通常是第一个参与者，或当前累加值）比较 dot 再决定是否取反。如果两两之间各自修正，三个姿势分布在超过 180° 的范围里时，可能出现 A 与 B 同侧、B 与 C 同侧、但 A 与 C 异侧的情况，结果会跳变。实际动画里很少遇到，但自己写混合节点时要意识到这一点。

3.5 加性动画（Additive）

普通混合是“在 A 和 B 之间取一个”，加性是“在 A 上**叠加**一个差值”。差值从一条动画减去一个基准姿势得到：

加性动画的定义与应用

```
制作（导入时预计算）：
    ΔT = T_anim − T_base
    ΔR = R_anim · R_base⁻¹                  // 四元数“相减”就是乘逆
    ΔS = S_anim / S_base（UE 内部存为相对 1 的偏移）

应用（Apply Additive，Alpha 为强度）：
    T = T_target + α·ΔT
    R = slerp(Identity, ΔR, α) · R_target
    S = S_target · (1 + α·(ΔS − 1))
```



图 3-2 加性动画只保存“相对基准姿势的变化量”，可以叠加到任意基础姿势上：受击、呼吸、身体倾斜、瞄准偏移都这么做

设置	选项	说明
Additive Anim Type	**Local Space**	差值在每根骨骼的局部空间计算和应用，最常用（受击、呼吸、倾斜）
	Mesh Space	差值在组件空间计算和应用，结果的“朝向”与基础姿势的脊柱弯曲无关，Aim Offset 必须用它
Base Pose Type	Reference Pose	相对绑定姿势（T/A-Pose）求差
	Selected Animation Scaled	相对另一条动画逐帧求差（时长不同会按比例缩放）
	Selected Animation Frame	相对另一条动画的某一帧求差（Aim Offset 常用：中间朝前的那一帧）

深入：局部空间加性 vs 网格空间加性

假设加性动画是“上半身往右看 30°”，基础姿势是“弯腰捡东西”。局部空间加性把 30° 加在每节脊柱的局部旋转上，而这些骨骼已经弯下去了，转动轴跟着倾斜，最终视线不是水平往右 30°。网格空间加性在组件空间里叠加 30°，**不管基础姿势怎么弯，视线都水平往右转 30°**。代价是网格空间加性要做组件空间转换，更贵。

3.6 Aim Offset：网格空间加性的混合空间

Aim Offset 是一种特殊的混合空间（第 4 章），轴通常是 Yaw 和 Pitch，每个采样是一个**网格空间加性姿势**（瞄准左上、正上、右上……）。运行时按瞄准角算出加性姿势，叠加到当前的运动姿势上。

- 采样动画的 Additive Anim Type 必须是 **Mesh Space**，Base Pose 一般用“Selected Animation Frame”，指向朝正前方的那一帧。
- 一般放在 AnimGraph 里运动状态机之后、上半身分层混合之后，这样无论跑、走、蹲，瞄准方向都能叠加上去。
- 角色转身时，常把 Yaw 限制在 ±90° 内，超出后触发原地转身（Turn In Place），让下半身跟上。

本章速记

- 姿势混合 = 每根骨骼：平移 lerp、缩放 lerp、旋转**带符号修正的 nlerp**。
- 欧拉插值会走反方向（E2：170° 和 −170° 混合得 0°）；q 与 −q 表示同一旋转，**dot < 0 要取反**。
- nlerp 比 slerp 快 4.86 倍，60° 内偏差 < 0.3°；多路混合用**加权求和再归一化**，结果与顺序无关（链式 slerp 差 1.7°）。
- 加性 = 动画 − 基准姿势；局部空间加性常用于受击 / 呼吸，网格空间加性用于 **Aim Offset**。

▶ **自测：给一个加性动画 Alpha = 2 会怎样？Alpha 为负呢?**

▶ **自测：什么情况下 nlerp 的误差会大到肉眼可见?**

CHAPTER 04

混合空间与同步组：让走跑自然过渡

输入一个速度和方向，输出一个“介于走和跑之间、偏右前”的姿势。要做到不滑步，权重要算对，脚步还要对齐。

本章目标

- 知道 1D / 2D 混合空间如何把输入参数变成采样权重：三角剖分 + 重心坐标
- 区分“输入平滑（Smoothing Time）”和“采样权重速度（Weight Speed）”
- 理解为什么混合不同长度的循环动画必须同步，以及归一化时间同步与同步标记的区别
- 知道 Leader / Follower 的规则，以及混合空间内部是怎么同步采样的

4.1 混合空间是什么

混合空间（Blend Space）是一种动画资产：定义 1 或 2 个输入轴（例如速度、方向），在轴上摆若干**采样**（每个采样是一条动画），运行时根据输入值算出每个采样的权重，把它们同步推进、求值、按权重混合。

资产	轴	采样	典型用途
Blend Space 1D	1 个	普通序列	Idle → Walk → Jog → Run 按速度混合
Blend Space	2 个	普通序列	八方向移动（Vx / Vy 或 方向 / 速度）
Aim Offset (1D)	1~2 个	**网格空间加性**序列	按 Yaw / Pitch 叠加瞄准偏移

4.2 权重怎么算

1D：只看左右两个邻居

采样按轴值排序，输入落在哪两个采样之间，就按距离线性分配：速度 290 落在 Walk(160) 和 Jog(420) 之间，Jog 权重 = (290−160)/(420−160) = 0.5。

2D：三角剖分 + 重心坐标

> **白话版：B4.5**
>
> “重心坐标”可以理解成：输入点把三角形切成三块，每个顶点的权重 = 它对面那块的面积占比。前置篇 B4.5 用同一组数字画了一张图。

UE5 默认把所有采样点做**Delaunay 三角剖分**（编辑器里可以点 Show Triangulation 看到），输入点落在哪个三角形里，三个顶点的**重心坐标**就是权重；其余采样权重为 0。重心坐标保证：权重非负、和为 1、而且按权重加权的采样坐标正好等于输入点——**混合出来的“速度”就是你要的速度**，这正是不滑步的前提。

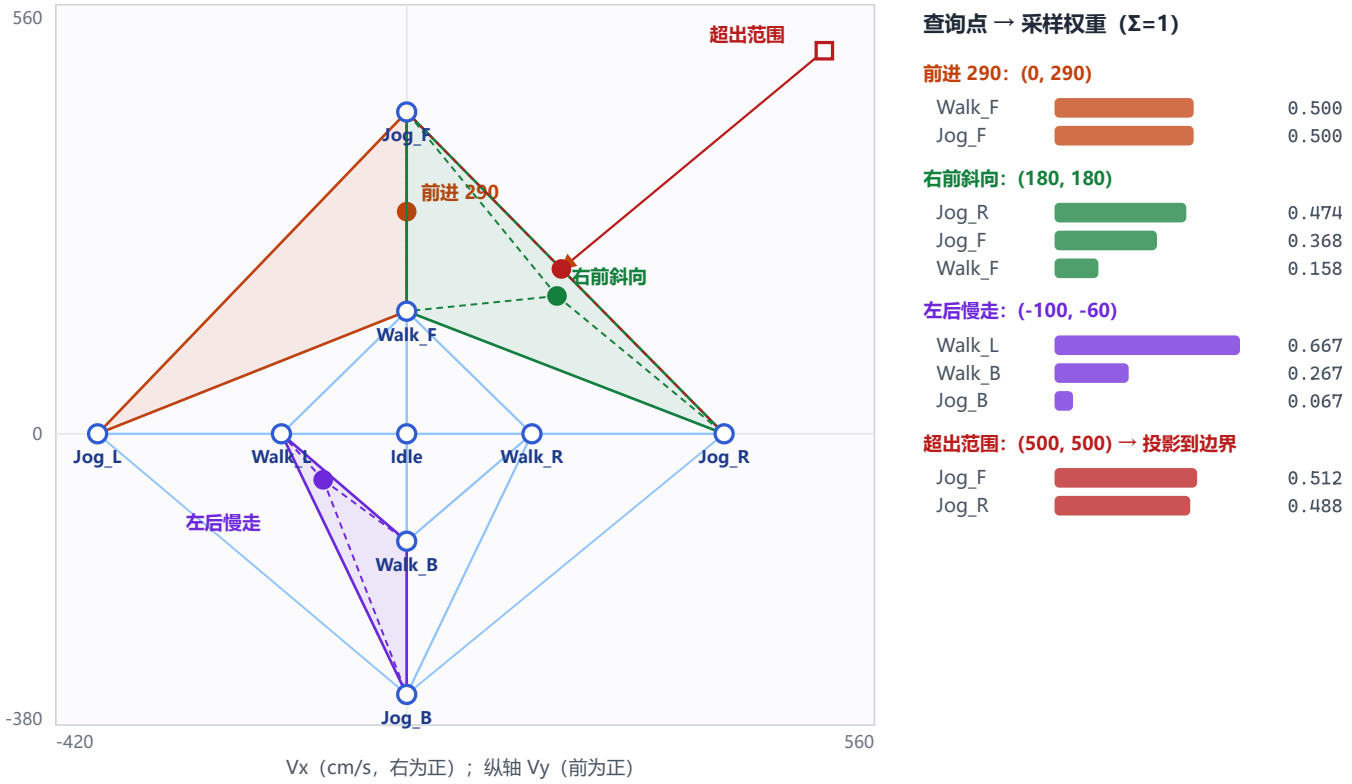


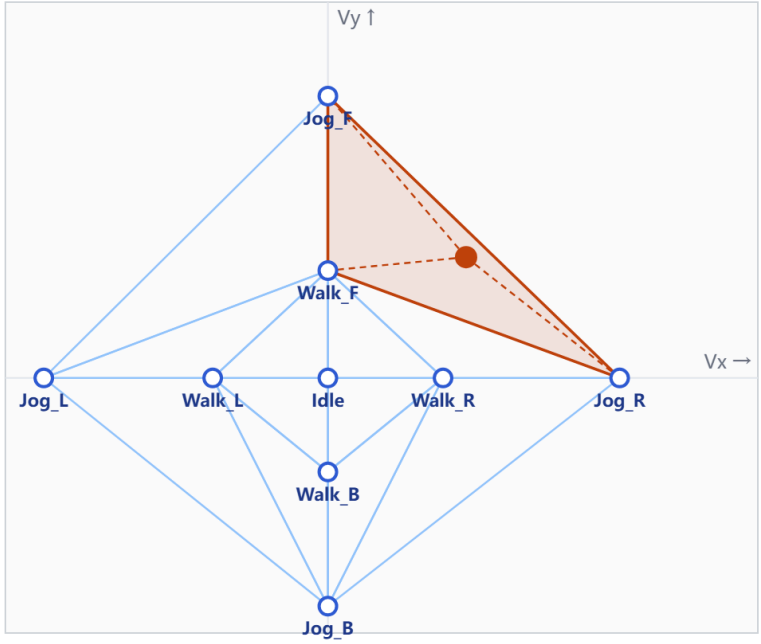
图 4-1 9 个采样的 Delaunay 三角剖分（12 个三角形，由构建脚本计算，与 E5 一致）。查询点所在三角形高亮，虚线连到参与混合的采样；落在三角网外的输入投影到最近的边界边

实测 E5：权重与重建

输入 (Vx, Vy)	权重	Σw	加权重建的坐标
(0, 0) 原地	Idle 1.000	1	(0, 0)
(0, 290) 前进	Walk_F 0.500 · Jog_F 0.500	1	(0, 290)
(180, 180) 右前	Walk_F 0.158 · Jog_F 0.368 · Jog_R 0.474	1	(180, 180)
(−100, −60) 左后	Walk_B 0.267 · Walk_L 0.667 · Jog_B 0.067	1	(−100, −60)
(500, 500) 超出	Jog_F 0.512 · Jog_R 0.488	1	(185, 215) ← 被限制在边界上

插值方式	做法	特点
三角剖分（默认）	Delaunay + 重心坐标，最多 3 个采样参与	采样可以随意摆放；斜向过渡顺滑；当采样均匀摆在网格上时有“哪条对角线”的歧义，用 Preferred Triangulation Direction（Tangential / Radial）指定，保证左右对称
网格（Use Grid，旧版）	先把轴切成 Grid Divisions 个格子，格子四角双线性插值，最多 4 个采样参与	采样必须落在网格交点上；主要为兼容旧项目和 Wrap Input（方向轴首尾相接）这类场景

交互实验 • 2D 混合空间：拖动输入点看权重



采样权重

Jog_R		0.474
Jog_F		0.368
Walk_F		0.158

```
输入    (180, 180)
Σw      1.000
重建    (180, 180)
三角形  12 个
```

在左图中按住并拖动橙色点（输入速度）。高亮三角形的三个顶点就是参与混合的采样；拖到外圈以外时，输入被投影到最近的边界上。

4.3 轴怎么选

- **方向 + 速度**（Direction −180~180，Speed 0~600）：直观，但方向轴首尾（±180° 都是后退）需要开 **Wrap Input**；速度为 0 时方向没有意义，Idle 要在 Speed=0 那一行摆满。
- **Vx + Vy（局部速度）**：如图 4-1，没有角度环绕问题，原点就是 Idle。
- **只有 1D 方向 + 播放速率**：UE5 的 Lyra / Game Animation Sample 走的是另一条路——方向用少量“基本方向”动画，速度不靠混合空间，而是用**距离匹配**和**步幅调整（Stride Warping）**去贴合真实速度（第 8 章）。

面试注意：采样的“速度”必须和动画里根骨骼的真实速度一致

采样摆在 Jog_F = 420 的位置，意思是“这条动画播放时脚每秒向后推 420 cm”。如果美术做的动画其实是 380 cm/s，角色以 420 移动时就会**滑步**。UE 的 Blend Space Analysis 可以按根骨骼位移自动把采样放到正确位置；也可以开启 **Axis to Scale Animation**，让引擎在输入超出混合结果时按比例调整播放速率。

4.4 两种“平滑”：别混为一谈

键盘输入从“左”瞬间切到“右”时，混合空间的输入会突变。UE 有两种完全不同的平滑：

选项	平滑的是什么	E5：1D 方向混合空间，−90° 突变到 +90°（0.3 s）
Smoothing Time / Type（输入平滑）	输入参数本身：输入从 −90 慢慢变到 +90，**中间会经过 0°**	途经 Fwd：Fwd 最大权重 **1.00**，占主导 9 帧——角色先往前走一下再往右
Weight Speed（采样权重速度）	每个采样的权重：直接从“Left=1”往“Right=1”淡过去	Fwd 最大权重 **0.00**——从左直接过渡到右，不经过中间状态

Smoothing Type 可选 Averaged、Linear、Cubic、Ease In/Out、Exponential、Spring Damper（可设阻尼比、最大速度）。**官方文档建议两者不要同时使用。**Weight Speed 还可以按骨骼单独设置（Per Bone Overrides，例如让头比身体更快转向目标）。

4.5 同步组：为什么要同步

走路循环 1.2 s、跑步循环 0.7 s。如果各自按自己的时间播放再混合，两条动画的脚步相位会不断错开：一个在迈左脚，一个在迈右脚，混合出来两只脚都悬在半空或者交叉。

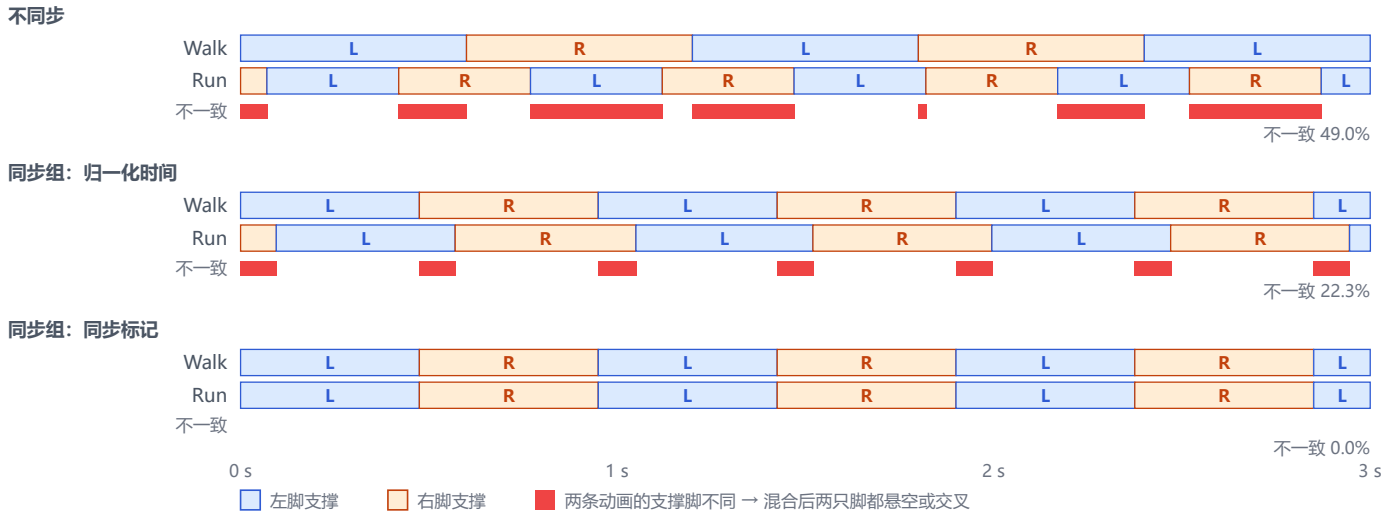


图 4-2 Walk（L 标记在 0.00、R 在 0.50）与 Run（L 在 0.10、R 在 0.60）各占 50%。不同步时一半时间支撑脚相反；归一化时间同步后仍有 20% 不一致（两条动画的落脚时机本身差了 10% 周期）；按同步标记对齐后完全一致

实测 E6：20 秒 @60fps

方式	支撑脚不一致的帧	说明
不同步（各播各的）	**50.4%**	相位差在 0~0.5 之间来回漂
同步组：归一化时间	**21.2%**	归一化位置相同，但左脚落地事件差 95 ms
同步组：同步标记	**0.0%**	按 L / R 标记对齐，归一化位置反而差 0.10

同步组的规则

- 动画播放节点（序列播放器、混合空间）的 Sync 设置里填一个 **Group Name**，同名的节点组成一组。
- 每帧组内**权重最高**、且角色允许当 Leader 的那个成为 **Leader**：它按自己的时长和播放速率推进时间。其余成员是 **Follower**：不自己推进，而是直接对齐到 Leader 的归一化位置（或标记位置）。
- **Group Role**：Can Be Leader（默认）、Always Leader、Always Follower、Transition Leader / Transition Follower（只在状态机过渡期间参与同步）等。
- 没有放进同步组的混合空间，内部采样**也是自动同步的**：混合空间作为一个整体，周期取各采样时长的加权平均（E6 的 0.5×1.2 + 0.5×0.7 = 0.95 s），各采样按同一个归一化位置（或同步标记）推进。

深入：Leader 切换时会怎样

两个独立的序列播放器放进同一组，Walk 权重 0.6 时 Leader 是 Walk，节奏是 1.2 s；权重越过 0.5 后 Leader 变成 Run，节奏立刻变成 0.7 s。因为 Follower 一直对齐着 Leader 的相位，所以**姿势不会跳**，但**步频会突然变**。这就是为什么移动混合通常放在一个混合空间里（周期按权重连续变化），而不是用两个播放器 + Blend 节点。

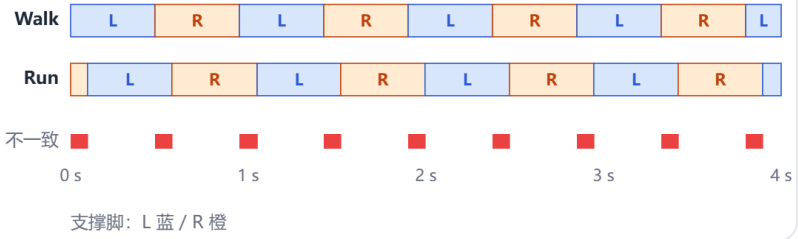
4.6 同步标记：按脚步对齐

在动画的 Sync Marker 轨道上标出 L、R（左脚 / 右脚落地）。开启标记同步后，Follower 不再对齐归一化时间，而是对齐“**距离上一个同名标记走了多远**”：Leader 刚过 L 标记 30%，Follower 也放到自己的

L 与下一个 R 之间 30% 的位置。

- 所有参与者必须有**同一套**标记名，否则退回归一化时间同步。
- 可以处理“两条动画落脚时机不一样”（E6），甚至“一个周期里步数不同”的情况。
- 起步、急停这类非循环动画也可以带标记，从循环过渡进来时从对应的脚开始。

交互实验 · 同步组与同步标记



支撑脚不一致	21.5%
最大相位差	0.00
周期	0.95 s（加权）
权重最高	Walk（并列）
Walk L 标记	0.00
Run L 标记	0.10

调 Run 的权重和它的左脚标记位置，切换同步方式，观察两条动画的支撑脚是否一致（红色 = 不一致）。

游戏客户端实战：混合空间常见问题排查

- **斜向移动时手 / 武器抖**：两个独立的混合空间（人物和武器各一个）三角剖分或网格设置不一致，同一输入算出的权重不同。让它们共用同一套采样布局，或让武器跟随角色骨骼。
- **急转向时角色先“刹一下”**：Vx/Vy 混合空间里输入从左穿过原点到右，经过了 Idle。换成 Weight Speed，或者用方向 + 速度轴并让速度保持。
- **脚步声响两次**：混合中的两条动画都有脚步 Notify。混合空间设 Notify Trigger Mode = Highest Weighted Animation。

本章速记

- 2D 混合空间：**Delaunay 三角剖分 + 重心坐标**，最多 3 个采样，权重之和为 1，加权坐标 = 输入（E5）；三角网外投影到边界。
- 采样位置要与动画真实速度一致，否则滑步；Blend Space Analysis / Axis to Scale Animation 辅助。
- **输入平滑会经过中间状态，Weight Speed 不会**（E5：Fwd 权重 1.00 对 0.00），两者别同时用。

- 不同长度循环不同步 → 50% 时间脚步相反；**同步组**：权重最高者当 Leader，其余对齐其归一化位置；**同步标记**按 L/R 对齐（E6：21.2% → 0%）。
- 混合空间内部采样自动同步，周期按权重加权；独立播放器的 Leader 切换会让步频突变。

▶ **自测：为什么 Idle 不需要和走跑同步，但起步动画需要同步标记?**

▶ **自测：2D 混合空间里，为什么采样不能全部共线?**

CHAPTER 05

分层骨骼混合：上半身换弹，下半身照跑

Layered Blend per Bone 是 AnimGraph 里出镜率最高的节点之一。它的两个参数——Blend Depth 和 Mesh Space Rotation Blend——也是面试里最爱问的细节。

本章目标

- 掌握 Layered Blend per Bone 的输入、Branch Filter 与 Blend Mask 两种模式
- 能准确说出 Blend Depth 为 0、正数、负数时每根骨骼的权重
- 理解局部空间与组件空间旋转混合的区别，以及组件空间混合的代价
- 会画一个“运动状态机 + 上半身 Slot + 瞄准偏移 + IK”的典型 AnimGraph

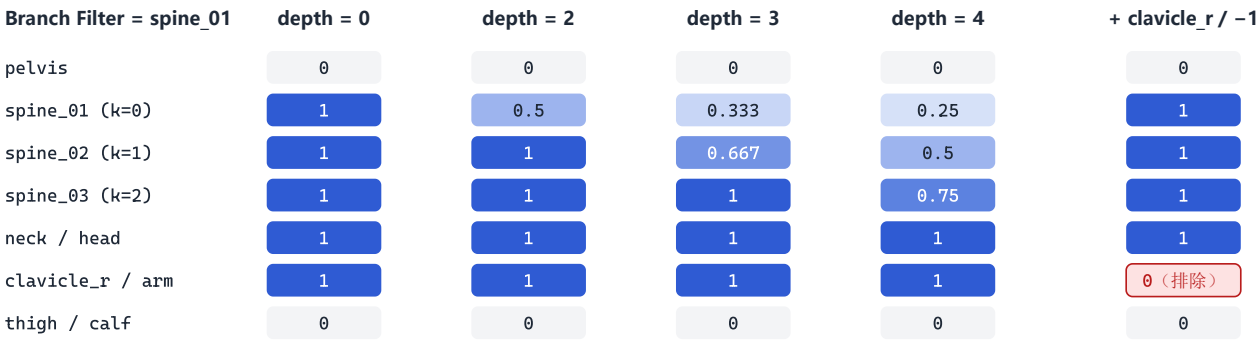
5.1 节点长什么样

- **Base Pose**：底层姿势，一般是运动状态机的输出。
- **Blend Poses 0..N**：要叠上去的姿势，例如上半身蒙太奇 Slot 的输出；每个有一个 **Blend Weight**（0~1，整体强度）。
- **Layer Setup**：每个 Blend Pose 对应一组规则，决定**哪些骨骼**用它、各用多少。两种模式：
  - **Branch Filter**：列出若干 (Bone Name, Blend Depth)，按骨骼分支计算权重。
  - **Blend Mask**：引用一个混合遮罩资产（Blend Profile 的遮罩模式），逐骨骼直接指定 0~1 的权重，可视化编辑、可复用、可运行时切换。
- 常用开关：**Mesh Space Rotation Blend**、Mesh Space Scale Blend、Curve Blend Option（曲线如何合并）、Blend Root Motion Based on Root Bone（Root Motion 按根骨骼权重混合）。

5.2 Blend Depth：权重怎么分配

设 Branch Filter 的骨骼为 B，B 子树里某骨骼到 B 的层数为 k（B 自己 k = 0）：

Blend Depth	权重	含义
0	B 及所有子孙 = 1	从 B 开始“硬切”
N > 0	min(1, (k+1) / N)	从 B 往下 N 层内线性渐入，之后全是 1；Depth = 1 与 0 效果相同
负数（−1）	B 及所有子孙 = 0	把这个分支从混合里**排除**，常与其他正向规则配合
不在任何分支里	0	保持 Base Pose



数值来自实测 E4（与 FAnimationRuntime::CreateMaskWeights 的规则一致）；最右一列为 spine_01/0 再加 clavicle_r/−1

图 5-1 Blend Depth 的效果：正数让权重从过滤骨骼开始逐层渐入，避免上下半身交界处“折断”；负数把某个分支挖掉（例如右手保持持枪，只让左手做动作）

面试注意：Blend Depth 不是“影响几层骨骼”

很多人以为 Depth = 3 表示“只影响 spine_01 往下 3 层”。实际上它是**渐变的层数**：3 层之内权重 1/3、2/3、1 逐步增加，**3 层之后的子孙全部是 1**，影响范围仍是整棵子树。想“只影响某几根”，应该用多个 Branch Filter 配合 −1 排除，或者直接用 Blend Mask。

5.3 局部空间混合的问题：上半身被骨盆带偏

场景：角色向右前方斜跑，跑步动画里骨盆朝右扭了 40°，脊柱三节各回扭 −10°（于是头朝 +10°）。上半身播放一个“朝正前方举枪”的动画（制作时骨盆、脊柱都朝前，右臂 +30°）。我们希望：**不管腿怎么跑，枪口都朝正前方。**



图 5-2　每根骨骼在组件空间里的朝向（构建时按 E4 的同一规则计算）。Local 混合时，spine_01 拿到的是瞄准动画的**局部**旋转，于是整个上半身随骨盆偏了 40°（depth 3 时偏 30°）；Mesh Space 混合直接取瞄准动画的**组件空间**朝向，头和右臂都对准目标

实测 E4

模式	头部朝向	右上臂朝向	spine_01 / 02 / 03
Local Space · depth 0	**40.00°**	**70.00°**	40 / 40 / 40
Mesh Space · depth 0	**0.00°**	**30.00°**	0 / 0 / 0
Local Space · depth 3	**30.00°**	**60.00°**	33.33 / 30 / 30
Mesh Space · depth 3	**0.00°**	**30.00°**	20.03 / 6.66 / 0（逐节回正）

耗时（68 骨骼 Mannequin，Branch Filter = spine_01）：局部空间 **0.260 us**/次；Mesh Space 旋转混合 **3.429 us**/次，**13.2 倍**——因为要把两份姿势都做一遍 FK、在组件空间插值、再转回局部空间。

Mesh Space · depth 3 是最常见的组合：脊柱逐节从“跟着骨盆”过渡到“对准目标”，上下半身的扭转被分摊到三节脊柱上，看起来像真人拧腰，而头和枪完全对准目标。

交互实验 · 分层骨骼混合：Blend Depth 与 Mesh Space

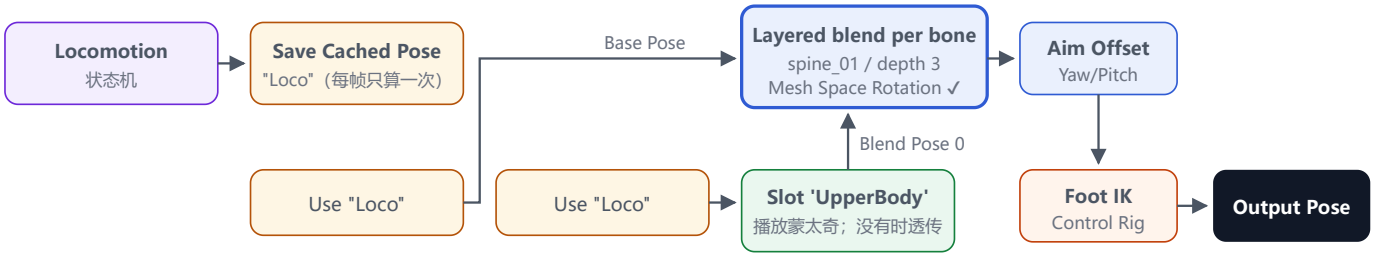


头部朝向　　40.00°（期望 0°）
右上臂朝向　　70.00°（期望 30°）
头部误差　　40.00°

`Local`：取瞄准姿势的局部旋转，上半身继承骨盆的扭转。
`Mesh`：取组件空间朝向，与骨盆无关。

骨架：pelvis → spine_01 → spine_02 → spine_03 → neck → head，spine_03 → clavicle_r → upperarm_r。Branch Filter = spine_01。上半身瞄准姿势：头朝 0°、右臂 30°（组件空间）。

5.4 典型的 AnimGraph 结构



顺序很重要：先合成全身姿势 → 再叠加瞄准（网格空间加性）→ 最后做 IK 修正（脚贴地、手握枪）

图 5-3　一个常见的射击游戏 AnimGraph。Cached Pose 让运动状态机只求值一次、被两处使用；没有蒙太奇时 Slot 直接输出输入姿势，分层混合等于没混

5.5 其他混合节点速览

节点	作用	要点
Blend	A、B 按 Alpha 混合	Alpha 为 0 或 1 时只求值一边（相关性裁剪）
Blend Poses by Bool / Int / Enum	按变量切换，带 Blend Time	切换是“过渡”，可选 Inertialization 方式
Blend Multi	多路按各自 Alpha 混合	可设为加性
Blend Bone by Channel	把某根骨骼的平移 / 旋转 / 缩放单独拷到另一根	可选计算空间
Apply Additive / Apply Mesh Space Additive	叠加加性姿势	受 LOD Threshold 控制，远处可以不叠
Save / Use Cached Pose	缓存一个子图的结果	Update 只调用一次（取最高权重的使用者），Evaluate 也只算一次

深入：权重为 0 的分支会不会被计算?

不会。AnimGraph 在 Update 阶段把权重传下去，权重接近 0 的子图被视为**不相关（not relevant）**，既不 Update 也不 Evaluate。这对性能很关键：状态机里不活跃的状态、Blend 里 Alpha=0 的一边都不花钱。它带来的副作用是——子图重新变得相关时，时间可能从头开始（序列播放器可选 “Reset on Become Relevant”），需要的话用同步组保持相位。UE5 还提供 **Anim Node Functions**（On Become Relevant / On Update / On Initial Update），可以在这些时机执行线程安全逻辑。

游戏客户端实战：分层混合常见问题

- **挥拳时胯不跟着转，看起来很僵**：过滤骨骼选得太靠上。可以从 pelvis 开始，同时把两条腿用 −1 排除，让胯部跟随上半身动作。
- **上下半身交界处“折断”**：Depth = 0 硬切。改成 2~3 层渐入，或用 Blend Mask 精细调。
- **蒙太奇的 Root Motion 被分层混合吃掉了**：根骨骼不在分支里，权重为 0。开启 Blend Root Motion Based on Root Bone，或在 Slot 之前处理 Root Motion。
- **跑动中开火，枪口方向抖**：用了 Local 混合，骨盆的摆动传到了枪上。开 Mesh Space Rotation Blend。

本章速记

- Layered Blend per Bone：Base + 多个 Blend Pose；Layer Setup 用 **Branch Filter** 或 **Blend Mask**。
- Blend Depth：**0 = 整个子树 1**；**N = 前 N 层 (k+1)/N 渐入，之后 1**；**−1 = 排除该分支。**
- Local 混合让上半身随骨盆偏（E4：头偏 40°）；**Mesh Space Rotation Blend** 保持组件空间朝向（0°），代价约 13.2 倍。
- 典型结构：状态机 → Cached Pose → Slot（上半身蒙太奇）→ 分层混合 → Aim Offset → IK → Output。
- 权重为 0 的分支不 Update 也不 Evaluate；重新相关时注意时间重置。

▶ **自测：分层混合里，平移也是在组件空间混合吗?**

▶ **自测：同一个分层混合节点里，两个 Blend Pose 的分支重叠了怎么办?**

PART THREE

第三部分　动画蓝图：把混合组织成一张图

动画蓝图决定“这一帧用哪些动画、按什么权重混”。它的线程模型、状态机与过渡、蒙太奇和 IK，是面试官区分“用过”和“懂”的地方。

CHAPTER 06

动画蓝图的架构与线程模型

EventGraph 和 AnimGraph 分别跑在哪个线程？Proxy 为什么存在？一个动画节点从初始化到输出姿势经历了什么？

本章目标

- 说清动画蓝图编译后是什么，EventGraph 与 AnimGraph 的分工
- 按顺序讲出一帧里 SkeletalMeshComponent 的动画调用链，并标出线程
- 理解 FAnimInstanceProxy、Property Access、线程安全函数、Fast Path 各自解决什么问题
- 能写一个最小的自定义动画节点，知道 Initialize / CacheBones / Update / Evaluate 的职责

6.1 动画蓝图编译后是什么

动画蓝图编译生成一个 `UAnimInstance` 的子类（`UAnimBlueprintGeneratedClass`）。AnimGraph 里的每个节点变成这个类里的一个 `FAnimNode_*` 结构体成员（如 `FAnimNode_SequencePlayer`、`FAnimNode_LayeredBoneBlend`），节点之间用 `FPoseLink` 连起来；EventGraph 和函数编译成普通蓝图字节码。

	EventGraph	AnimGraph
做什么	读角色 / 移动组件的状态，算出变量（速度、方向、是否在空中、瞄准角）	用这些变量驱动节点树，输出最终姿势
入口	`Event Blueprint Update Animation`（C++：`NativeUpdateAnimation`）	Output Pose 节点向下拉取
线程	**游戏线程**	开启多线程更新后在**工作线程**
能否访问 UObject	可以（Cast、调函数）	不能随意访问，只能读 Proxy 里的数据或线程安全的属性

UE5 推荐写法

把 EventGraph 里的逻辑搬到 **Blueprint Thread Safe Update Animation**（C++：`NativeThreadSafeUpdateAnimation`）里，在工作线程执行；需要读角色数据时用 **Property Access**（属性访问）节点，它由引擎在安全的时机把数据拷贝过来。Lyra 的动画蓝图几乎没有 EventGraph 逻辑，就是这么做的。

6.2 一帧的调用链

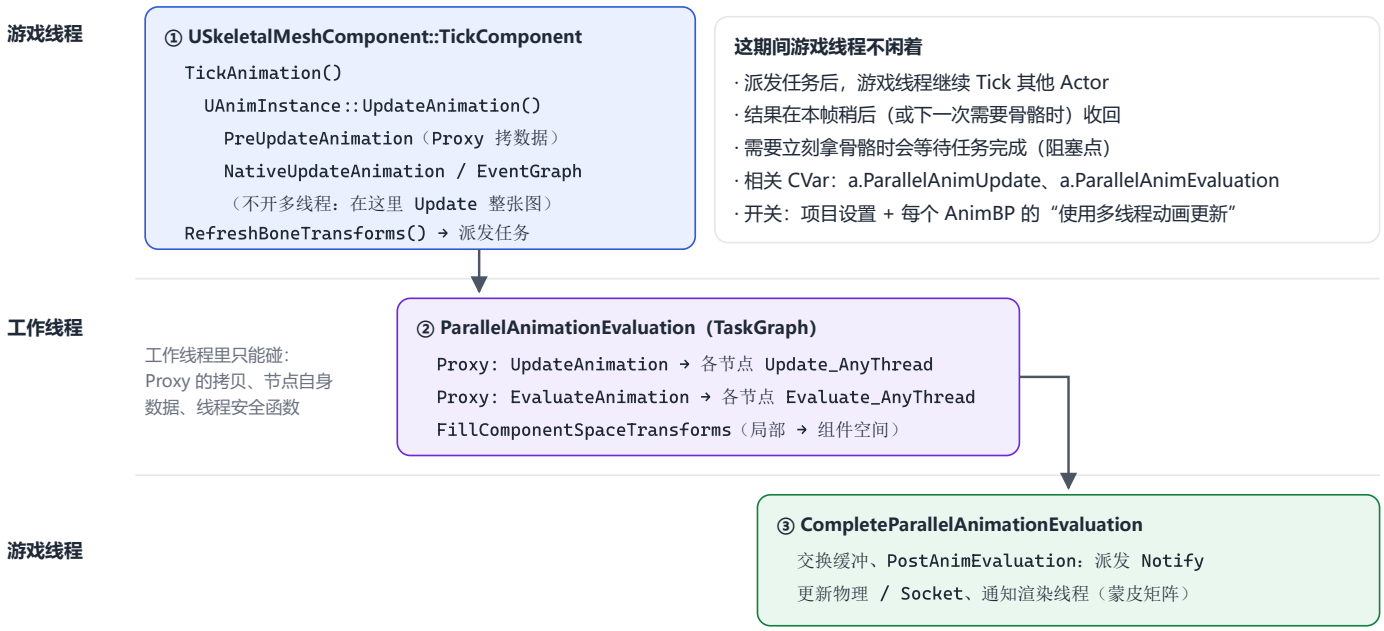


图 6-1 一个 SkeletalMeshComponent 每帧的动画调用链。① 在游戏线程准备，② 在工作线程计算（多线程开启时），③ 回到游戏线程收尾。Notify 在 ③ 派发，所以比姿势晚一步

深入：Root Motion 让调用顺序变了

角色播放带 Root Motion 的蒙太奇时，`UCharacterMovementComponent` 需要在**移动之前**就知道这一帧动画位移了多少。所以它会在自己的 Tick 里主动调用 `TickCharacterPose`，先推进动画时间并提取 Root Motion，再用这段位移执行移动；之后 SkeletalMeshComponent 的 Tick 不会再重复推进时间。这也是“Root Motion 角色的动画和移动是强耦合的”的含义（第 7 章）。

6.3 为什么要有 Proxy

工作线程计算姿势时，游戏线程可能正在改 Actor 的属性、销毁对象、跑 GC。如果动画节点直接读 `Character->GetVelocity()`，就是数据竞争。UE 的解法是 **FAnimInstanceProxy**：

- 动画实例里真正参与计算的状态（节点树、同步组、蒙太奇评估数据、要读的变量）放在 Proxy 里。
- 游戏线程在 `PreUpdateAnimation` 阶段把需要的数据**拷贝**进 Proxy；工作线程只读写 Proxy。
- 收尾阶段再把结果（Notify 队列、曲线值等）交还游戏线程。

C++ 动画实例：游戏线程拷数据，工作线程算

```cpp
UCLASS()
class UMyAnimInstance : public UAnimInstance
{
    GENERATED_BODY()
protected:
    UPROPERTY(BlueprintReadOnly) float GroundSpeed = 0.f;
    UPROPERTY(BlueprintReadOnly) float Direction = 0.f;
    UPROPERTY(BlueprintReadOnly) bool  bIsInAir = false;

    UPROPERTY(Transient) TObjectPtr<ACharacter> OwnerCharacter;   // 只缓存一次，不要每帧 Cast

    virtual void NativeInitializeAnimation() override
    {
        OwnerCharacter = Cast<ACharacter>(TryGetPawnOwner());
    }
    // 游戏线程：只做“读取 + 拷贝”，不做重计算
    virtual void NativeUpdateAnimation(float DeltaSeconds) override
    {
        if (!OwnerCharacter) return;
        Velocity  = OwnerCharacter->GetVelocity();
        ActorRot  = OwnerCharacter->GetActorRotation();
        bIsInAir  = OwnerCharacter->GetCharacterMovement()->IsFalling();
    }
    // 工作线程：只用上面拷好的值算派生量
    virtual void NativeThreadSafeUpdateAnimation(float DeltaSeconds) override
    {
        GroundSpeed = FVector(Velocity.X, Velocity.Y, 0.f).Size();
        Direction   = UKismetAnimationLibrary::CalculateDirection(Velocity, ActorRot);
    }
private:
    FVector  Velocity = FVector::ZeroVector;
    FRotator ActorRot = FRotator::ZeroRotator;
};
```

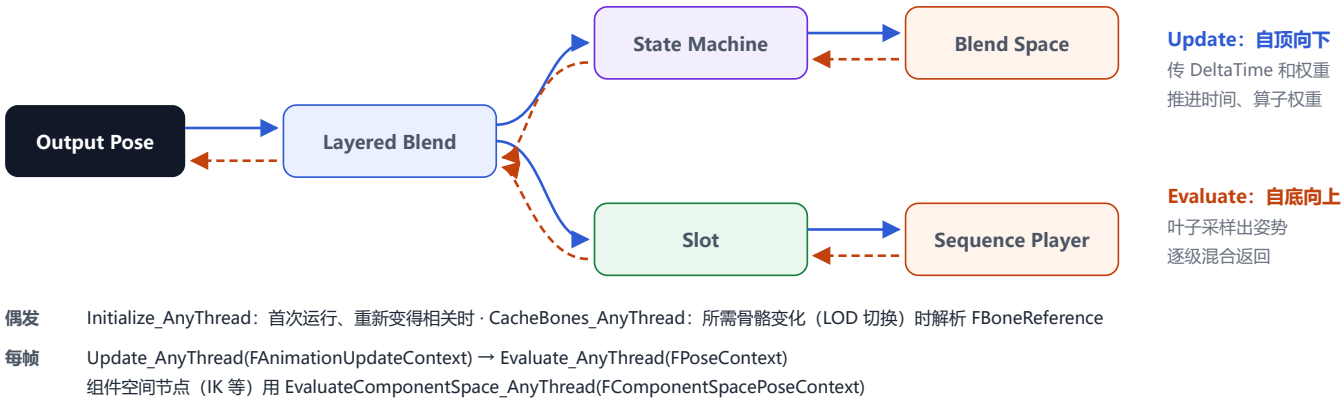
面试注意：EventGraph 里的性能坑

- **每帧 Cast**：`Try Get Pawn Owner → Cast To Character` 放在 Update 事件里，每帧每个角色都做一次。应在 Initialize 时缓存。
- **蓝图 VM 开销**：EventGraph 里的每个节点都走蓝图虚拟机，几十个角色乘起来就是游戏线程的大头。能挪到线程安全函数或 C++ 的就挪。
- **在动画蓝图里做射线检测、生成 Actor**：这些属于游戏逻辑，应该在角色或组件里做，动画蓝图只读结果。

6.4 Fast Path、Property Access 与线程安全函数

机制	解决什么	怎么用
Fast Path	AnimGraph 节点引脚的值如果来自蓝图逻辑，每次 Update 都要跑一段蓝图字节码	引脚直接连成员变量（或只做 Break Struct、取反 Bool 等简单操作），编译器把它变成直接内存拷贝，节点右上角出现闪电图标；否则编译时给出性能警告
Property Access	工作线程里需要读游戏线程对象的属性	用属性访问节点 / 绑定写路径（如 `TryGetPawnOwner.CharacterMovement.Velocity`），引擎在安全时机批量拷贝
Thread Safe 函数	自定义计算放到工作线程	函数标记为 Thread Safe，编译器检查里面不调用非线程安全的东西；可在 Blueprint Thread Safe Update Animation 或 Anim Node Functions 里调用
Anim Node Functions	节点级的初始化 / 更新逻辑	节点的 On Initial Update / On Become Relevant / On Update 绑定线程安全函数，例如进入状态时重置播放时间

6.5 一个动画节点的一生



偶发	Initialize_AnyThread：首次运行、重新变得相关时 · CacheBones_AnyThread：所需骨骼变化（LOD 切换）时解析 FBoneReference
每帧	Update_AnyThread(FAnimationUpdateContext) → Evaluate_AnyThread(FPoseContext) 组件空间节点（IK 等）用 EvaluateComponentSpace_AnyThread(FComponentSpacePoseContext)

图 6-2　AnimGraph 节点的生命周期。Update 从 Output 往叶子传权重（权重为 0 的子图被跳过），Evaluate 由 Output 向下拉取、叶子先算出姿势再逐级混合返回

一个最小的自定义动画节点：给脊柱加侧倾（局部空间）

```
USTRUCT(BlueprintInternalUseOnly)
struct FAnimNode_SpineLean : public FAnimNode_Base
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, Category = Links)
    FPoseLink Source;                                   // 输入姿势

    UPROPERTY(EditAnywhere, Category = Settings)
    FBoneReference SpineBone;                           // 按名字引用骨骼，运行时解析成 Compact

    UPROPERTY(EditAnywhere, Category = Settings, meta = (PinShownByDefault))
    float LeanDegrees = 0.f;                            // 暴露成引脚，最好用 Fast Path 绑定变量

    virtual void Initialize_AnyThread(const FAnimationInitializeContext& Context) override
    {
        FAnimNode_Base::Initialize_AnyThread(Context);
        Source.Initialize(Context);
    }
    virtual void CacheBones_AnyThread(const FAnimationCacheBonesContext& Context) override
    {
        Source.CacheBones(Context);
        SpineBone.Initialize(Context.AnimInstanceProxy->GetRequiredBones());    // LOD 变化时会
    }
    virtual void Update_AnyThread(const FAnimationUpdateContext& Context) override
    {
        GetEvaluateGraphExposedInputs().Execute(Context);  // 刷新引脚绑定的值
        Source.Update(Context);                             // 继续往下传
    }
    virtual void Evaluate_AnyThread(FPoseContext& Output) override
    {
        Source.Evaluate(Output);                            // 先拿到输入姿势
        const FCompactPoseBoneIndex Idx = SpineBone.GetCompactPoseIndex(Output.Pose.GetBoneCor
        if (Idx.IsValid())                                  // 当前 LOD 可能没有这根骨骼
        {
            FTransform& T = Output.Pose[Idx];
            const FQuat Lean(FVector::ForwardVector, FMath::DegreesToRadians(LeanDegrees));
            T.SetRotation((Lean * T.GetRotation()).GetNormalized());
        }
    }
};
// 编辑器里还需要一个 UAnimGraphNode_SpineLean（放在 Editor 模块）提供节点外观和标题
```

面试注意：自定义节点的三条铁律

1. **函数名带 _AnyThread 的都可能在工作线程跑**：不能访问 UObject 的可变状态、不能 Spawn、不能调非线程安全的引擎函数。
2. **必须把调用往下传**：Initialize / CacheBones / Update / Evaluate 都要调用输入 `FPoseLink` 的对应函数，否则子图不会运行。
3. **骨骼下标只能用 Compact Pose Index**，并在 CacheBones 里重新解析（第 1 章）。

6.6 拆分动画蓝图：Linked Anim Graph 与 Anim Layer

一个角色要支持十几种武器时，把所有逻辑塞进一个动画蓝图会变得不可维护。UE 提供两种拆分方式：

- **Linked Anim Graph**：在 AnimGraph 里放一个节点，运行另一个动画蓝图的 AnimGraph，像调用子函数一样。
- **Anim Layer + Anim Layer Interface**：主动画蓝图只定义“接口”（例如 FullBody_Idle、UpperBody_Aim 这些层），具体实现放在各个武器的动画蓝图里，运行时用 `LinkAnimClassLayers` 切换。Lyra 的“每种武器一个动画层蓝图”就是这么做的。
- **Animation Blueprint Template**：不绑定具体 Skeleton 的模板，可以被不同角色复用，具体的动画资产作为变量传入。

游戏客户端实战：切换武器层时的注意点

链接新的动画层会让新子图“重新变得相关”，序列时间和状态机状态都会重置，可能出现一帧的姿势跳变。通常在切换时配合一次惯性化（Inertialization）请求，并且把运动同步组放在主蓝图里，让新层的循环动画接上原来的脚步相位。

本章速记

- 动画蓝图 = UAnimInstance 子类；**EventGraph 在游戏线程**读状态写变量，**AnimGraph** 可在工作线程更新与求值。
- 调用链：TickComponent → TickAnimation（UpdateAnimation，NativeUpdate）→ RefreshBoneTransforms 派发 → 工作线程 Update + Evaluate + 组件空间 → Complete（派发 Notify、通知渲染）。

- **Proxy** 隔离数据；UE5 用 **Property Access + Thread Safe Update** 取代 EventGraph；**Fast Path** 让引脚绑定免跑蓝图 VM。
- 节点四件套：Initialize / CacheBones（解析骨骼）/ Update（时间、权重，自顶向下）/ Evaluate（姿势，自底向上）；记得往下传。
- 大型角色用 Linked Anim Graph / Anim Layer 拆分，切层时注意状态重置。

▶ **自测：关闭“多线程动画更新”后，动画还会在工作线程上算吗?**

▶ **自测：为什么普通 Notify 不在工作线程直接触发?**

CHAPTER 07

状态机、过渡与惯性化，蒙太奇与 Root Motion

状态机决定“现在该放哪类动画”，过渡决定“怎么换过去”，蒙太奇负责“代码说放就放”的一次性动作，Root Motion 决定“谁来移动角色”。

本章目标

- 掌握状态机的组成：状态、过渡规则、Conduit、State Alias、优先级
- 理解交叉淡化的代价，以及惯性化“只算目标姿势”的原理和限制
- 说清蒙太奇的 Slot / Section / Notify 结构和常用 API
- 知道 Root Motion 的四种模式，以及为什么网络游戏通常只用蒙太奇的 Root Motion

7.1 状态机的组成

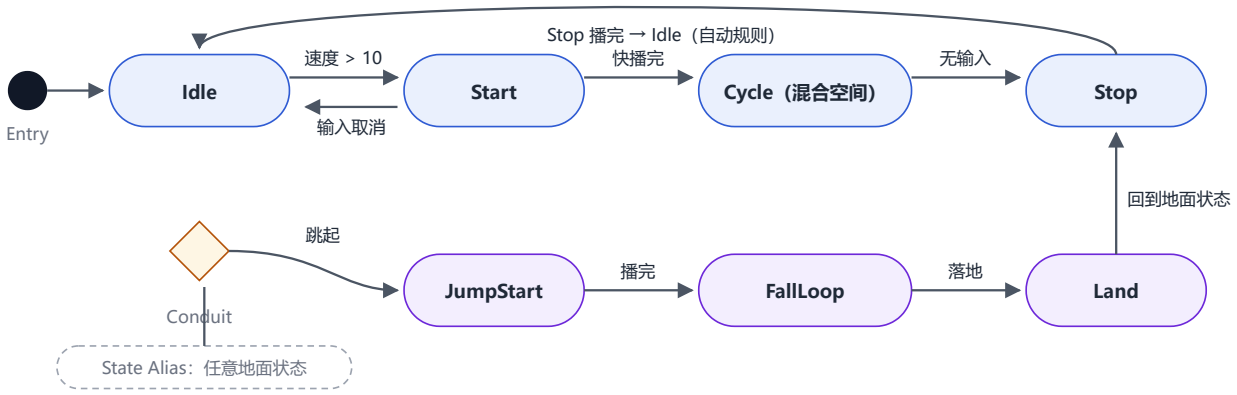


图 7-1 一个典型的运动状态机：起步 → 循环 → 急停，跳跃三段式。Conduit 把“从任意地面状态起跳”的共同判断收拢到一处；State Alias 代表一组状态，避免每个状态都连一条线

元素	说明
State	里面是一张子 AnimGraph（一个序列、一个混合空间、甚至嵌套状态机）
Transition Rule	返回 bool 的小图，满足就开始过渡。可勾选 **Automatic Rule Based on Sequence Player in State**：源状态的序列快播完时自动过渡（时间由过渡时长决定）
Priority Order	同一状态有多条过渡同时满足时，按优先级取一条
Conduit	不播放动画的“路口”，用来合并共同条件或做多路分发
State Alias	代表多个状态的别名，从它出发的过渡对这些状态都生效
过渡设置	Duration、Blend Logic（Standard Blend / **Inertialization** / Custom）、Blend Mode 曲线、Blend Profile（按骨骼不同速度）

面试注意：状态机里的规则要“读变量”而不是“算逻辑”

过渡规则每帧都要对当前状态的所有出边求值。把复杂判断（Cast、函数调用）写进规则里，开销会随状态数和角色数放大，还可能在工作线程里访问不安全的数据。正确做法是在线程安全更新里算好 `bShouldMove`、`bIsFalling` 这些布尔量，规则里只读变量（这样也能走 Fast Path）。

7.2 交叉淡化：过渡期间两边都要算

标准过渡（Standard Blend）在 Duration 内把源状态和目标状态的姿势按权重混合。**过渡期间两个状态都要 Update、Evaluate**：源状态里如果是一个 3 采样的混合空间，目标状态也是，那这几帧就要解压、混合 6 条动画。过渡被打断时（例如 A→B 过渡中又触发 B→C），UE 会把正在进行的过渡当作源，形成嵌套混合，最坏情况下同时求值三四个状态。

7.3 惯性化：只算目标，把差值衰减掉

惯性化（Inertialization，来自 Gears of War 4 的 GDC 2018 分享）换了个思路：切换的那一刻**立即只求值目标姿势**，同时记录切换瞬间源与目标的**差值 x_0 和差值的速度 v_0**，之后每帧把一个随时间衰减到 0 的偏移加到目标姿势上。衰减曲线是一个五次多项式，满足：

惯性化的偏移曲线（逐骨骼、逐分量）

```
x(0) = x0, x'(0) = v0                          // 从切换瞬间的位置差和速度差出发
x(t1) = 0, x'(t1) = 0, x''(t1) = 0             // 在 t1（过渡时长）处平滑归零
v0 若方向背离目标则钳为 0；若 v0 会让偏移提前过零，则缩短 t1 = min(t1, -5·x0/v0)
a0 = max(0, (-8·v0·t1 - 20·x0) / t1²)
x(t) = A t⁵ + B t⁴ + C t³ + a0/2 · t² + v0 t + x0
输出姿势 = 目标姿势(t) + x(t)                  // 旋转用轴角形式的差值
```

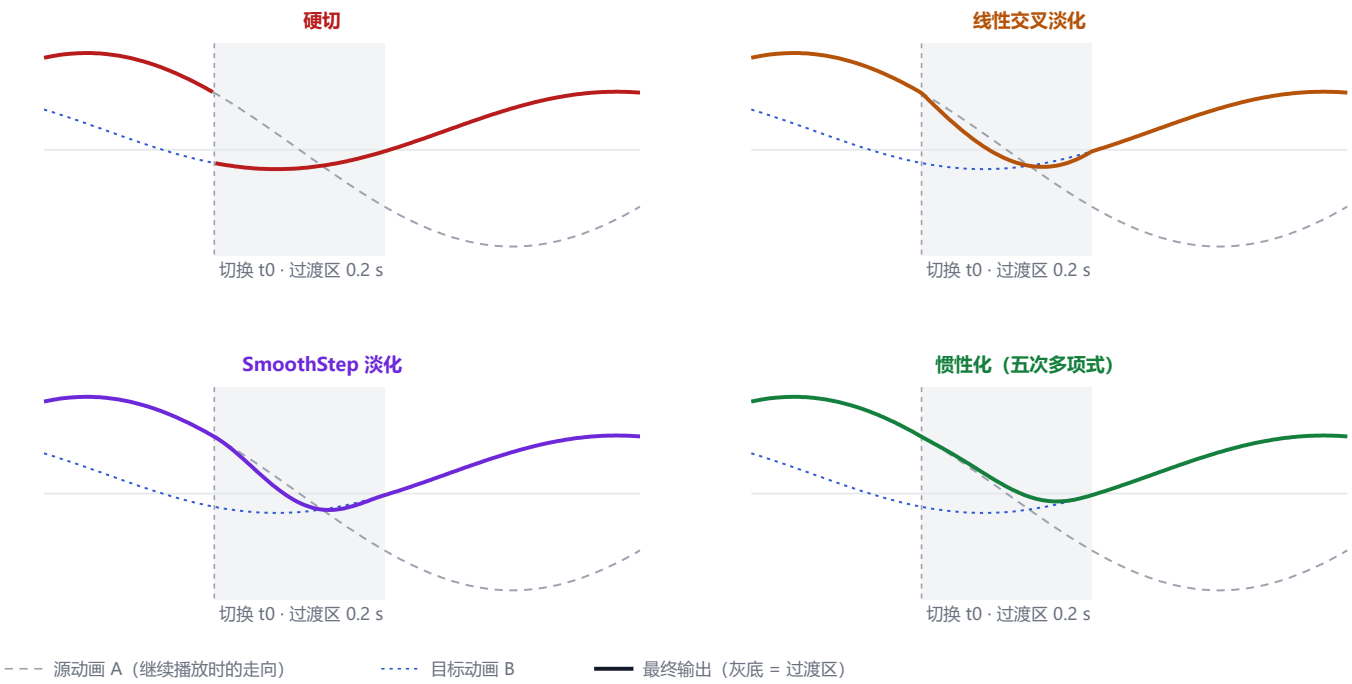


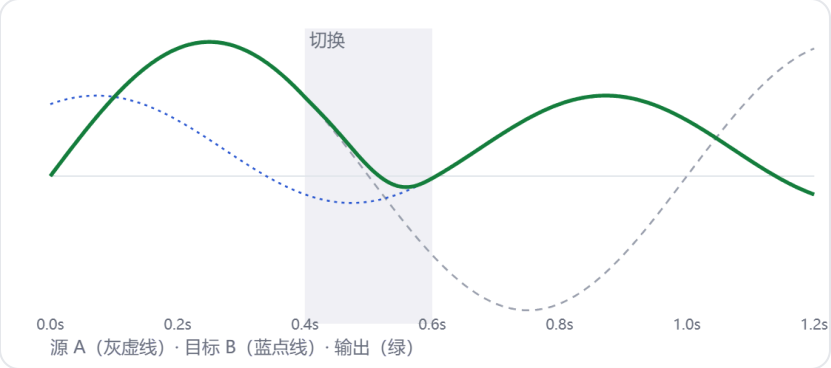
图 7-2 同一次切换的四种处理（构建时按 E7 的同一公式计算）。硬切有 38° 的跳变；线性淡化位置连续但速度有折角；惯性化从源动画的运动趋势自然地“拐”向目标

实测 E7：单根骨骼角度信号，0.40 s 切换，过渡 0.2 s，120 Hz 采样

方法	最大单帧跳变	最大速度突变	过渡期内姿势求值次数（60 fps）
硬切	**38.30°**	**4515.0 °/s**	12
线性交叉淡化	3.60°	**183.3 °/s**	24
SmoothStep 淡化	3.33°	51.7 °/s	24
惯性化	**2.29°**	**39.5 °/s**	**12**

参考：源动画自身每帧最大变化 2.08°。切换瞬间位置差 36.2°、速度差 −160.1 °/s——差值本来就在朝目标收缩，惯性化利用了这个速度，所以过渡几乎和动画自身的运动一样平顺。

交互实验 • 状态过渡：硬切 / 交叉淡化 / 惯性化



最大单帧跳变 2.29°
最大速度突变 39.5°/s
过渡期求值 12 次（60fps）

切换瞬间：
位置差 x0 = 36.2°
速度差 v0 = −160.1°/s

移动切换时刻：源动画正在快速运动时切换（例如 0.40 s），惯性化的优势最明显；在源动画速度为 0 附近切换，几种方法差别不大。

在 UE 里怎么用

1. 在 AnimGraph 里、过渡发生位置的**下游**放一个 **Inertialization** 节点（通常靠近 Output Pose）。
2. 把状态机过渡、Blend Poses by Enum 等节点的混合方式设为 Inertialization。它们不再自己混合，而是向下游的 Inertialization 节点发出请求。
3. 缺少这个节点时会在消息日志里报运行时错误；多个请求同时到达时取最短的时长。

面试注意：惯性化的限制

- **源姿势停止求值**：源动画里过渡开始之后的 Notify 不会再触发，依赖这些 Notify 的逻辑要改。
- **适合短过渡**（0.1~0.3 s），不适合需要长时间“两个动作同时存在”的混合。
- **频繁打断会劣化**：每次打断都基于当前（已带偏移的）姿势重新计算差值。
- **Dead Blending**：UE 5.3 新增的实验性节点，用源动画的速度向前外推再与目标交叉淡化，姿势差异很大时效果通常更好，但官方不建议在发行项目中依赖它。

7.4 蒙太奇：代码驱动的一次性动画

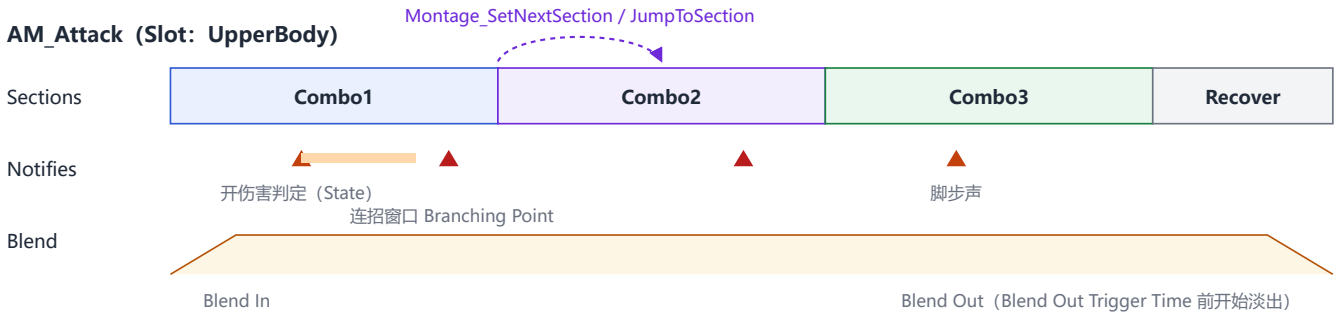


图 7-3 一个连招蒙太奇：分段（Section）可以按默认顺序串接，也可以由代码在连招窗口内跳转；Notify 驱动伤害判定和音效；首尾有淡入淡出

- **Slot**：蒙太奇不进状态机，而是播放到 AnimGraph 里的 **Slot 节点**上。Slot 名形如 `DefaultGroup.UpperBody`。**同一 Slot Group 同时只能播放一个蒙太奇**，新播放的会把旧的淡出。
- **Section**：把一条蒙太奇切成若干段，可设置每段结束后跳到哪段（可循环），用于连招、读条循环、开始-循环-结束三段式技能。
- **常用 API**：`Montage_Play` / `PlayAnimMontage`、`Montage_JumpToSection`、`Montage_SetNextSection`、`Montage_Stop`、`Montage_SetPlayRate`；委托 `OnMontageBlendingOut`、`OnMontageEnded`（注意被打断时也会回调，参数里有 `bInterrupted`）。GAS 里对应 `PlayMontageAndWait` 这个 Ability Task。

Branching Point 与普通 Notify

普通 Notify 排队到游戏线程统一派发，可能晚一帧；**Branching Point** 在推进蒙太奇时间时立即同步触发，触发点精确到这一帧扫过的位置，适合“在第 0.45 秒检查是否按了下一次攻击键，按了就跳 Section”这种对时机敏感的逻辑。代价是它会打断时间推进、在游戏线程执行，所以只在需要时用。

7.5 Root Motion：谁来移动角色

模式（Root Motion Mode）	行为	适用
No Root Motion Extraction	root 骨骼的位移留在姿势里，网格会“跑出”胶囊体	很少用
Ignore Root Motion	提取出来但丢掉，root 被锁在原地	只要原地动画
Root Motion from Montages Only（默认）	只有蒙太奇的位移交给移动组件	网络游戏的标准做法
Root Motion from Everything	所有动画（含混合空间、状态机）的位移按权重混合后交给移动	单机、动作精确的移动；网络预测难做

In-Place + 代码移动

移动由 CharacterMovement 决定，动画只负责“看起来像在走”。**响应快、好做网络预测**，但容易滑步，需要混合空间速度匹配、步幅调整来补救。

Root Motion

动画决定位移，**脚和地面严丝合缝**，适合攻击突进、翻滚、攀爬、处决。缺点是操控不够跟手，网络同步要额外处理。

深入：Root Motion 在网络中怎么同步

CharacterMovementComponent 对**蒙太奇的 Root Motion** 做了预测支持：客户端本地播放蒙太奇并用提取出的位移移动，把移动当作普通的预测移动发给服务器；服务器也播放同一个蒙太奇，用相同位移校验；模拟代理通过复制的蒙太奇状态（当前蒙太奇、播放位置、速率等）对齐。之所以“只支持蒙太奇”，是因为蒙太奇的播放是确定的、可以按位置重放；而状态机 / 混合空间的 Root Motion 依赖每帧的权重和输入，服务器和客户端很难算得完全一样。程序化的位移（击退、冲刺）可以用 **Root Motion Source**，它同样被移动组件预测和同步。

本章速记

- 状态机 = 状态 + 过渡规则（可自动按剩余时间）+ Conduit + State Alias + 优先级；规则里只读预先算好的变量。
- 交叉淡化过渡期**两边都求值**，打断会叠成多层混合。
- **惯性化**：只求值目标，加一个按五次多项式衰减的差值；E7：跳变 2.29°、速度突变 39.5°/s、求值次数减半。需要下游 Inertialization 节点；源 Notify 不再触发。
- 蒙太奇播到 **Slot**，同组互斥；Section 做连招；Branching Point 精确触发。
- Root Motion 默认只取蒙太奇的，因为蒙太奇可确定性重放，便于网络预测；程序化位移用 Root Motion Source。

▶ **自测：同时在 UpperBody 和 FullBody 两个 Slot 上播放蒙太奇，会互相打断吗?**

▶ **自测：攻击蒙太奇带 Root Motion，角色撞墙后为什么还在“原地踏步往前冲”？**

CHAPTER 08

IK 与程序化动画：让动画适应世界

动画是按平地、按标准体型做的。真实场景里有台阶、斜坡、不同长度的武器、要去够的门把手——这些靠 IK 和各种“Warping”在运行时修正。

本章目标

- 推导两骨骼 IK 的解析解，理解 Pole（Joint Target）的作用
- 知道 FABRIK / CCD 这类迭代解法的收敛特性和代价
- 讲清脚步 IK 的完整流程，以及射线检测该放在哪个线程
- 了解 UE5 的 Control Rig、IK Rig、距离匹配、步幅 / 朝向调整、Motion Warping 和 Motion Matching

8.1 FK 与 IK

FK（正向运动学）：给每节骨骼的局部旋转，算出末端在哪——动画序列就是 FK 数据。IK（逆向运动学）：给末端的目标位置，反求每节骨骼该怎么转。游戏里常见的 IK：

脚步 IK

台阶、斜坡上脚贴地，骨盆随之下沉。

手部 IK

左手始终握住枪的护木（不同武器握点不同），双手搬箱子。

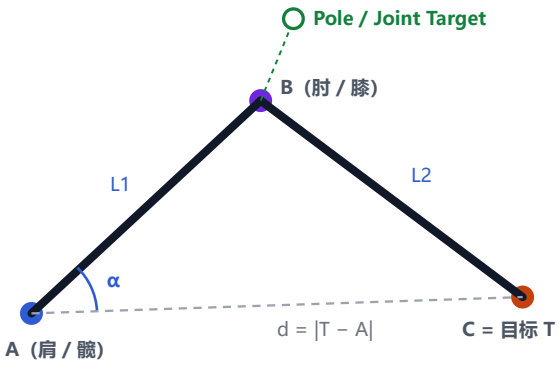
Look At / Aim

头和眼睛看向目标；枪口精确指向准星。

交互

伸手去按按钮、开门、攀爬时手扶边缘。

8.2 两骨骼 IK：解析解



三步求解

1. d = clamp(|T−A|, |L1−L2|, L1+L2)　（够不着就伸直）
2. 余弦定理：cos α = (L1² + d² − L2²) / (2·L1·d)
3. 弯曲平面由 A、T 和 Pole 确定：
   dir = (T−A)/|T−A|，
   bend = Pole 在垂直于 dir 方向上的分量（归一化）
   B = A + dir·L1·cos α + bend·L1·sin α

最后把上臂旋转到 A→B，前臂旋转到 B→T

没有 Pole 时，肘可以绕 A→T 轴转到任意位置，结果不唯一、会抖

图 8-1　两骨骼 IK 的几何：三角形三边已知（L1、L2、d），肘部只剩“绕 A→T 轴转多少”这一个自由度，由 Pole 决定

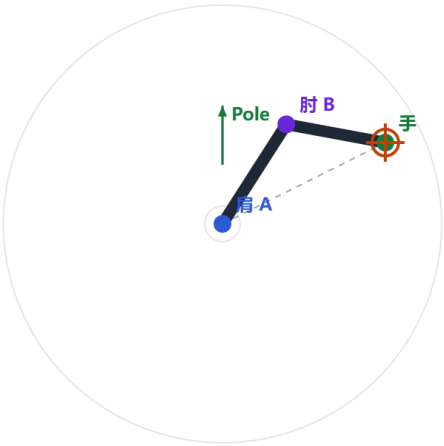
两骨骼 IK（demo 中的实现，与 UE AnimationCore::SolveTwoBoneIK 思路相同）

```
void TwoBoneIK(FVector A, float L1, float L2, FVector Target, FVector Pole, FVector& Elbow, FV
{
    const FVector ToT  = Target - A;
    const float   Dist = ToT.Size();
    const FVector Dir  = ToT / Dist;
    const float   D    = FMath::Clamp(Dist, FMath::Abs(L1 - L2) + 1e-3f, L1 + L2 - 1e-4f);
    const float   CosA = (L1 * L1 + D * D - L2 * L2) / (2.f * L1 * D);
    const float   SinA = FMath::Sqrt(FMath::Max(0.f, 1.f - CosA * CosA));
    const FVector Bend = (Pole - Dir * FVector::DotProduct(Pole, Dir)).GetSafeNormal();    // P
    Elbow = A + Dir * (L1 * CosA) + Bend * (L1 * SinA);
    End   = A + Dir * D;                                        // 够不着时停在伸直的位置
}
```

实测 E8：上臂 30、前臂 28，2 万个随机目标

- 可达 16330 个：末端误差最大 **0.0000**；不可达 3670 个：手臂伸直指向目标；骨长误差 0；肘部全部落在 Pole 一侧。
- 耗时 **13.9 ns/次**——解析解几乎是免费的，所以手脚 IK 全都用它。

交互实验 • 两骨骼 IK




```
|T-A|    123.0
可达范围 [12, 148]
可达
肩角 α   30.8°
肘角     112.1°
末端误差 0.00
拉伸比例 1.00
```

按住并拖动目标（橙色）。灰色圆环是可达范围；把目标拉到最远处附近，再切换 Pole 方向，观察肘部翻转。

UE 的 **Two Bone IK** 节点参数：IK Bone（末端骨骼，例如 hand_l / foot_r，向上两级自动确定链）、Effector Location 与其空间（组件、世界、某骨骼空间）、Joint Target Location（即 Pole）、Allow Stretching（够不着时允许拉长，可设起始比例和最大比例）、Take Rotation from Effector Space / Maintain Effector Relative Rotation（末端旋转怎么处理）。

面试注意：两骨骼 IK 的三个坑

- **手臂伸直时 Pole 失效、肘部翻转**：A、B、T 共线时弯曲平面不确定，Pole 的微小变化会让肘部翻到另一边。办法：让目标略在最大长度之内（clamp 时留余量），Pole 放在合理方向并随身体旋转。
- **空间搞错**：Effector 用世界坐标却按组件空间解释，角色一转身手就飞了。用骨骼空间（如“相对右手”）给左手 IK 目标最稳。
- **IK 要在混合之后、最后阶段做**：IK 是对最终姿势的修正，放在混合之前会被后续混合冲掉。

8.3 多骨骼：FABRIK、CCD 与全身 IK

链条超过两节时，没有唯一的解析解，用迭代：

- **FABRIK**（Forward And Backward Reaching IK）：先把末端放到目标、从末端往根逐节拉回骨长（Backward）；再把根放回原位、从根往末端逐节拉回（Forward）；重复到误差小于容差。只做点和长度运算，不算角度，很快。
- **CCD**（Cyclic Coordinate Descent）：从末端前一节开始，每节旋转使“这节到末端”的方向对准“这节到目标”，循环往复。实现简单，但容易让靠近末端的关节转得过多。

- **全身 IK（Full Body IK）**：同时考虑多个末端（双手双脚）和关节限制，UE5 在 Control Rig / IK Rig 里提供 Full Body IK 解算器。

实测 E8：FABRIK，5 节链（每节 10），2 万个可达目标，最多 50 次迭代

容差	平均迭代	最多迭代	50 次未收敛	耗时
1.00	1.4	13	0	0.21 us
0.10	2.7	50	3	0.35 us
0.01	4.1	50	25	0.49 us

平均很快，但**长尾明显**：初始链条接近共线、目标靠近根部等情况收敛很慢。实际使用一定要设最大迭代次数和合理容差，并接受“差一点”的结果。

8.4 脚步 IK 的完整流程

1. **射线检测（游戏线程）**：在角色或 EventGraph 里，从每只脚的骨骼位置向下做 Line / Sphere Trace，得到地面高度和法线。物理查询不能在动画工作线程里随意做。
2. **算偏移**：每只脚的高度差 = 地面高度 − 动画里脚的高度；**骨盆下沉量 = 两脚中下沉最多的那个**（否则低的那只脚够不着地）。
3. **平滑**：偏移和脚的旋转都做插值（例如 15~20/s 的指数插值），避免走台阶时一帧一跳。
4. **AnimGraph**：转到组件空间 → 调整 pelvis 高度 → 两只脚各一个 Two Bone IK（目标 = 原位置 + 偏移，Pole 在膝盖前方）→ 按地面法线旋转脚掌 → 转回局部空间。
5. **开关**：空中、播放特定蒙太奇时关闭（用曲线或变量控制 Alpha）；远处角色在 LOD 中跳过。

游戏客户端实战：为什么用“IK 骨骼”（ik_foot_l 等）

Mannequin 里有一组不参与蒙皮的骨骼：`ik_foot_root / ik_foot_l / ik_foot_r / ik_hand_root / ik_hand_gun / ik_hand_l / ik_hand_r`。动画导出时把脚、手的位置烘焙到这些骨骼上。运行时 IK 目标直接取这些骨骼的位置，即使上游混合改变了腿的姿势，也能知道“这条动画原本想让脚放在哪”；左手 IK 也可以取 ik_hand_l 相对 ik_hand_gun 的位置，自动适配不同武器。

8.5 UE5 的程序化动画工具

工具	做什么	要点
Control Rig	节点化的绑定与程序化动画系统	可在 AnimGraph 里作为节点运行（Forward Solve），做 IK、程序化修正；也用于 Sequencer 里手 K 动画
IK Rig / IK Retargeter	定义 IK 目标与解算器栈；不同骨架之间按“骨骼链”重定向	取代 UE4 只能同骨架重定向的限制；支持运行时重定向
距离匹配（Distance Matching）	起步、急停、转身动画按“离停止点还有多远”选择播放位置，而不是按时间	动画里带距离曲线；角色按代码物理减速，动画刚好在停下时播到最后一帧，不滑步
步幅调整（Stride Warping）	按实际速度 / 动画速度的比例拉长或缩短腿的步幅	少量动画覆盖连续的速度范围
朝向调整（Orientation Warping）	把下半身扭向实际移动方向，上半身保持朝前	只需 4 个基本方向动画就能覆盖任意方向
Motion Warping	在蒙太奇的指定时间窗内，把 Root Motion 的位移 / 旋转调整到某个目标点	翻越、处决、攻击吸附：动画落点精确对准目标
Motion Matching（Pose Search）	不用状态机，每帧在动画数据库里搜索“与当前姿势和期望轨迹最匹配”的那一帧接着播	UE 5.4 起随 Game Animation Sample 推广；需要大量动作捕捉数据、搜索有 CPU 开销
物理动画	布娃娃、受击物理、次级运动	Physics Asset；AnimGraph 的 RigidBody 节点（辫子、披风）、AnimDynamics；Physical Animation Component 用马达驱动刚体跟随动画

深入：Motion Matching 的代价函数

每一帧（或每隔几帧）计算一个查询向量：当前若干骨骼的位置和速度（如双脚、骨盆）+ 未来若干时刻的期望轨迹（位置、朝向，来自玩家输入和移动预测）。数据库里每一帧都预先算好同样格式的特征向量，搜索**加权距离最小**的那一帧，如果它和当前播放位置足够不同就跳过去，并用惯性化平滑切换。工程难点在于特征的归一化和权重、搜索加速（KD 树 / 聚类）、数据覆盖率，以及与玩法需要的“确定性响应”之间的平衡。

本章速记

- 两骨骼 IK：**clamp 距离** → **余弦定理求角** → **Pole 定平面**；E8：误差 0、13.9 ns/次。伸直时 Pole 失效会翻肘。
- 多骨骼用 FABRIK / CCD / Full Body IK；FABRIK 平均几次迭代就收敛，但有长尾（E8：容差 0.01 时 50 次不收敛 25/20000）。
- 脚步 IK：游戏线程射线 → 骨盆下沉取最大值 → 平滑 → 组件空间两骨骼 IK + 脚掌对齐法线。
- UE5：Control Rig、IK Rig 重定向、距离匹配、步幅 / 朝向调整、Motion Warping、Motion Matching。

▶ **自测：为什么脚步 IK 时骨盆要往下沉，而不是只把一只脚往上抬?**

▶ **自测：Motion Warping 和 Root Motion Source 有什么区别?**

PART FOUR

第四部分　性能与实战

一个角色的动画只要几微秒，几百个角色就是几毫秒。动画优化的思路只有三条：算得快、少算、不算。

CHAPTER 09

动画性能优化：并行、降频、减骨与共享

先知道钱花在哪，再按“并行 → 降频 → 减骨 → 不可见不算 → 共享姿势”的顺序下手。

本章目标

- 知道动画开销分布在哪些线程、哪些环节，用什么工具看
- 理解 URO、Animation Budget Allocator、可见性 Tick 选项、LOD 减骨各自省了什么
- 区分 Leader Pose、Copy Pose From Mesh、Skeletal Mesh Merge、Animation Sharing 的适用场景
- 能给出一份大规模角色场景的动画优化清单

9.1 钱花在哪

阶段	线程	主要开销	优化方向
EventGraph / NativeUpdate	游戏线程	蓝图 VM、Cast、函数调用	挪到线程安全更新、Property Access、C++
AnimGraph Update	工作线程	节点遍历、引脚绑定、状态机规则	Fast Path、减少节点、权重为 0 的分支自动跳过
AnimGraph Evaluate	工作线程	**解压**、混合、组件空间转换、IK	减少同时参与混合的动画数、惯性化、LOD 跳过 IK、减骨
Complete	游戏线程	Notify 派发、Socket / 子组件更新、物理同步	减少 Notify、少挂子组件
渲染	渲染线程 / GPU	骨骼矩阵上传、GPU 蒙皮、Morph Target	LOD、合并网格减少 Draw Call、远处用顶点动画贴图

工具：**Unreal Insights**（打开 animation 相关 trace 通道看每个组件的 Update / Evaluate 耗时）、`stat anim`、**Rewind Debugger**（UE5，按帧回放动画蓝图的节点权重和状态机）、动画蓝图编辑器里的调试实例。

9.2 并行：把求值摊到所有核上

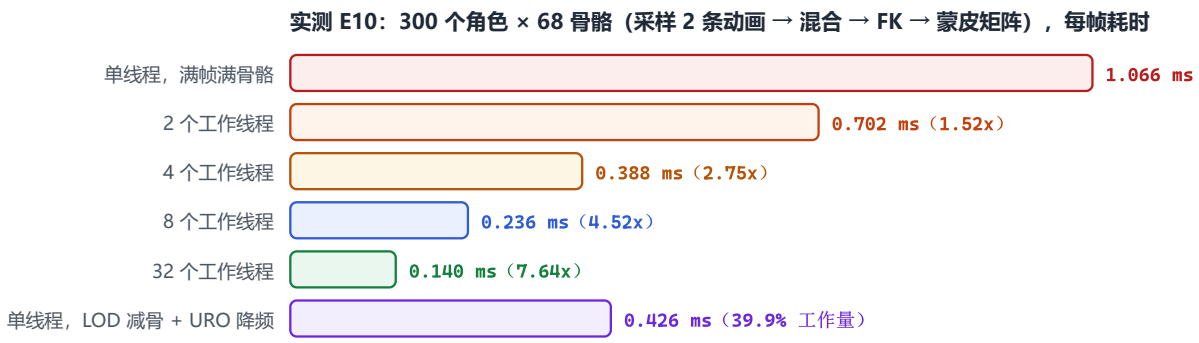


图 9-1 E10：每个角色约 3.55 us。并行有效但远不是线性（32 线程只有 7.64x：任务粒度、同步开销、内存带宽）；单线程下“少算”的收益介于 2 线程与 4 线程并行之间，而且是真正减少了 CPU 总消耗

UE 里每个 SkeletalMeshComponent 的求值是一个独立任务，天然可以并行。实测说明两点：

- **并行收益有上限**：每个任务只有几微秒时，分发与等待的开销占比上升；而且游戏线程最终要等所有任务完成（Complete 阶段），最慢的那个角色决定等待时间。
- **“少算”往往比“并行”更划算**：并行只是把同样的工作摊开，CPU 总消耗不变（移动端还有发热和功耗问题）；降频和减骨是真正减少了工作量。

9.3 URO：远处的角色少更新几次

Update Rate Optimizations（SkeletalMeshComponent 的 `bEnableUpdateRateOptimizations`）按角色在屏幕上的大小（或 LOD 级别）决定更新频率：近处每帧更新，远处每 2、3、4 帧更新一次。

- **跳过的帧怎么办**：可以开启跳帧插值（Interpolate Skipped Frames），在两次求值结果之间插值，画面仍然连续；不开的话远处角色动作会“一卡一卡”。插值本身有开销，所以有一个评估频率阈值，超过就不插值。
- **时间不会丢**：跳过的帧累积 DeltaTime，下次更新一次性推进，Notify 也不会丢（但可能集中在一帧触发）。

- **调试**：组件上的 `bDisplayDebugUpdateRateOptimizations` 会用颜色标出每个角色当前的更新频率。

> **实测 E10：LOD 减骨 + URO 降频**
>
> 300 个角色中，近处 20% 满帧满骨；中间 30% 隔 1 帧更新；远处 50% 每 4 帧更新一次且只算 23 根主干骨骼（去掉手指、扭转骨、IK 骨）。单线程 **0.426 ms/帧，是全量的 39.9%**，与理论工作量 39.2% 吻合——这类优化的收益可以事先算出来。

9.4 Animation Budget Allocator：给动画定预算

URO 是“按距离 / 屏幕大小”的固定规则；**Animation Budget Allocator** 插件是“按总预算”的动态调度：设定动画每帧最多花多少毫秒，系统按每个角色的重要性（Significance，通常由距离、是否在视野、是否是玩家相关决定）给它们分配更新频率，超预算时自动降低次要角色的频率并插值。需要用 `USkeletalMeshComponentBudgeted` 替换普通组件。适合角色数量波动很大的场景（大逃杀、大型战场）。

9.5 不可见就不算

VisibilityBasedAnimTickOption	不可见时	适用
AlwaysTickPoseAndRefreshBones	照常更新并刷新骨骼	玩家角色、需要精确 Socket / 碰撞的对象
AlwaysTickPose	推进动画时间（Notify、蒙太奇照常），但不刷新骨骼	不可见时仍需要动画事件的 AI
OnlyTickMontagesWhenNotRendered	只推进蒙太奇，其余不更新	依赖蒙太奇回调推进逻辑的角色
OnlyTickPoseWhenRendered	完全不更新	纯表现的 NPC、群众

面试注意：不可见不更新带来的逻辑 bug

- 技能逻辑依赖 Notify（例如“出手帧造成伤害”），角色在镜头外就**永远不出伤害**。关键玩法事件要么不依赖动画 Notify，要么对这类角色用 AlwaysTickPose。
- **专用服务器**上没有“渲染”，默认设置可能导致服务器上的骨骼从不刷新，命中检测用的是参考姿势。服务器端需要按骨骼做命中判定的角色要显式设置更新选项。
- Root Motion 角色在服务器上必须推进蒙太奇，否则移动和客户端对不上。

9.6 LOD：减骨与跳过节点

- **网格 LOD 减骨**：低 LOD 去掉手指、扭转骨、面部骨骼，蒙皮权重转移到父骨骼。求值时 Required Bones 变少，所有逐骨骼的工作（解压、混合、FK、蒙皮矩阵）同比减少（第 1 章的拓扑序保证这是一个简单的过滤）。
- **节点 LOD Threshold**：AnimGraph 里很多节点（Two Bone IK、Apply Additive、Control Rig 等）有 LOD Threshold，超过该 LOD 就直接透传，不做计算。远处角色不需要脚贴地、不需要呼吸叠加。
- **动画曲线 / Notify 的 LOD 过滤**：低 LOD 不计算某些曲线、不触发某些 Notify（例如脚步声）。

9.7 共享姿势：一个求值，多处使用

方案	做法	适用	注意
Leader Pose Component（5.1 前叫 Master Pose）	子 Mesh 直接用主 Mesh 的骨骼变换，自己不求值	换装部件（头、身体、衣服）	子 Mesh 不能有自己的动画逻辑；骨骼必须是主骨架的子集
Copy Pose From Mesh	子 Mesh 跑自己的动画蓝图，第一步把主 Mesh 的姿势按骨骼名拷过来，之后还能叠加物理、IK	需要次级动画（布料骨骼、物理头发）的部件	比 Leader Pose 贵，但比完整求值便宜
Skeletal Mesh Merge	运行时把多个部件合并成一个 SkeletalMesh	换装组合固定后减少 Draw Call 和组件数	合并有一次性开销；材质要能合并
Animation Sharing 插件	大量 NPC 按“状态”共享少数几个求值实例的姿势	观众、群众、远处士兵	同状态的角色动作完全同步，需要加随机偏移
顶点动画贴图（VAT）	把动画烘焙成贴图，GPU 实例化绘制，CPU 不跑动画	成百上千的远景角色	不能交互、不能混合（或只能简单切换），UE5 有 AnimToTexture 插件

9.8 一份优化清单

类别	检查项
动画蓝图	开启多线程更新；逻辑放线程安全函数 + Property Access；引脚走 Fast Path（编译无性能警告）；不在 Update 里 Cast
图结构	Cached Pose 复用子图；过渡用惯性化；减少同时参与混合的动画数；远处 LOD 跳过 IK / 加性 / Control Rig
更新频率	开启 URO 并配置跳帧插值；大量角色用 Budget Allocator；不可见时按需选择 Tick 选项
数据	合理的压缩设置（ACL）；LOD 减骨；曲线数量受控；Notify 不滥用
组件	换装用 Leader Pose；必要时合并网格；少挂需要每帧更新 Transform 的子组件
群体	Animation Sharing；远景 VAT + 实例化

本章速记

- 开销：游戏线程（EventGraph）+ 工作线程（Update/Evaluate，解压最贵）+ 渲染（蒙皮）。看 Insights、Rewind Debugger。
- 并行有上限（E10：32 线程 7.64x）；**少算**更划算（LOD+URO：39.9%，与理论吻合）。
- URO 按屏幕大小降频 + 跳帧插值；Budget Allocator 按总预算动态调度。
- VisibilityBasedAnimTickOption 四档；小心镜头外 Notify 不触发、服务器骨骼不刷新。
- 共享：**Leader Pose**（不求值）< Copy Pose（部分求值）；Merge 减 Draw Call；Animation Sharing / VAT 做群体。

▶ **自测：URO 降到每 4 帧更新一次，角色的 Root Motion 会不会变慢?**

▶ **自测：给换装部件用 Leader Pose 后，部件上的物理骨骼（披风）不动了，为什么?**

CHAPTER 10

网络同步、排错清单与面试总复习

最后一章：多人游戏里动画怎么同步，线上常见动画 bug 怎么定位；然后是全书实测总表、40 道面试追问和速记卡。

10.1 多人游戏中的动画同步

原则：动画本身不复制，复制的是“驱动动画的状态”。每个客户端用复制来的状态在本地跑同一个动画蓝图。

要同步的东西	怎么同步
移动相关（速度、是否在空中、蹲伏）	CharacterMovementComponent 的移动复制本身就带来了速度和位置；蹲伏等标志位是角色的复制属性。动画蓝图在各端从角色读取
瞄准方向	Pawn 的 `GetBaseAimRotation()`（远端用复制的 RemoteViewPitch 等），各端算 Aim Offset
一次性动作（攻击、技能、受击）	服务器决定，**蒙太奇**通过角色或 GAS 的复制结构（当前蒙太奇、播放位置、速率、Section）同步给模拟代理；纯表现的可用多播 RPC
Root Motion 蒙太奇	CharacterMovement 对蒙太奇 Root Motion 做预测与校正（第 7 章）
命中判定用的姿势	服务器需要真实骨骼位置时，要保证服务器端动画更新（第 9 章），并做延迟补偿（回溯到客户端开枪时刻的姿势 / 碰撞体）

面试注意：本地预测的蒙太奇

玩家按下攻击键，如果等服务器确认再播放，会有一个 RTT 的延迟，手感很差。常见做法（GAS 的做法）：**客户端立即本地播放**（预测），同时请求服务器；服务器确认后复制给其他客户端，本地客户端收到的复制会被识别为“已在播放”而不重复播；服务器拒绝时本地回滚（停止蒙太奇）。所以蒙太奇的“预测键”和“谁是权威”是网络动画的核心问题。

10.2 线上动画 bug 排错清单

现象	常见原因	怎么查 / 怎么改
角色 T-Pose / A-Pose	动画蓝图没设置或编译失败；Skeleton 不匹配；某 LOD 下 AnimGraph 输出无效；不可见不更新后刚进入视野	看 Message Log、动画蓝图调试实例；检查 AnimClass 与 Skeleton；检查 VisibilityBasedAnimTickOption
滑步	混合空间采样位置与动画真实速度不符；同步组缺失；移动速度与动画速度不匹配	Blend Space Analysis；加同步组 / 同步标记；步幅调整、距离匹配
远处角色动作卡顿	URO 降频且未开跳帧插值；Budget Allocator 限流	开 URO 调试显示；调整阈值或开启插值
手指 / 武器微抖	压缩误差在层级末端放大	调整压缩误差或虚拟顶点距离；对武器骨骼单独设置
脚步声响两次	混合中的两条动画都触发 Notify	Trigger Weight Threshold；混合空间只触发最高权重
蒙太奇不播放	AnimGraph 里没有对应 Slot 节点 / Slot 名不一致 / Slot 之后被别的节点覆盖；同组蒙太奇被打断	检查 Slot 名；在 Rewind Debugger 里看蒙太奇权重
Root Motion 不生效	Root Motion Mode 不对；动画未勾选 Enable Root Motion；分层混合把根骨骼排除	改模式；开启 Blend Root Motion Based on Root Bone
上半身动作朝向偏	分层混合用 Local Space	开 Mesh Space Rotation Blend（第 5 章）
切 LOD / 切武器时姿势跳一下	骨骼集合变化导致节点重置；子图重新相关、时间归零	FBoneReference 在 CacheBones 中重新解析；惯性化过渡；同步组保持相位
IK 时肘 / 膝翻转	目标接近最大长度，Pole 失效	clamp 留余量，调整 Pole（第 8 章）

10.3 全书实测总表

实验	结论（数字来自 demos/run_output.txt）
E1 FK 三种写法	68 骨骼：线性数组 1.12 us/角色；指针树 3.54x；逐根回溯 5.65x
E2 旋转混合	170° 与 −170°：欧拉插值 0°、nlerp/slerp 180°；nlerp 偏差 60° 时 0.27°、179° 时 8.00°；slerp 慢 4.86x；加权求和与顺序无关，链式 slerp 相差 1.7°
E3 蒙皮	LBS 扭转半径 = cos(θ/2)，180° 时为 0；DQS 恒为 1；CPU 上 DQS 慢 4.33x
E4 分层混合	Blend Depth 权重 (k+1)/N；骨盆扭 40° 时 Local 混合头偏 40°、Mesh Space 为 0°；Mesh Space 慢 13.2x
E5 混合空间	9 采样 12 个三角形；权重和为 1 且重建坐标 = 输入；输入平滑途经 Fwd（权重 1.00），Weight Speed 不经过（0.00）
E6 同步	支撑脚不一致：不同步 50.4%，归一化同步 21.2%，同步标记 0%
E7 过渡	最大跳变：硬切 38.3°、线性 3.6°、惯性化 2.29°；速度突变 4515 / 183 / 39.5 °/s；惯性化求值次数减半
E8 IK	两骨骼解析解误差 0、13.9 ns；FABRIK 容差 0.01 平均 4.1 次迭代，有长尾
E9 压缩	去常量 3.37x → 量化 8.60x → 精简最高 12.24x；局部 0.5° 误差在组件空间放大到 1.64°
E11 四元数基础	Z 轴 90° → (0.7071, 0, 0, 0.7071)；$q \cdot v \cdot q^{-1}$ 与罗德里格斯相差 < 7.4e-5；60° 与 420° 的四元数互为相反数；X、Y 两个 90° 换顺序差 120°；组合旋转四元数 0.78 ns vs 矩阵 7.64 ns，转向量矩阵 0.50 ns vs 四元数 1.64 ns
E12 欧拉角问题	pitch 90° 时 (30,90,0) ≡ (0,90,−30) ≡ (60,90,30)；越过头顶 0.02° 让 yaw 跳 180°；矩阵连乘 100 万次压扁 1.18%，未归一化四元数放大 5.3%
E10 整体开销	300 角色 1.066 ms/帧单线程；32 线程 7.64x；LOD + URO 只需 39.9% 工作量

10.4 面试追问 40 题

骨骼与数据（1~8）

▶ 1. Skeleton 和 SkeletalMesh 的区别？为什么动画绑定在 Skeleton 上？

▶ 2. 骨骼为什么按拓扑序存放？有什么好处？

▶ 3. 局部空间、组件空间、世界空间分别用在哪？怎么互转？

▶ 4. Compact Pose Bone Index 是什么？为什么自定义节点不能直接用骨骼下标？

▶ 5. 蒙皮矩阵怎么算？GPU 蒙皮做了什么？

▶ 6. LBS 的糖纸扭曲怎么来的？UE 怎么解决？

▶ 7. 动画压缩有哪些手段？ACL 的特点？

▶ 8. 为什么局部空间的误差阈值不够？

混合（9~18）

▶ 9. 为什么旋转混合不能用欧拉角？

▶ 10. nlerp 和 slerp 区别？UE 用哪个？

▶ 11. 多路混合为什么要“加权求和再归一化”？

▶ 12. 加性动画是什么？局部空间和网格空间加性的区别？

▶ 13. 2D 混合空间的权重怎么算？

▶ 14. Smoothing Time 和 Weight Speed 有什么区别?

▶ 15. 为什么要同步组? Leader 怎么选?

▶ 16. 同步标记解决了归一化同步解决不了的什么问题?

▶ 17. Layered Blend per Bone 的 Blend Depth 怎么算权重?

▶ 18. Mesh Space Rotation Blend 解决什么问题? 代价?

动画蓝图与线程（19~26）

▶ 19. EventGraph 和 AnimGraph 分别在哪个线程?

▶ 20. FAnimInstanceProxy 的作用?

▶ 21. 什么是 Fast Path? 怎么判断节点走了 Fast Path?

▶ 22. Property Access 和线程安全函数是干什么的?

▶ 23. 说一下 SkeletalMeshComponent 一帧的动画流程。

▶ 24. 自定义 AnimNode 要实现哪些函数?

▶ 25. 权重为 0 的分支会不会计算? 有什么副作用?

▶ 26. Linked Anim Layer 有什么用?

状态机、过渡、蒙太奇（27~33）

▶ 27. 状态机过渡期间的开销？被打断时会怎样？

▶ 28. 惯性化的原理和优点？

▶ 29. 使用惯性化要注意什么？

▶ 30. 蒙太奇和状态机怎么配合？Slot 是什么？

▶ 31. 做连招用蒙太奇的什么功能？

▶ 32. Root Motion 的几种模式？为什么网络游戏只用蒙太奇的？

▶ 33. In-Place 动画和 Root Motion 各自优缺点？

IK、新特性、性能（34~40）

▶ 34. 两骨骼 IK 怎么解？Pole 的作用？

▶ 35. 脚步 IK 的流程？射线放哪？

▶ 36. FABRIK 和 CCD 的区别？

▶ 37. 距离匹配、步幅调整、朝向调整分别解决什么？

▶ 38. URO 是什么？会带来什么问题？

▶ 39. Leader Pose 和 Copy Pose From Mesh 怎么选？

▶ 40. 场景里有 300 个 AI 角色，动画开销太高，你会怎么优化？

10.5 速记卡

骨骼与数据

- Skeleton 是命名约定，动画绑它；骨骼 = 拓扑序数组；三套下标，自定义节点用 Compact Pose Index。
- comp[i] = local[i] × comp[parent]；蒙皮矩阵 = 参考姿势逆 × 当前组件空间。
- LBS 扭转半径 cos(θ/2) → 扭转骨 + 修正形变；压缩：去常量 → 量化 → 精简 / 可变位率（ACL，5.3 默认），误差按组件空间衡量。

混合

- 旋转：四元数 + 最短路径 + 加权求和归一化（nlerp）；欧拉插值会走反方向。
- 加性 = 动画 − 基准；Aim Offset = 网格空间加性的混合空间。
- 混合空间：Delaunay + 重心坐标；Smoothing Time ≠ Weight Speed；同步组（Leader = 最高权重）+ 同步标记（按脚步对齐）。
- 分层混合：Blend Depth 0 / N / −1；Mesh Space Rotation Blend 防止上半身被骨盆带偏。

动画蓝图

- EventGraph 游戏线程，AnimGraph 工作线程（Proxy 隔离）；Property Access + Thread Safe Update；Fast Path。
- 节点：Initialize / CacheBones / Update（自顶向下）/ Evaluate（自底向上）。
- 过渡：交叉淡化两边都算；惯性化只算目标 + 五次多项式衰减；蒙太奇播到 Slot，Section 做连招；Root Motion 默认只取蒙太奇。
- IK：两骨骼解析解 + Pole；脚步 IK 骨盆下沉取最大；UE5 距离匹配 / 步幅 / 朝向调整 / Motion Warping / Motion Matching。

性能

- 并行有上限，少算更划算：URO、Budget Allocator、LOD 减骨与节点 LOD Threshold、不可见不更新、Leader Pose、Animation Sharing、VAT。
- 警惕：镜头外 Notify 不触发、服务器骨骼不刷新、降频后 Root Motion 顿挫。

APPENDIX · A

对照 Notion 原笔记：哪些可以直接用，哪些要改说法

原笔记（《UE 动画系统》Notion 导出）的框架和大部分结论是对的，但有一些说法过于绝对，或者数字和机制不准确。面试官顺着这些点追问时最容易翻车，这里逐条给出更稳妥的说法和依据。

A.1 原笔记里讲得好、可以直接用的地方

- **“逻辑与姿态解耦”的总框架**：EventGraph 收集数据、AnimGraph 求姿势；状态机管宏观状态、混合空间管连续参数、分层混合管局部覆盖、蒙太奇管代码触发的一次性动作——这条主线完全正确，可以作为回答的骨架。
- **Skeleton 是“数据协议 / 字典”**，不含顶点和蒙皮；`FReferenceSkeleton` 用扁平数组 + 父下标存骨骼（第 1 章 E1 的实测佐证了这种布局的好处）。
- **位移重定向的三种模式**：Root 用 Animation、躯干四肢用 Skeleton、Pelvis 用 AnimationScaled。
- **蒙皮的 CPU / GPU 分工**、逆绑定矩阵、Physics Asset 做命中检测而不是读回 GPU 顶点、VAT 做群体——都准确。
- **Mesh Space Rotation Blend 的作用**、蒙太奇 Section 做连招、Branching Point 比普通 Notify 更精确、状态机过渡期双份采样、惯性化只算目标——都准确（第 5、7 章有实测）。
- **1D 混合空间 + 朝向调整（Orientation Warping）**是 Lyra / Game Animation Sample 的思路，这个趋势判断是对的。

A.2 需要修正或补充的说法

#	原笔记说法	问题	更准确的说法	依据
1	Blend Depth = 3 时，spine_01 / 02 / 03 分别得到 25% / 50% / 75%，neck 才到 100%	**数字对应的是 Depth = 4**	Depth = N 时第 k 层（过滤骨骼自己 k = 0）权重是 min(1, (k+1)/N)。Depth = 3：33% / 67% / 100%；Depth = 4 才是 25% / 50% / 75% / 100%。负数表示排除这个分支	E4、图 5-1
2	混合空间里权重为 0 的动画“依然可能被采样”，所以不能把跳跃、攻击放进去	**机制说错了**	只有输入点所在三角形里权重大于 0 的（最多 3 个）采样会被求值；但空间里所有动画资源都会被引用和加载。不把跳跃、攻击放进去的真正原因是：它们是离散事件，需要打断、触发通知、有开始和结束，不是“随参数连续变化”的东西	E5、第 4 章
3	1D 混合空间“能直接砍掉近 30% 的采样开销”	**数字没有依据**	1D 最多 2 个、2D 最多 3 个采样同时参与。省多少取决于解压和混合占总开销的比例，要用 Insights 实测，不要背具体百分比	第 4、9 章
4	SLERP “保证以最短路径、绝对匀速地旋转”	**漏了前提**	slerp 本身不保证走短路：q 和 −q 是同一个旋转，要先检查点积，小于 0 时取反。UE 的 `FQuat::Slerp` 内部做了这一步，手写时容易漏	B3.6、E2
5	动画序列在两帧之间“通过 Slerp 实时插值”	**不够准确**	关键帧插值和姿势混合通常用 nlerp（线性插值再归一化，UE 里如 `FQuat::FastLerp` + 归一化）：相邻关键帧夹角很小，nlerp 误差可忽略，且快约 4.9 倍	E2、第 3 章
6	四元数“连乘比矩阵乘法更快”，所以全面优于矩阵	**只对了一半**	组合两个旋转四元数更快（0.78 vs 7.64 ns）；但旋转大量向量时矩阵更快（0.50 vs 1.64 ns），所以 GPU 蒙皮用矩阵。各有分工	E11(h)
7	四元数“只需要简单地归一化就能变回合法旋转”，暗示不会形变	**要补一句**	四元数一样会漂移：不归一化连乘 100 万次后 |q| = 1.026，变换后的向量被放大 5.3%（比矩阵的 1.18% 还明显）。它的优势是修复只需一次归一化	E12(d)

8	AnimGraph 里“一旦使用 NOT 等运算节点就跌入蓝图虚拟机”	**与官方文档不符**	官方列出的 Fast Path 包括：成员变量、**取反的布尔成员变量**、嵌套结构体成员（Break Struct，Break Transform 等有转换的除外）。乘法、函数调用、Select 等计算才会失去 Fast Path	UE 文档《动画优化》
9	“保持 Fast Path，才允许被丢进工作线程无锁并发”	**两个概念混在一起**	Fast Path 解决的是“引脚取值不走蓝图虚拟机”；多线程更新是另一个开关。官方文档还指出：需要 Root Motion 驱动角色移动时，Update 阶段无法并行（角色移动不是多线程的）	UE 文档、第 6 章
10	“资深开发会重写 NativeUpdateAnimation，彻底废弃 EventGraph 连线”	**不够新**	`NativeUpdateAnimation` 仍然在游戏线程。UE5 推荐把计算放进 `NativeThreadSafeUpdateAnimation` / Blueprint Thread Safe Update Animation，用 Property Access 读数据	第 6 章
11	Proxy 双缓冲，“全程无锁操作”	**说过头了**	Proxy 的作用是让工作线程只碰拷贝出来的数据，避免数据竞争；游戏线程仍要在同步点等待任务完成（需要骨骼结果时会阻塞），并不是“没有同步”	第 6 章
12	虚拟骨骼“记录参考姿态下的相对变换”，“在求值最后一步执行偏移计算”	**机制不准确**	官方文档：虚拟骨骼是**源（父）骨骼的子骨骼，但变换匹配目标骨骼**，随动画逐帧变化，不是参考姿势下的固定偏移。它不参与蒙皮	UE 文档《虚拟骨骼》
13	用 Socket 做 IK 目标“往往会拿到上一帧数据”，所以必须用虚拟骨骼	**要分情况**	在 AnimGraph 里，Two Bone IK 等节点的目标可以直接选骨骼或 Socket，用的是当前正在计算的姿势，没有延迟；一帧延迟出现在“游戏线程里调用 GetSocketTransform 再把结果传给动画蓝图”时。虚拟骨骼的真正价值是表达**两根骨骼之间**的相对关系，并且不用改 DCC 骨架	第 1、8 章
14	顶点用 float4 存 4 个权重、uint4 存 4 个骨骼索引；最多 4	**细节不对**	引擎实际用 8 位或 16 位整数压缩存权重和索引；每顶点最大影响数可按 LOD 配置，	第 2 章

	根是硬件限制		“4 根”是很多项目的美术规范和性能取舍，不是硬限制	
15	“只要在 Server 端调用 PlayAnimMontage……就能同步当前播放位置和速率”	**机制说错了**	`PlayAnimMontage` 本身只在本地播放。ACharacter 只为 **Root Motion 蒙太奇**向模拟代理复制状态（RepRootMotion）；普通蒙太奇要靠多播 RPC，或 GAS 的 RepAnimMontageInfo 同步	第 7、10 章
16	串联 3 个 Layered Blend per Bone “底层要凭空多算 3 遍骨架矩阵”	**说过头了**	分层混合节点本身只是逐骨骼插值（E4：68 骨 0.26 us）；开销来自它的每个输入分支各自求值。同一子图用 Cached Pose 只算一次。合并 Slot、减少无用分支是对的，但“不能串联”不对	E4、第 5 章
17	Sync Marker：“强行拉伸或压缩其他跟随动画的播放速率”	**说法不精确**	同步组里权重最高者当 Leader 推进时间，Follower 不自己推进，而是被直接放到与 Leader 对应的位置（归一化位置，或同一对标记之间相同的比例）。混合空间内部的采样本来就自动同步	E6、第 4 章
18	2D 混合空间斜向时“必然”导致双腿麻花交叉、穿模	**过于绝对**	采样同步、方向动画质量好时斜向混合是可用的（大量项目在用）。朝向调整的主要好处是用更少的动画覆盖所有方向，并避开交叉步问题	第 4、8 章
19	惯性化“是 3A 项目优化同屏动画性能的必选方案”	**说过头了**	它适合短过渡（0.1~0.3 s）；源动画在过渡开始后的 Notify 不再触发，频繁打断会劣化。UE 5.3 的 Dead Blending 是实验性替代方案	E7、第 7 章
20	动画压缩“剔除所有 Scale 轨道，瞬间节省 30% 以上”	**不精确**	压缩会去掉所有**常量轨道**（不只是缩放：大部分骨头的平移也是常量，就是骨长）。E9 中这一步就把体积降到 1/3.37；再加量化可到 8.60 倍	E9、第 2 章

A.3 面试时怎么用这份对照

- **少说“绝对”“唯一”“必选”**：原笔记里“绝对不行”“唯一正确解”“必选方案”这类词，面试官一追问反例就站不住。换成“通常”“在 XX 场景下更合适”，再给出原因。
- **具体数字要么来自实测，要么不说**：“省 30%”这种数字没有出处时，宁可说“省多少要用 Insights 测”。本书的数字都能在 `demos/run_output.txt` 里找到来源。
- **把“机制”说对比把“名词”说全更重要**：比如 Blend Depth 的公式、虚拟骨骼挂在谁下面、PlayAnimMontage 会不会自动同步——这些一旦说错，前面讲得再多也会被扣分。
